Formal Verification Report: Lido Lend

- Date: September 15th, 2026
- Author: AlexZoid (Cyfrin private formal verification engagement)
- Certora Prover version: 8.16.2

Table of Contents

1. Verification Scope
2. Methodology
    - Setup Scenes
    - Types of Properties
    - Types of Assumptions
3. Properties
    - Narrowed Scenes
    - GuardianSignals
    - LidoLendMarketController
    - LidoLend
    - LidoLendERC4626
    - LidoLend_LinkedController
    - LidoLendERC4626_LinkedCore
    - Erc4626ExitBook_LinkedVaultCore
    - MorphoExitBook_LinkedCore
    - AdaptiveCurveIrm
    - LinearKinkIrm
    - IrmRouter
    - IrmConfigFactory
4. Mutation Testing
5. Reproducing the Results
    - Prerequisites
    - Remote Execution
    - Local Execution
    - Running the Suites
6. Resources

Verification Scope

Lido Lend is a Morpho Blue fork for isolated, permissionless loop markets. What separates it from the original is a per-market policy layer: a controller hook every core operation must pass, a guardian committee that can slow or block a borrower, and a par-value exit that lets a supplier leave against borrower debt when free liquidity cannot cover the withdrawal. The scope spans three codebases, each read at the commit the follow-up audit reviewed:

Repository	Commit
lido-lend-markets	`d70a1701`
lido-lend	`490f9d7b`
lido-lend-irm	`998de75d`

The contracts in scope:

Forked core (`lido-lend/src/`)

1. **LidoLend** (`LidoLend.sol`): the forked Morpho Blue core, which can bind a market to a controller at creation and adds the par-value `settle` exit, burning a supplier's shares to cancel borrower debt and seize the matching collateral.

Markets layer (`lido-lend-markets/src/`)

2. **GuardianSignals** (`GuardianSignals.sol`): the guardian registry, a committee shared across markets whose members flag accounts to block their borrowing and suspect them to raise their collateral activation delay.
3. **LidoLendMarketController** (`controllers/LidoLendMarketController.sol`): the per-market policy controller the core calls on every supply, withdraw, borrow and settle. It caps supply, holds new collateral until it matures, turns away banned borrowers and orders borrowers by risk, so settlement meets the riskiest first.
4. **LidoLendERC4626** (`LidoLendERC4626.sol`): the lender vault, a share token over one core market that supplies deposits into the core, takes its deposit and withdrawal limits from the controller, and settles against borrower debt when liquidity runs short.
5. **Erc4626ExitBook** (`periphery/Erc4626ExitBook.sol`): the ERC-4626 exit book, where a lender posts an exit request backed by its vault shares and a filler redeems those shares to pay it, over the vaults governance allows.
6. **MorphoExitBook** (`periphery/MorphoExitBook.sol`): the Morpho exit book, the same order book backed by a lender's supply position instead of vault shares, filled by withdrawing from it on the markets governance allows.

Rate models (`lido-lend-irm/src/`)

7. **AdaptiveCurveIrm** (`adaptive-curve-irm/AdaptiveCurveIrm.sol`): the adaptive rate model, pricing each market's borrow rate from utilization and steering its rate at target through an exponential adaptation bounded by a per-config band.
8. **LinearKinkIrm** (`linear-kink-irm/LinearKinkIrm.sol`): the linear kink rate model, quoting a rate that climbs one slope below the optimal utilization and a steeper one above it, capped by a protocol-wide ceiling.
9. **IrmRouter** (`irm-router/IrmRouter.sol`): the routing facade the core prices through, dispatching each market's rate query to the model configured for that market behind a one-way initialization latch.
10. **IrmConfigFactory** (`irm-config-factory/IrmConfigFactory.sol`): the deduplicating factory of immutable rate model configs, deploying a fresh instance for new parameters and returning the registered one whenever the same parameters come back.

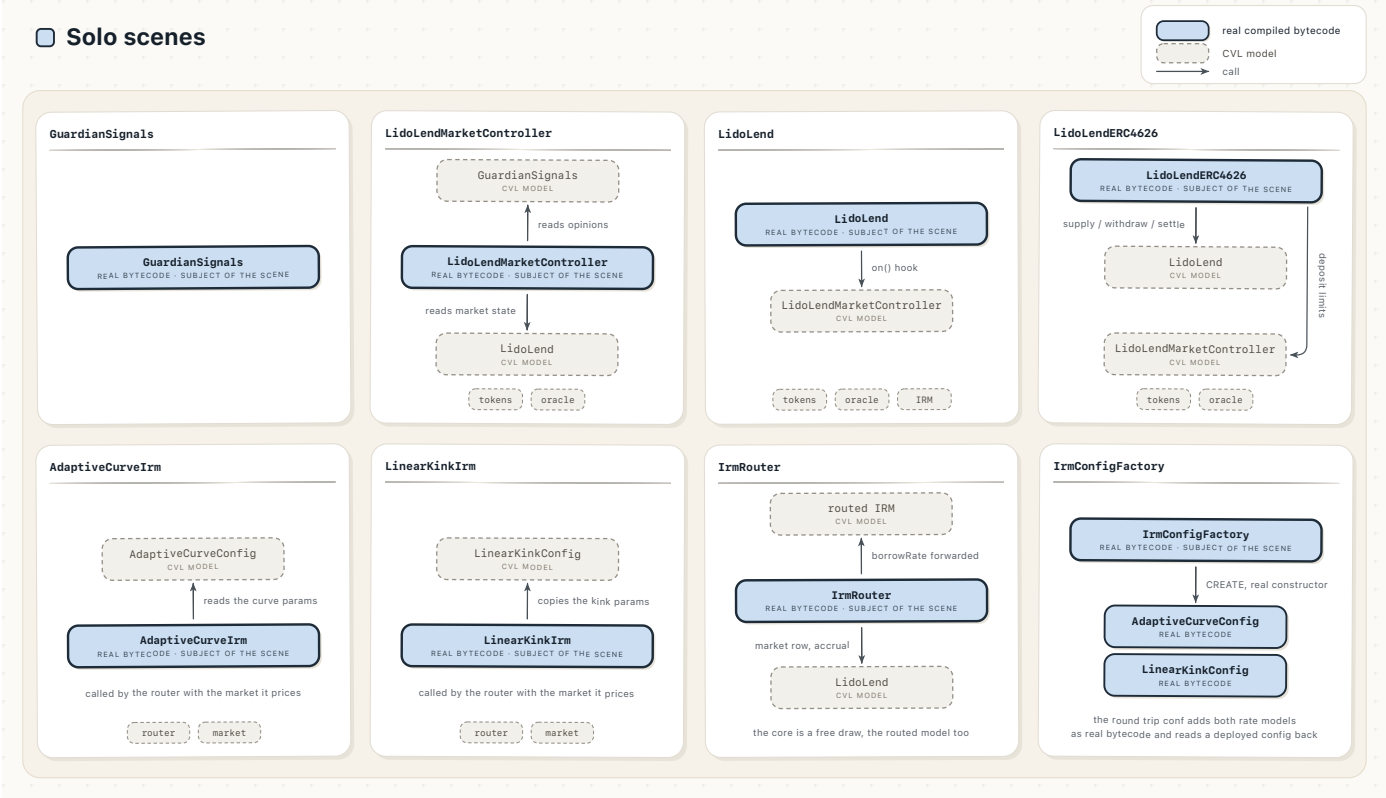
Methodology

Certora Formal Verification (FV) provides mathematical proofs of smart contract correctness by verifying code against a formal specification. Unlike testing and fuzzing which examine specific execution paths, Certora FV examines all possible states and execution paths.

The process involves crafting properties in CVL (Certora Verification Language) and submitting them alongside compiled Solidity smart contracts to the prover. The prover transforms the contract bytecode and rules into a mathematical model and determines the validity of rules.

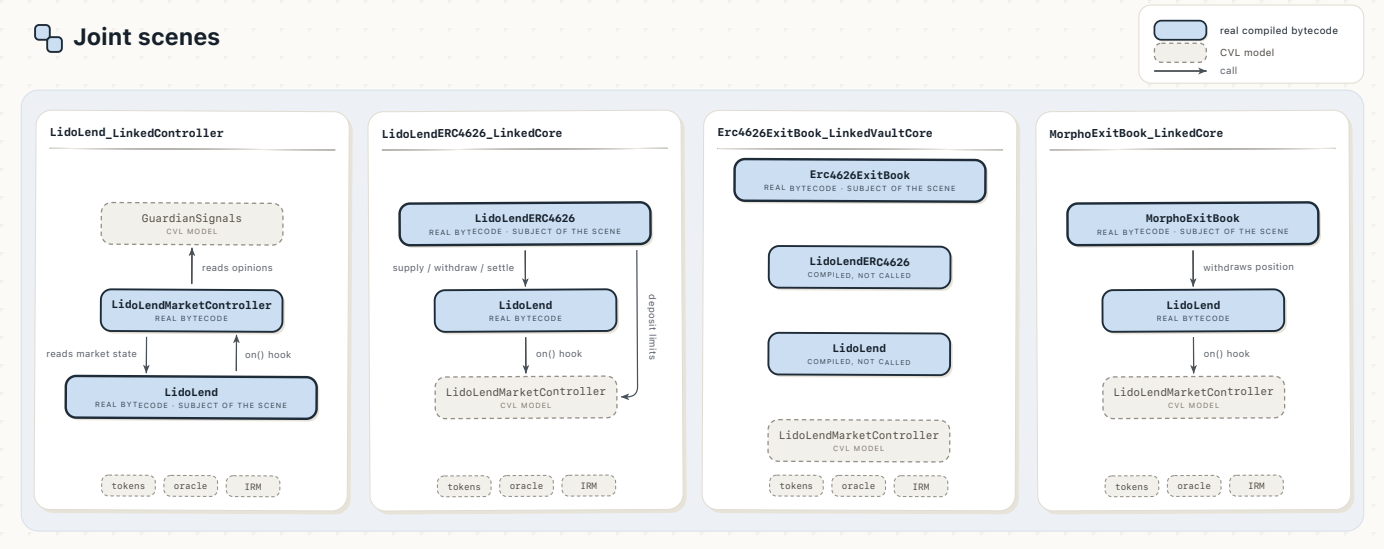
Setup Scenes

A scene is the set of contracts the prover compiles for one verification run. Each is **real** bytecode or a **CVL model**, a stand-in written in the specification language.



A **solo** scene verifies one target alone. Its **valid state**, a pack of invariants stated there, is assumed through `requireInvariant` by the rules that need it.

A property spanning two contracts runs on a joint scene `A_LinkedB`, with its own section below. The two exit books have no solo scene, so every property of theirs runs on its joint scene. `LidoLend_LinkedController` and `LidoLendERC4626_LinkedCore` run both contracts' real bytecode and assume both valid states. `MorphoExitBook_LinkedCore` runs the real core but assumes none of its invariants. `Erc4626ExitBook_LinkedVaultCore` compiles the vault and the core for their state only: the book's calls to a vault resolve into the shared ERC4626 model.



The setup also narrows: one symbolic market where a scene has that dimension, bounded sets where a property sums (position holders, token ledgers, guardian sets), every loop unrolled to a fixed count. Each scene below lists its own bounds.

Types of Properties

The official Certora methodology defines several property categories. A **parametric** category is checked against every external function of the scene, including ones added after the specification.

- **Valid State** (parametric): system-wide invariants that hold in every reachable state; once proven, other properties reuse them as preconditions through `requireInvariant`.
- **Variable Transitions** (parametric): the permitted before/after relation on a single state variable across any call: monotonic counters, write-once fields, values that only move within a bound.
- **State Transitions** (parametric): the moves between valid states, how a valid state may change and in what order.
- **High Level**: properties of one or several specific function calls (share conservation, a supply and withdraw round trip).
- **Unit Tests**: the exact effect of one named call on the state it writes.
- **Accrue Interest**: an entry point behaves the same whether or not interest was accrued earlier in the block.
- **ERC4626**: EIP-4626 conformance of the vault's previews, limits and conversions.
- **Config Trust Boundary**: `satisfy` runs showing what a rate model's trust in its config holds back.
- **Reachability**: `satisfy` witnesses that each workflow can execute and take effect.
- **Reverts**: an external call reverts exactly on its documented preconditions.
- **Access Control** (parametric): every privileged entry point enforces its owner or role gate.
- **Reentrancy**: what a contract may do while another contract holds control: no storage touched after an untrusted call, no delegatecall, no hooked action while a hook runs.
- **Oracle Outage**: which calls never read the market's oracle, and which can still complete or always fail while the oracle reverts.

Types of Assumptions

Each `require` limits which states the prover checks, and its tag says why.

- **SAFE**: excludes no reachable state (`msg.sender != 0`).
- **SCOPE**: pins which flow the rule reasons about; a sibling rule covers what it excludes, or that part is immaterial (a bystander cell, when the cell acted on has its own rule).

- **UNSAFE**: drops reachable, uncovered states, for tractability or a tool limit (the guardian committee bounded to a fixed size).
- **TRUSTED**: guaranteed by a trusted party outside the scene (the initialized proxy state, established at deployment).
- **PROVED**: not an assumption: a fact proven elsewhere, reused as a precondition (an invariant via `requireInvariant`).
- **ASSERT**: not an assumption: inside a CVL model, a `require` standing in for a revert of the real code (the `AccessControlUnauthorizedAccount` check in the role model).

Properties

Every verdict below comes from a locally built prover, the version named in the header.

- **Properties**: every invariant and rule the suite states
- **Proved** ✅: verified by the prover
- **Timeout** ⏱️: the prover ran the rule and did not decide it within the solver budget its conf carries
- **Mutations** 🎯: rules with a mutant the prover killed, a deliberately faulted copy of the code on which the rule fails
- **Violations** ❌: rules that fail
- **Narrowed** 🔻: after a verdict or a 🎯, the result was taken in a narrowed scene

Contract	Properties	Proved ✅	Timeout ⏱️	Mutations 🎯	Violations ❌
GuardianSignals	206	206	0	206	0
LidoLendMarketController	198	194	4	198	0
LidoLend	206	190	16	206	0
LidoLendERC4626	120	113	7	120	0
LidoLend_LinkedController	48	40	8	48	0
LidoLendERC4626_LinkedCore	20	20	0	20	0
Erc4626ExitBook_LinkedVaultCore	117	117	0	117	0
MorphoExitBook_LinkedCore	145	144	1	145	0
AdaptiveCurveIrm	48	45	3	48	0
LinearKinkIrm	40	40	0	40	0
IrmRouter	24	24	0	24	0
IrmConfigFactory	28	28	0	28	0
Total	**1200**	**1161**	**39**	**1200**	**0**

Narrowed Scenes

Some verdicts and kills are taken in a lighter conf beside the rule's own, which simplifies the scene or restates the rule for the prover. The head of that conf's spec names what it simplifies and any pin or restatement it adds.

GuardianSignals

- Sets holding up to 3 of the contract's 32
- Loops running up to 3 iterations

Valid State

Properties	Status	Mutations
GS-VS-01 `suspectStampImpliesMembership`		

A guardian's report time is stored only while its suspect report stands.	✅	🎯
GS-VS-02 `membershipImpliesSuspectStamp` Every live suspect report on an account carries the time it was filed.	✅	🎯
GS-VS-03 `heartbeatNotInFuture` A guardian's last scanned time never runs ahead of the current time.	✅	🎯
GS-VS-04 `suspectedAtNotInFuture` A guardian's suspect report is never dated in the future.	✅	🎯
GS-VS-05 `quorumBoundedByCommittee` The ban quorum never requires more votes than the committee has guardians.	✅	🎯
GS-VS-06 `suspectWindowBounded` The suspect window never exceeds the contract's hard cap of 30 days.	✅	🎯
GS-VS-07 `suspectSnapshotNotInFuture` The report date recorded for an account is never in the future.	✅	🎯
GS-VS-08 `suspectCountBoundedByCommittee` An account's suspect count never exceeds the modeled size of the guardian committee.	✅	🎯
GS-VS-09 `suspectCountAnchoredBySnapshot` An account with a live suspect count always carries a report date.	✅	🎯
GS-VS-10 `emptyCountImpliesZeroSnapshot` An account with no suspect count carries no report date either.	✅	🎯
GS-VS-11 `suspectTimestampBoundedBySnapshot` While an account's suspect count stands, no report on it is dated after the recorded date.	✅	🎯🎯
GS-VS-12 `quorumAtLeastOne` Once the contract is initialized, the ban quorum is at least one guardian.	✅	🎯
GS-VS-13 `frontierNotInFuture` The guardian scan frontier never runs ahead of the current block time.	✅	🎯
GS-VS-14 `suspectCountWithinLiveSet` The stored suspect tally never counts more reports than the account actually carries.	✅	🎯
GS-VS-15 `onlyASeatHoldsADelegate` Only an account that holds a committee seat carries an appointed hot key.	✅	🎯

Variable Transitions

Properties	Status	Mutations
GS-VT-01 `vtGsHeartbeatRisesOrClears` A guardian's last scan time only moves forward or resets to zero.	✅	🎯🎯
GS-VT-02 `vtGsCommitteeSizeStepsByOne` The committee gains or loses at most one member per call.	✅	🎯
GS-VT-03 `vtGsSuspectStampRisesOrClears` A guardian's suspect report time only moves forward or resets to zero.	✅	🎯
GS-VT-04 `vtGsFlaggerSetGrowsByOne`		

An account's strike list gains at most one guardian per call.	✅	🎯
GS-VT-05 `vtGsSuspectSetGrowsByOne` An account's suspect list gains at most one guardian per call.	✅	🎯
GS-VT-06 `quorumFrontierNeverRegresses` The guardian scan frontier only ever moves forward, never back to an earlier time.	✅	🎯
GS-VT-07 `quorumFrontierMovesOnlyInSync` Only the quorum scan changes the guardian frontier, whether it is called directly or reached through a heartbeat.	✅	🎯
GS-VT-08 `vtGsDelegateNeverBecomesASeat` A call never leaves a seated guardian's hot key sitting on the committee itself.	✅	🎯

State Transitions

Properties	Status	Mutations
GS-ST-01 `stGsSnapshotRestamped` When an account's suspect tally changes, its date is cleared or set to a report still in force.	✅	🎯 🎯
GS-ST-02 `stGsFlagLatchNeedsQuorum` A ban latched by a new strike fires only once the strikes reach quorum.	✅	🎯 🎯
GS-ST-03 `stGsHeartbeatSeatOnly` Only a sitting committee member's scan time can move.	✅	🎯
GS-ST-04 `stGsSuspectAddStampsSeat` A guardian joining an account's suspect set is stamped with the current time.	✅	🎯

High Level

Properties	Status	Mutations
GS-HL-01 `hlGsFlagLeavesOtherAccounts` A guardian flagging one account leaves every other account's strike count untouched.	✅	🎯
GS-HL-02 `hlGsUnbanLeavesOtherAccounts` An unban leaves every other account's ban exactly as it was.	✅	🎯
GS-HL-03 `hlGsPruneKeepsBan` The cleanup never lifts a latched ban and latches one only when the surviving strikes reach quorum.	✅	🎯
GS-HL-04 `hlGsUnflagKeepsBan` A guardian retracting a strike never lifts a ban and imposes one only when the strikes already reached the quorum.	✅	🎯
GS-HL-05 `hlGsBanKeepsFlaggers` A governance ban leaves the guardians' existing strikes on the account untouched.	✅	🎯
GS-HL-06 `hlGsPruneEvictsExGuardianFlag` The cleanup drops strikes cast by anyone no longer on the guardian committee.	✅	🎯
GS-HL-07 `hlGsPruneEvictsExGuardianSuspect` The cleanup drops suspect reports filed by anyone no longer on the guardian committee.	✅	🎯 🎯

GS-HL-08 `hlGsFlagUnflagRoundTrip` A guardian flagging then retracting leaves the account's strike count as it was.	✅	🎯
GS-HL-09 `hlGsRemoveGuardianLeavesOtherMembers` Removing one guardian leaves every other committee member in place.	✅	🎯
GS-HL-10 `hlGsAddGuardianKeepsStrikes` Adding a guardian changes no account's strike count and leaves the ban threshold where it was.	✅	🎯
GS-HL-11 `hlGsRemoveGuardianKeepsStrikes` Removing a guardian changes no account's strike count and leaves the ban threshold where it was.	✅	🎯
GS-HL-12 `hlGsRotationKeepsBan` Adding a guardian to the committee leaves every account's ban exactly as it was.	✅	🎯
GS-HL-13 `hlGsBanLiftedOnlyByUnban` A latched ban survives every call except an unban.	✅	🎯
GS-HL-14 `hlGsPruneNeverWorsensOpinion` The permissionless cleanup can only relieve an account: never more strikes, no new ban, no later expiry.	✅	🎯
GS-HL-15 `hlGsRepeatFlagLatchesNoBan` A guardian flagging the same account twice leaves its ban exactly as it was.	✅	🎯
GS-HL-16 `hlGsFirstSuspectReportCountsOne` The first suspect report on an account counts as exactly one live suspicion.	✅	🎯🎯
GS-HL-17 `hlGsSoleReporterDrainsToZero` When the only reporting guardian retracts, the account's suspicion clears completely.	✅	🎯
GS-HL-18 `hlGsExpiredReportNotCounted` A new suspect report sweeps out a lapsed one, which never reaches the tally.	✅	🎯
GS-HL-19 `hlGsBoundaryReportStillCounted` A suspect report sitting exactly at the window edge still counts.	✅	🎯🎯🎯
GS-HL-20 `hlGsRepeatSuspectRefreshesStamp` A guardian reporting the same account again refreshes its report without counting twice.	✅	🎯
GS-HL-21 `hlGsSetQuorumBanCharacterization` Changing the quorum rewrites no signals and scores existing strikes against the new threshold.	✅	🎯🎯
GS-HL-22 `hlGsFlagKeepsSoftChannel` A guardian's strike never touches the suspect reports standing against the same account.	✅	🎯
GS-HL-23 `hlGsUnflagKeepsSoftChannel` Retracting a strike never touches the suspect reports standing against the same account.	✅	🎯
GS-HL-24 `hlGsSuspectKeepsHardChannel` A suspect report never adds a strike and never bans the account it names.	✅	🎯
GS-HL-25 `hlGsUnsuspectKeepsHardChannel` Retracting a suspect report never changes an account's strikes or its ban.	✅	🎯

GS-HL-26 `hlGsUnbanEndsEffectiveBan` After an unban the account no longer counts as banned, whatever strikes stood against it.	✅	🎯
GS-HL-27 `hlGsBatchFlagFlagsBothTargets` One batch call lands the guardian's strike on every account listed.	✅	🎯
GS-HL-28 `hlGsBatchDuplicateFlagCountsOnce` Listing the same account twice in one call still lands a single strike.	✅	🎯
GS-HL-29 `hlGsSetSuspectWindowExpiryCharacterization` Changing the suspect window rewrites no signals and re-dates every expiry from the stored report time.	✅	🎯
GS-HL-30 `hlGsRemoveGuardianKeepsEffectiveBan` Removing a guardian moves no account's ban, so the strikes that guardian already cast keep counting.	✅	🎯
GS-HL-31 `hlGsStaleSuspectCountServedAfterExpiry` A suspect report that has lapsed on its own still counts in the account's reported suspicion.	✅	🎯
GS-HL-32 `hlGsRetractionAnchorsToSurvivor` When one of two reporters retracts, the account's expiry follows the surviving report.	✅	🎯
GS-HL-33 `hlGsFreshReportExtendsSurvivorExpiry` Another guardian's fresh report extends the expiry an older live report is scored by.	✅	🎯
GS-HL-34 `quorumRaiseKeepsStaleFrontier` Raising the quorum leaves the frontier where the smaller quorum had already set it.	✅	🎯
GS-HL-35 `hlGsPruneNeverLiftsLatchedBan` The cleanup ends a ban only by evicting the strikes behind it, never by clearing a latch.	✅	🎯
GS-HL-36 `hlGsRetractionNeverExtendsExpiry` Retracting a suspect report never pushes an account's suspicion expiry later.	✅	🎯

Unit Tests

Properties	Status	Mutations
GS-UT-01 `utGsSetQuorumStoresValue` Setting the ban quorum stores exactly the number governance asked for.	✅	🎯
GS-UT-02 `utGsSetQuorumKeepsCommitteeSize` Setting the ban quorum leaves the committee the same size.	✅	🎯
GS-UT-03 `utGsSetSuspectWindowStoresValue` Setting the suspect window stores exactly the length governance asked for.	✅	🎯
GS-UT-04 `utGsAddGuardianMakesMember` Adding a guardian seats it on the committee.	✅	🎯
GS-UT-05 `utGsAddGuardianGrowsCommitteeByOne` Adding a guardian who is not yet seated grows the committee by exactly one.	✅	🎯
GS-UT-06 `utGsRemoveGuardianDropsMember` Removing a guardian takes it off the committee.	✅	🎯
GS-UT-07 `utGsRemoveGuardianShrinksCommitteeByOne`	✅	🎯

Removing a seated guardian shrinks the committee by exactly one.		
GS-UT-08 `utGsHeartbeatStoresReportedTimestamp` A guardian's heartbeat stores the scan time the guardian reports, not the current time.	✅	🎯
GS-UT-09 `utGsBanSetsLatch` A ban by governance latches on the named account in the named market.	✅	🎯
GS-UT-10 `utGsBanLeavesOtherAccounts` Banning one account leaves the ban status of every other account and market unchanged.	✅	🎯
GS-UT-11 `utGsUnbanClearsLatch` An unban clears the stored ban on the account in the named market.	✅	🎯
GS-UT-12 `utGsUnbanEmptiesFlaggerSet` An unban also clears every strike standing against the account.	✅	🎯🎯
GS-UT-13 `utGsOpinionBannedIsEffectiveBan` The guardians' opinion reports an account banned exactly when a ban is latched or its strikes reach quorum.	✅	🎯🎯
GS-UT-14 `utGsOpinionStrikeCountIsFlaggerSetSize` The reported strike count is the number of guardians currently flagging the account.	✅	🎯🎯
GS-UT-15 `utGsOpinionSuspectCountIsSnapshot` The reported suspect count is the stored snapshot, not a fresh scan of the reports.	✅	🎯🎯
GS-UT-16 `utGsOpinionExpiryIsSnapshotPlusWindow` The reported expiry of a suspicion is its snapshot time plus the window in force when read.	✅	🎯
GS-UT-17 `utGsOpinionFrontierIsStored` Every opinion carries the committee's current scan frontier timestamp.	✅	🎯
GS-UT-18 `utGsRemoveGuardianKeepsFlagMembership` Only flagging, retracting, the cleanup or an unban can move who flags an account.	✅	🎯
GS-UT-19 `utGsRemoveGuardianKeepsSuspectMembership` Only a suspect report, its retraction or the cleanup can move who suspects an account.	✅	🎯
GS-UT-20 `utGsRemoveGuardianKeepsBanLatch` Removing a guardian moves neither an account's ban nor the quorum bans are scored against.	✅	🎯
GS-UT-21 `utGsRemoveGuardianClearsOwnHeartbeat` Removing a guardian erases only that guardian's last scan time and delegate.	✅	🎯🎯
GS-UT-22 `utGsRemoveGuardianKeepsFrontier` Removing a guardian leaves the committee's scan frontier exactly where it stood.	✅	🎯
GS-UT-23 `utGsAddGuardianKeepsSuspectSnapshot` Only a suspect report, its retraction or the cleanup changes an account's suspicion snapshot and its date.	✅	🎯
GS-UT-24 `utGsAddGuardianKeepsFrontier` Adding a guardian leaves the committee's scan frontier exactly where it stood.	✅	🎯
GS-UT-25 `utGsFlagInsertsCaller` A strike is recorded under the guardian that raised it.	✅	🎯

GS-UT-26 `utGsUnflagRemovesCallerFlag` A guardian retracting a strike leaves no strike of its own on the account.	✅	🎯
GS-UT-27 `utGsUnflagKeepsOtherFlaggers` One guardian's retraction leaves every other guardian's strike on the account standing.	✅	🎯
GS-UT-28 `utGsUnflagDropsExactlyCallerStrike` A retraction lowers the strike count by exactly one when that guardian had a strike, otherwise not at all.	✅	🎯
GS-UT-29 `utGsSuspectInsertsCaller` A suspect report is filed under the guardian who raised it.	✅	🎯
GS-UT-30 `utGsSuspectStampsCallerNow` A suspect report is stamped with the time it is filed, refreshing a repeat report.	✅	🎯
GS-UT-31 `utGsSuspectLeavesSiblingSets` Reporting one account leaves the suspicions standing against every other account and market untouched.	✅	🎯
GS-UT-32 `utGsUnsuspectRemovesCallerReport` A guardian withdrawing its suspicion leaves no report of its own on the account.	✅	🎯
GS-UT-33 `utGsUnsuspectClearsCallerStamp` Withdrawing a suspicion clears the time stamp that report carried.	✅	🎯
GS-UT-34 `utGsPruneKeepsLiveGuardianFlag` The permissionless cleanup keeps the strikes of guardians still seated on the committee.	✅	🎯
GS-UT-35 `utGsPruneKeepsLiveGuardianSuspect` The permissionless cleanup keeps a seated guardian's suspect report, whether or not it has lapsed.	✅	🎯
GS-UT-36 `utGsHeartbeatsRowPerMember` The heartbeat listing has one row per committee member, with the two lists the same length.	✅	🎯
GS-UT-37 `utGsHeartbeatsValueMatchesRecord` Each timestamp in the heartbeat listing is that guardian's own recorded scan time.	✅	🎯
GS-UT-38 `utGsFlagNeverShrinksFlaggers` Flagging never removes a strike from any account.	✅	🎯
GS-UT-39 `utGsUnflagNeverGrowsFlaggers` Retracting a strike never adds one to any account.	✅	🎯
GS-UT-40 `utGsUnsuspectNeverGrowsSuspectSet` Withdrawing a suspicion never adds a suspect report to any account.	✅	🎯
GS-UT-41 `utGsPruneShrinkOnly` The permissionless cleanup can only remove strikes and suspect reports, never add them.	✅	🎯
GS-UT-42 `utGsGetQuorumIsStored` The reported ban quorum is exactly the threshold governance stored.	✅	🎯
GS-UT-43 `utGsGetSuspectWindowIsStored` The reported suspect window is exactly the window governance stored.	✅	🎯
GS-UT-44 `utGsGetQuorumFrontierIsStored`		

The reported scan frontier is exactly the one the contract recorded.	✅	🎯
GS-UT-45 `utGsGetGuardianCountIsCommitteeSize` The reported guardian count is the true size of the committee.	✅	🎯
GS-UT-46 `utGsGetHeartbeatIsGuardianStamp` A reported heartbeat belongs to the guardian asked about, not to the caller.	✅	🎯
GS-UT-47 `utGsIsGuardianIsMembership` The membership check answers for the account asked about, not for the caller.	✅	🎯
GS-UT-48 `utGsIsBannedIsEffectiveBan` The ban check answers yes exactly when a ban is latched or the strikes reach quorum.	✅	🎯🎯🎯
GS-UT-49 `utGsFlaggedByRowPerFlagger` The strike listing returns as many addresses as there are guardians flagging the account.	✅	🎯
GS-UT-50 `utGsSuspectedByRowPerReporter` The suspect listing returns as many guardians and report times as there are live reports.	✅	🎯
GS-UT-51 `utGsSuspectedByStampMatchesRecord` Each listed report time is the one recorded for the guardian beside it.	✅	🎯
GS-UT-52 `syncBelowQuorumLeavesFrontierUntouched` A scan leaves the frontier alone when no quorum is set or the committee is smaller than the quorum.	✅	🎯
GS-UT-53 `frontierAdvanceIsBackedByQuorum` The frontier advances only as far as a quorum of guardians has already scanned.	✅	🎯
GS-UT-54 `frontierAdvanceIsNotAboveQuorumthHighest` After an advance, fewer than a quorum of guardians have scanned past the new frontier.	✅	🎯
GS-UT-55 `syncKeepsFrontierOnlyWhenItAlreadyDominatesTheCommittee` When the committee can reach quorum, a scan leaves the frontier alone only if fewer than a quorum have passed it.	✅	🎯
GS-UT-56 `utGsSetQuorumKeepsFrontier` Changing the quorum never moves the committee's scan frontier.	✅	🎯
GS-UT-57 `utGsSetDelegateStoresValue` Appointing a delegate files it under the appointing guardian and under no other.	✅	🎯
GS-UT-58 `utGsSetDelegateKeepsSeatStanding` Appointing a delegate costs a guardian neither its committee seat nor its last scan time.	✅	🎯
GS-UT-59 `utGsGetDelegateIsStoredDelegate` The delegate reported for a guardian is that guardian's own, not the caller's.	✅	🎯
GS-UT-60 `utGsReplaceGuardianSwapsSeats` Replacing a guardian seats the newcomer, unseats the old one, and moves neither count nor quorum.	✅	🎯
GS-UT-61 `utGsHeartbeatWithoutCrossingKeepsFrontier` A heartbeat moves the frontier only when it carries its own seat past it.	✅	🎯
GS-UT-62 `utGsHeartbeatMovesOnlyInBeatOrRotation` A guardian's last scan time moves only on a heartbeat and clears only when the seat	✅	🎯🎯

leaves.		
GS-UT-63 `utGsReplaceGuardianClearsOutgoingSeat` Replacing a guardian clears the outgoing seat's delegate and last scan time.	✅	🎯
GS-UT-64 `utGsDelegateRecordsUnderSeat` A delegate acting for a guardian records under that guardian, never under itself.	✅	🎯

Reachability

Properties	Status	Mutations
GS-RC-01 `rcGsAddGuardianIsReachable` A manager can add a new guardian and the committee grows.	✅	🎯
GS-RC-02 `rcGsRemoveGuardianIsReachable` A manager can remove a guardian while the committee stays at or above quorum.	✅	🎯
GS-RC-03 `rcGsHeartbeatIsReachable` A guardian can report a heartbeat that advances its last scanned time.	✅	🎯
GS-RC-04 `rcGsUnflagIsReachable` A guardian can retract its strike and leave the account's strike list.	✅	🎯
GS-RC-05 `rcGsUnsuspectIsReachable` A guardian can retract its suspect report, clearing the time it was filed.	✅	🎯
GS-RC-06 `rcGsPruneIsReachable` Anyone can prune a former guardian's stale strike from an account.	✅	🎯
GS-RC-07 `rcGsBanIsReachable` An access limiter can force a ban on an account.	✅	🎯
GS-RC-08 `rcGsRemoveGuardianDownToQuorumIsReachable` A manager can remove a guardian, leaving the committee exactly at quorum.	✅	🎯
GS-RC-09 `rcGsFlagLatchesBanAtQuorumIsReachable` A strike that brings the count up to quorum can ban the account.	✅	🎯
GS-RC-10 `rcGsUnbanClearsFlaggersIsReachable` An access restorer can lift a ban and clear the strikes behind it.	✅	🎯
GS-RC-11 `rcGsUnflagEmptiesSetIsReachable` The last guardian to retract can clear an account's strikes entirely.	✅	🎯
GS-RC-12 `rcGsPruneDropsStaleSuspectIsReachable` Anyone can prune a former guardian's stale suspect report from an account.	✅	🎯
GS-RC-13 `rcGsSecondDistinctFlaggerIsReachable` A second guardian can add its own strike to the same account.	✅	🎯
GS-RC-14 `rcGsSecondDistinctSuspecterIsReachable` A second guardian can file its own suspect report on the same account.	✅	🎯
GS-RC-15 `rcGsAddThenHeartbeatIsReachable` A newly seated guardian can report a heartbeat in the same block.	✅	🎯
GS-RC-16 `rcGsFlagThenUnflagRoundTripIsReachable`	✅	🎯

A guardian can flag an account and then retract, restoring the count.		
GS-RC-17 `rcGsQuorumLoweringRetroBansIsReachable` Lowering the quorum can ban an account with no new flag.	✅	🎯
GS-RC-18 `rcGsQuorumRaiseUnbansUnlatchedIsReachable` Raising the quorum can lift a ban that was never written down.	✅	🎯
GS-RC-19 `rcGsUnflagLatchesBeforeDropIsReachable` A retraction can record the ban already in force before the strike count drops.	✅	🎯
GS-RC-20 `rcGsOpinionPopulatedIsReachable` An account can carry one guardian's strike and another guardian's unexpired suspect report.	✅	🎯
GS-RC-21 `rcGsDynamicBanWithoutLatchIsReachable` An account can count as banned on flag count alone, with no ban stored.	✅	🎯
GS-RC-22 `rcGsSuspectThenUnsuspectRoundTripIsReachable` A guardian can file a suspect report and then retract it, restoring the count.	✅	🎯
GS-RC-23 `rcGsRelatchAfterUnbanIsReachable` An unbanned account can be flagged straight back into a ban.	✅	🎯
GS-RC-24 `rcGsWindowCutRetroExpiresIsReachable` Shortening the suspect window alone can expire a live suspicion.	✅	🎯
GS-RC-25 `quorumRaiseStaleFrontierWitness` It is possible for a raised quorum to inherit a frontier only the smaller quorum backed.	✅	🎯
GS-RC-26 `setQuorumStaleFrontierWitness` Right after the quorum is set, a quorum of guardians can already have scanned past the stored frontier.	✅	🎯
GS-RC-27 `opinionFrontierStaleUntilSync` A guardian opinion can report a frontier that a permissionless scan advances in the same block.	✅	🎯
GS-RC-28 `rcGsStaleSuspectCountIsReachable` An account can be shown a suspicion that rests only on a lapsed report.	✅	🎯
GS-RC-29 `rcGsStaleFlagCompletesQuorumIsReachable` A departed guardian's strike can be the one that tips an account into a ban.	✅	🎯
GS-RC-30 `rcGsBanQuorumMetByExGuardiansIsReachable` An account can stay banned on strikes cast only by guardians who have left.	✅	🎯
GS-RC-31 `rcGsHeartbeatUnboundTimestampIsReachable` A guardian's heartbeat can pair a block long past with the current time.	✅	🎯
GS-RC-32 `rcGsReAddClearsHeartbeatIsReachable` A guardian removed and later re-added can come back with a cleared heartbeat.	✅	🎯
GS-RC-33 `rcGsPruneClearsStaleSuspectCountIsReachable` Pruning a former guardian's report can stop it counting toward the delay at once.	✅	🎯
GS-RC-34 `rcGsRetractionCutsExpiryIsReachable` Retracting one of two suspect reports can shorten how long the remaining one counts.	✅	🎯

GS-RC-35 `rcGsPruneLatchesSurvivorQuorumIsReachable` Pruning stale flags can still ban the account on the flags that survive.	✅	🎯
GS-RC-36 `rcGsDelegateActsForSeatIsReachable` A guardian's delegate can send every signal and heartbeat on the guardian's behalf.	✅	🎯

Reverts

Properties	Status	Mutations
GS-RV-01 `rvGsAddGuardianRevertsIff` Governance adding a guardian fails exactly when the address is zero or already sits on the committee.	✅	🎯🎯
GS-RV-02 `rvGsRemoveGuardianRevertsIff` Governance removing a guardian fails exactly when the guardian is not seated or the committee would fall below quorum.	✅	🎯
GS-RV-03 `rvGsSetQuorumRevertsIff` Governance setting the ban quorum fails exactly when the value is zero or above the committee size.	✅	🎯🎯
GS-RV-04 `rvGsSetSuspectWindowRevertsIff` Governance setting the suspect window fails exactly when it exceeds the 30 day cap.	✅	🎯
GS-RV-05 `rvGsHeartbeatRevertsWhenNotGuardian` Only a guardian, or the delegate it appointed, can record a heartbeat for that seat.	✅	🎯
GS-RV-06 `rvGsHeartbeatRevertsOnFutureBlockNumber` A heartbeat naming a block that does not exist yet fails.	✅	🎯
GS-RV-07 `rvGsHeartbeatRevertsOnFutureTimestamp` A heartbeat claiming a time later than the current block fails.	✅	🎯
GS-RV-08 `rvGsHeartbeatRevertsOnRegressedTimestamp` A heartbeat that backdates the named seat's last scan time fails.	✅	🎯
GS-RV-09 `rvGsHeartbeatRevertsOnZeroBlockHash` A heartbeat carrying a zero block hash fails.	✅	🎯
GS-RV-10 `rvGsFlagRevertsWhenNotGuardian` Only a guardian, or the delegate it appointed, can flag an account for that seat.	✅	🎯
GS-RV-11 `rvGsFlagRevertsOnZeroAccount` Flagging the zero address fails, so no strike is recorded against it.	✅	🎯
GS-RV-12 `rvGsUnflagRevertsWhenNotGuardian` Only a guardian, or the delegate it appointed, can withdraw that seat's strike.	✅	🎯
GS-RV-13 `rvGsSuspectRevertsWhenNotGuardian` Only a guardian, or the delegate it appointed, can file a suspect report for that seat.	✅	🎯
GS-RV-14 `rvGsSuspectRevertsOnZeroAccount` Filing a suspect report against the zero address fails.	✅	🎯
GS-RV-15 `rvGsUnsuspectRevertsWhenNotGuardian` Only a guardian, or the delegate it appointed, can retract that seat's suspect report.	✅	🎯

GS-RV-16 `rvGsPruneNeverReverts` The permissionless cleanup of stale flags and reports never fails, whoever calls it.	✅	🎯
GS-RV-17 `rvGsBanRevertsIff` A ban by the access limiter fails exactly when the named account is the zero address.	✅	🎯
GS-RV-18 `rvGsUnbanNeverReverts` Lifting a ban never fails for the restorer, whatever state the account is in.	✅	🎯
GS-RV-19 `rvGsGetGuardiansNeverReverts` Listing the committee together with each member's delegate never fails.	✅	🎯
GS-RV-20 `rvGsGetHeartbeatsNeverReverts` Listing every guardian's last scan time never fails.	✅	🎯
GS-RV-21 `rvGsGetOpinionNeverReverts` Reading the guardians' opinion on an account never fails, whatever reports stand against it.	✅	🎯
GS-RV-22 `rvGsIsBannedNeverReverts` Checking whether an account is banned in a market never fails.	✅	🎯
GS-RV-23 `rvGsGetFlaggedByNeverReverts` Listing the guardians who flagged an account never fails.	✅	🎯
GS-RV-24 `rvGsGetSuspectedByNeverReverts` Listing the guardians who reported an account, with their report times, never fails.	✅	🎯
GS-RV-25 `rvGsHeartbeatRevertsOnAgedOutBlock` A heartbeat naming the current block, or one more than 256 blocks old, fails.	✅	🎯
GS-RV-26 `rvGsSetDelegateRevertsIff` Appointing or revoking a delegate fails exactly when the caller holds no committee seat or the key already sits on the committee.	✅	🎯
GS-RV-27 `rvGsReplaceGuardianRevertsIff` Governance swapping a guardian fails exactly when the incoming address is zero or seated, or the outgoing one is absent.	✅	🎯
GS-RV-28 `rvGsSyncQuorumFrontierNeverReverts` Anyone can run the scan that refreshes the guardian frontier, and the call never fails.	✅	🎯
GS-RV-29 `rvGsSetDelegateRevertsWhenKeyIsTaken` A hot key another guardian already uses cannot be appointed a second time.	✅	🎯
GS-RV-30 `rvGsAddGuardianRevertsWhenCandidateIsAKey` An address that already answers as a seat's hot key cannot be seated as a guardian.	✅	🎯

Access Control

Properties	Status	Mutations
GS-AC-01 `acGsOnlyManagerMovesQuorum` Only a governance manager can change how many guardian flags it takes to ban an account.	✅	🎯
GS-AC-02 `acGsOnlyManagerMovesSuspectWindow` Only a governance manager can change how long a guardian's suspect report keeps counting.	✅	🎯

GS-AC-03 `acGsBanLatchedOnlyByLimiterOrFlagger` An account is banned only by the access limiter or by guardian strikes reaching quorum.	✅	🎯
GS-AC-04 `acGsBanLiftedOnlyByRestorer` Only the access restorer can lift a latched ban.	✅	🎯
GS-AC-05 `acGsOnlyManagerMovesCommitteeMembership` Only a governance manager can add or remove a seat on the guardian committee.	✅	🎯
GS-AC-06 `acGsOnlyGuardianMovesHeartbeat` Only a guardian or its delegate writes that guardian's last scan time, and only a manager clears it.	✅	🎯
GS-AC-07 `acGsFlaggerEntryIsCallerOnly` A strike is recorded only under the seat the caller acts for, never another guardian's.	✅	🎯🎯
GS-AC-08 `acGsSuspectEntryIsCallerOnly` A suspect report is filed only under the seat the caller acts for, never another guardian's.	✅	🎯🎯
GS-AC-09 `acGsSuspectStampWrittenOnlyBySelf` A suspect report's timestamp is set only by the guardian it stands for, or that guardian's delegate.	✅	🎯🎯
GS-AC-10 `roleGrantedOnlyByItsAdmin` Only a role's admin can grant that role.	✅	🎯
GS-AC-11 `roleRevokedOnlyByItsAdminOrItsHolder` Only a role's admin can revoke that role, apart from a holder giving up their own.	✅	🎯
GS-AC-12 `acGsPermissionlessUnbanOnlyByPrune` A caller with no role and no seat can end a ban only through the permissionless cleanup, and never a latched one.	✅	🎯
GS-AC-13 `acGsDelegateRowMovesOnlyForItsSeat` A guardian's delegate key changes only at that guardian's own hand, or clears when its seat leaves the committee.	✅	🎯

LidoLendMarketController

- GuardianSignals linked as a CVL model
- Core totals read without pending interest, the CVL model of the core not accruing
- Settlement queue holding up to 7 borrowers
- The no-call reentrancy row watching state-changing calls only, STATICCALL not watched
- The controller issuing only read-only calls on clean source, so its no-call row is backed by a killed mutant
- Loops running up to 3 iterations

Valid State

Properties	Status	Mutations
MC-VS-01 `heapIndexToSlot` Every borrower in the settlement queue is indexed at exactly the position they occupy.	✅	🎯
MC-VS-02 `heapSlotToAccount`		

The position recorded for a queued borrower really holds that borrower.	✅	🎯
MC-VS-03 `heapSlotBounded` A queued borrower's recorded position always lies inside the queue.	✅	🎯
MC-VS-04 `heapTailKeyZero` No place outside the live settlement queue keeps a leftover risk score.	✅	🎯
MC-VS-05 `heapParentKeyDominatesChild` The settlement queue always holds the borrower with the highest recorded risk at its top.	✅	🎯🎯
MC-VS-06 `activatingRequiresTimerStart` Collateral still waiting to activate always has its activation timer running.	✅	🎯🎯
MC-VS-07 `clearedTimerClearsDelay` Once the activation timer is cleared, no stale waiting period is left behind.	✅	🎯
MC-VS-08 `timerStartNotInFuture` A deposit's activation timer never starts in the future.	✅	🎯
MC-VS-09 `inflowSnapshotNotInFuture` The inflow throttle's last update is never stamped in the future.	✅	🎯
MC-VS-10 `activatingDelayCapped` The wait recorded on a borrower's deposit never exceeds seven days.	✅	🎯
MC-VS-11 `minDelayLeMaxDelay` The activation wait floor never exceeds the activation wait ceiling.	✅	🎯
MC-VS-12 `maxActivationDelayCapped` The activation wait ceiling set by governance never exceeds seven days.	✅	🎯
MC-VS-13 `activatingLockDurationCapped` The withdrawal lock on freshly deposited collateral never lasts more than seven days.	✅	🎯
MC-VS-14 `quorumDelayZeroOrAboveMaxDelay` A guardian quorum fallback delay, when set, always outlasts the activation wait ceiling.	✅	🎯
MC-VS-15 `delayPerStrikeCapped` A single guardian strike never adds more than seven days to a borrower's wait.	✅	🎯
MC-VS-16 `delayPerSuspectCapped` A single suspect report never adds more than seven days to a borrower's wait.	✅	🎯
MC-VS-17 `sliceFloorBpsCapped` The settlement policy's debt fraction is never configured above 100 percent.	✅	🎯
MC-VS-18 `ltvStepBpsCapped` The settlement policy's loan to value step is never configured above 100 percent.	✅	🎯
MC-VS-19 `settlementLtvBelowWad` A borrower's own settlement target always sits below 100 percent loan to value.	✅	🎯
MC-VS-20 `timerStartRequiresActivating` A running activation timer always has collateral waiting inside it.	✅	🎯
MC-VS-21 `timerStartRequiresDelay` A running activation timer always carries a nonzero waiting period.	✅	🎯

MC-VS-22 `maxGuardiansQuorumDelayCapped` The guardian quorum fallback delay never exceeds seven days.	✅	🎯

Variable Transitions

Properties	Status	Mutations
MC-VT-01 `vtMcMarketIdFrozen` The controller stays bound to one market and can never be repointed at another.	✅	🎯
MC-VT-02 `vtMcGuardianRegistryFrozen` The guardian registry the controller consults for bans and strikes can never be replaced.	✅	🎯
MC-VT-03 `vtMcHeapLengthStepsByOne` The settlement queue grows or shrinks by at most one borrower per call, and never loses its reserved first slot.	✅	🎯🎯
MC-VT-04 `vtMcHeapSentinelUntouched` The settlement queue's reserved first slot never changes.	✅	🎯
MC-VT-05 `vtMcInflowClockLandsAtNow` The inflow throttle's decay clock is only ever stamped with the current time.	✅	🎯
MC-VT-06 `vtMcTimerStartClearsOrLandsAtNow` A borrower's activation timer is only ever cleared or started at the current time.	✅	🎯🎯
MC-VT-07 `vtMcResumeStampNeverPast` The time a pause ends is never moved into the past.	✅	🎯
MC-VT-08 `vtMcMinActivationDelayNeverZeroed` The activation wait floor is never set back to zero.	✅	🎯
MC-VT-09 `vtMcMinActivationAmountNeverZeroed` The threshold a deposit must reach to enter activation is never set back to zero.	✅	🎯
MC-VT-10 `vtMcMaxDelayInflowNeverZeroed` The inflow reference the activation wait scales against is never set back to zero.	✅	🎯

State Transitions

Properties	Status	Mutations
MC-ST-01 `stMcActivatingDeadlineMonotone` Adding collateral never pulls a borrower's stored unlock deadline earlier.	✅	🎯
MC-ST-02 `stMcEnqueueGrowsHeap` A borrower joining the settlement queue adds exactly one entry to it.	✅	🎯
MC-ST-03 `stMcDequeueShrinksHeap` A borrower leaving the settlement queue removes exactly one entry from it.	✅	🎯
MC-ST-04 `stMcInflowStampsSnapshot` Any rise in the inflow throttle restarts its decay clock at the current time.	✅	🎯
MC-ST-05 `stMcSettlementLtvSelfService` Only the borrower themselves can move their own settlement target.	✅	🎯

MC-ST-06 `stMcTimerEntryFeedsThrottle` Collateral entering the activation timer is metered into the inflow throttle, up to its ceiling.	✅	🎯

High Level

Properties	Status	Mutations
MC-HL-01 `hlMcOnLeavesMinDelay` Market traffic never changes the activation wait floor.	✅	🎯
MC-HL-02 `hlMcOnLeavesMaxDelay` Market traffic never changes the activation wait ceiling.	✅	🎯
MC-HL-03 `hlMcOnLeavesSupplyCap` Market traffic never changes the market's supply cap.	✅	🎯
MC-HL-04 `hlMcOnLeavesSettlementLtv` Market traffic never changes a borrower's settle-down LTV target.	✅	🎯
MC-HL-05 `hlMcOnLeavesInstantActivationList` Market traffic never grants or revokes a caller's instant activation.	✅	🎯
MC-HL-06 `hlMcActivationDelayMonotoneBanded` A larger deposit is never quoted a shorter activation wait, and every quote stays between the configured floor and ceiling.	✅🔻	🎯
MC-HL-07 `hlMcSupplyCollateralTouchesOneBorrower` A collateral deposit changes the activation buckets and timer of at most one borrower.	✅	🎯
MC-HL-08 `hlMcSetActivationConfigLeavesPositions` Changing the activation policy never touches any borrower's pending collateral or its timer.	✅	🎯
MC-HL-09 `hlMcPauseResumeRoundTrip` Resuming ends the pause at once and leaves the supply cap and the activation delays untouched.	✅	🎯
MC-HL-10 `hlMcResumeLeavesFrozen` Resuming a paused market never lifts a freeze.	✅	🎯
MC-HL-11 `hlMcPauseForLeavesFrozen` Pausing the market never sets or lifts a freeze.	✅	🎯
MC-HL-12 `hlMcFreezeLeavesResumeTimestamp` Freezing the market never changes when a pause ends.	✅	🎯
MC-HL-13 `hlMcUnfreezeLeavesResumeTimestamp` Lifting a freeze never changes when a pause ends.	✅	🎯
MC-HL-14 `hlMcZeroPriceSizesNoSettlement` When the oracle reports a zero price, no settlement quote cuts anything, and each slice reports the borrower's debt untouched.	✅	🎯🎯
MC-HL-15 `hlMcMaxDepositOracleIndependent` The remaining deposit room does not move when the oracle price moves.	✅	🎯
MC-HL-16 `hlMcMaxWithdrawableOracleIndependent` A lender's withdrawable amount does not move when the oracle price moves.	✅	🎯

MC-HL-17 `hlMcSettlementViewsAgreeSupply` The next-to-settle view reports the lender shares from whichever of the borrower's two settlement quotes is larger.	✅🔻	🎯🎯🔻
MC-HL-18 `hlMcSettlementViewsAgreeBorrow` The next-to-settle view reports the debt shares from whichever of the borrower's two settlement quotes is larger.	✅🔻	🎯
MC-HL-19 `hlMcSettlementViewsAgreeRemainder` The next-to-settle view reports the debt left over by whichever of the two settlement quotes is larger.	✅🔻	🎯🎯🔻
MC-HL-20 `hlMcInflowCapBreaksOnlyViaConfig` Only a governance policy change can leave the inflow throttle above its ceiling.	✅	🎯
MC-HL-21 `hlMcBorrowMaturationGateExact` A borrow matures collateral only after the full wait, guardian surcharges included, and an open quorum gate.	✅	🎯🎯🎯
MC-HL-22 `hlMcFrozenOrBannedDefersMaturation` While the market is frozen or the borrower is banned, a deposit never matures the waiting collateral.	✅	🎯
MC-HL-23 `hlMcNonGrantedDepositEntersActivating` An ordinary caller's deposit that lifts the borrower's dust over the activation threshold enters the timer with it.	✅	🎯
MC-HL-24 `hlMcInflowDecayLaws` Without new deposits the inflow throttle only decays, and the wait it quotes never grows.	✅🔻	🎯
MC-HL-25 `hlMcTopUpDeadlineIsMaxOfOldAndQuote` A top-up unlocks at the later of the standing deadline and the one its own size earns.	✅🔻	🎯
MC-HL-26 `hlMcWithdrawUnderLockKeepsFloor` While the lock holds, a withdrawal that completes leaves both inactive buckets exactly as they were.	✅	🎯🎯
MC-HL-27 `hlMcSettlementSliceConservesDebt` A quoted settlement cut never exceeds the borrower's debt, and any nonzero cut plus the remainder equals it.	✅🔻	🎯
MC-HL-28 `hlMcTimedPauseHoldsThroughWindow` A timed pause stays in force for its whole window.	✅	🎯
MC-HL-29 `hlMcTimedPauseExpiresWithoutResume` A timed pause ends by itself when its window elapses, with no resume call.	✅	🎯
MC-HL-30 `hlMcLockLiftsAfterDeadline` Once the lock has elapsed, a covering withdrawal clears the borrower's whole pending collateral.	✅	🎯🎯
MC-HL-31 `hlMcVoluntaryDrainIsAllOrNothing` A debt-free borrower's withdrawal either leaves the inactive buckets whole under a live lock, or clears them both.	✅	🎯
MC-HL-32 `hlMcRootSliceHonorsPolicyFloors` The queue-root policy quote honors both settlement floors and never exceeds the borrower's debt.	✅🔻	🎯

MC-HL-33 `hlMcSlicePartitionsByEligibility` A borrower's personal quote is its settle-down target capped at its debt, and zero without an opt-in.	✅	🎯
MC-HL-34 `hlMcSameTxSupplyWithdrawUndoesInflow` A deposit fully withdrawn in the same transaction leaves the inflow throttle exactly where it stood.	✅	🎯
MC-HL-35 `hlMcDustDepositLeavesThrottle` A deposit that leaves the borrower's total under the activation threshold never feeds the inflow throttle or restarts its decay.	✅	🎯
MC-HL-36 `hlMcSupplyGateIgnoresTheClock` Outside a pause, a lender's deposit fares the same at any time and starts no activation timer.	✅	🎯
MC-HL-37 `hlMcNoRestrictionIsTerminal` An unfreeze followed by a resume always reopens the market, whatever call came before.	✅	🎯
MC-HL-38 `hlMcFallbackAloneMaturesSurchargedLot` A deposit whose surcharged wait outlasts the quorum window still matures without a quorum scan.	✅	🎯
MC-HL-39 `hlMcSubThresholdDepositStaysDust` An ordinary caller's deposit that keeps the borrower's dust under the activation threshold joins it and starts no timer.	✅	🎯
MC-HL-40 `hlMcBadDebtVerdictMatchesTheRoundedPredicate` The riskiest borrower counts as bad debt exactly when its debt, priced up, exceeds its collateral, priced down.	✅	🎯
MC-HL-41 `hlMcZeroValuedRootClearsByExactShareRepay` Anyone can repay the whole debt of the riskiest borrower when its collateral prices to nothing, clearing it from the queue.	✅	🎯
MC-HL-42 `hlMcCrossTxWithdrawLeavesInflowCharged` A deposit withdrawn in a later transaction leaves the inflow throttle charged.	✅	🎯
MC-HL-43 `hlMcSettlementQuoteFitsBudget` A settlement quote never asks the caller for more supply shares than the budget it was given.	⏱️	🎯
MC-HL-44 `hlMcInstantDepositLandsActiveAndLeavesDust` A trusted caller's deposit is active at once whatever its size, and leaves dust and the inflow throttle untouched.	✅	🎯🎯
MC-HL-45 `hlMcInflowUndoIsPerBorrower` A borrower who added nothing to the inflow throttle takes nothing back on exit, so another borrower's charge stands.	✅	🎯
MC-HL-46 `hlMcSameTxSeizureUndoesInflow` A deposit seized back in the same transaction leaves the inflow throttle exactly where it stood.	✅	🎯
MC-HL-47 `hlMcSeizeTakesActiveThenDustThenActivating` A seizure takes active collateral first, then dust, then the activating lot, and leaves any surviving lot on its own timer.	✅	🎯

Unit Tests

Properties	Status	Mutations
MC-UT-01 `utMcMaxDepositMatchesCapHeadroom` The deposit limit the market reports is the room left under the supply cap, and zero while paused.	✅	🎯
MC-UT-02 `utMcMaxWithdrawableMatchesFormula` A supplier's reported withdrawable amount is their own supply capped by idle liquidity, and zero while paused.	✅	🎯
MC-UT-03 `utMcHasBadDebtMatchesRootPredicate` The market reports bad debt exactly when the riskiest queued borrower's debt exceeds their collateral value.	✅	🎯🎯
MC-UT-04 `utMcNextSettlementBorrowerIsHeapRoot` The next borrower to settle is the riskiest one in the queue, and an empty queue offers none.	✅	🎯
MC-UT-05 `utMcPositionOfUnlockTimeFormula` A borrower's position summary reports their collateral split, debt and unlock time, and a banned borrower's collateral never unlocks.	✅	🎯🎯🎯
MC-UT-06 `utMcSetSupplyCapStoresValue` The manager's new supply cap is exactly the cap the market then reports.	✅	🎯
MC-UT-07 `utMcFreezeLatchesFlag` The market reports itself frozen as soon as the freezer freezes it.	✅	🎯
MC-UT-08 `utMcRootSliceHonorsLtvStep` The settlement policy's step rule cuts a borrower's loan to value by one step, never more than their debt.	✅	🎯🎯
MC-UT-09 `utMcRootSliceSaturatesToFullDebt` A borrower whose loan to value is within one policy step is quoted their whole debt.	✅	🎯
MC-UT-10 `utMcRoleIdsArePairwiseDistinct` The six authorities the controller recognizes are six distinct roles.	✅	🎯
MC-UT-11 `utMcGuardianSignalsViewMirrorsStorage` The controller reports the guardian registry it was wired to.	✅	🎯
MC-UT-12 `utMcSetInstantActivationStoresFlag` Granting or revoking instant activation lands on the named caller alone.	✅	🎯
MC-UT-13 `utMcRootSliceSweepReadsFullCutFloor` A cut is lifted to a full close by the collateral it would strand, not by the loan amount it takes.	✅	🎯

Reachability

Properties	Status	Mutations
MC-RC-01 `rcMcSetCollateralActivationConfigIsReachable`		

The manager can move the floor or the ceiling on the activation wait.	✅	🎯
MC-RC-02 `rcMcSetSettlementConfigIsReachable` The manager can change how deep a settlement may cut a borrower's debt.	✅	🎯
MC-RC-03 `rcMcOnBorrowGrowsHeapIsReachable` A borrow can add a new borrower to the settlement queue.	✅	🎯
MC-RC-04 `rcMcOnSupplyCollateralCreditsActivatingIsReachable` A collateral deposit can enter the activation timer.	✅	🎯
MC-RC-05 `rcMcOnWithdrawCollateralDebitsBucketsIsReachable` A borrower can withdraw dust or collateral that is still waiting to activate.	✅	🎯
MC-RC-06 `rcMcOnRepayShrinksHeapIsReachable` A borrower can repay the whole debt and leave the settlement queue.	✅	🎯
MC-RC-07 `rcMcOnSettleShrinksHeapIsReachable` A settled borrower can leave the settlement queue.	✅	🎯
MC-RC-08 `rcMcOnLiquidateSeizesBucketsIsReachable` A liquidation can seize dust or collateral still waiting to activate.	✅	🎯
MC-RC-09 `rcMcOnSupplyPassesLiveCapIsReachable` A deposit can fill the market exactly to its supply cap and still pass.	✅	🎯
MC-RC-10 `rcMcOnNoOpEventsSucceedWithEmptyPayloadIsReachable` A lender withdrawal and an interest accrual can pass the policy hook carrying no data.	✅	🎯
MC-RC-11 `rcMcOnFirstEnqueueFromEmptyHeapIsReachable` The first borrower can enter an empty settlement queue.	✅	🎯
MC-RC-12 `rcMcOnRekeysNodeAtFullHeapIsReachable` A queued borrower's risk ranking can change while the queue keeps its size.	✅	🎯
MC-RC-13 `rcMcFreezeThenUnfreezeIsReachable` The market can be frozen and then unfrozen again.	✅	🎯🎯
MC-RC-14 `rcMcPauseThenResumeIsReachable` The market can be paused and then resumed before the pause expires.	✅	🎯🎯
MC-RC-15 `rcMcSetThenReadSettlementLtvIsReachable` A borrower's chosen settlement target can be read back exactly as it was set.	✅	🎯
MC-RC-16 `rcMcResumeWhileFrozenIsReachable` The pause can be lifted while the market stays frozen.	✅	🎯
MC-RC-17 `rcMcNextSettlementBorrowerRealIsReachable` The settlement queue can name a real borrower to settle next.	✅	🎯
MC-RC-18 `rcMcHasBadDebtIsReachable` The market can report bad debt on the borrower first in line to settle.	✅	🎯
MC-RC-19 `rcMcSettlementForSizesPositiveCutIsReachable` A borrower can be quoted a positive settlement cut.	✅	🎯
MC-RC-20 `rcMcMaxDepositPositiveUnderCapIsReachable` The market can still report room to deposit while a supply cap is set.	✅	🎯

MC-RC-21 `rcMcMaxWithdrawableAssetsPositiveIsReachable` A lender can be shown a positive withdrawable amount.	✅	🎯
MC-RC-22 `rcMcActivationDelayStrictlyGrowsIsReachable` A bigger deposit can face a strictly longer activation wait.	✅	🎯
MC-RC-23 `rcMcOnLastDequeueEmptiesHeapIsReachable` The last borrower leaving can return the settlement queue to empty.	✅	🎯
MC-RC-24 `rcMcOnSupplyCollateralInstantActivationIsReachable` A trusted caller's large deposit can count as active at once with no timer.	✅	🎯
MC-RC-25 `rcMcOnSupplyCollateralDustStaysIsReachable` A deposit below the threshold can stay dust and leave the timed lot alone.	✅	🎯
MC-RC-26 `rcMcOnBorrowMaturesLotIsReachable` An unflagged borrower's collateral can mature the moment its wait elapses.	✅	🎯
MC-RC-27 `rcMcOnSupplyWithCapHeadroomIsReachable` A deposit can pass the cap check with the market total below the cap.	✅	🎯
MC-RC-28 `rcMcOnBorrowFallbackMaturesSurchargedLotIsReachable` A flagged deposit can still mature on the fallback when guardians have not scanned it.	✅	🎯
MC-RC-29 `rcMcOnSupplyCollateralInstantDustStaysIsReachable` A trusted caller's deposit can leave a borrower's existing dust exactly where it was.	✅	🎯
MC-RC-30 `rcMcMaturedLotReportedActivatingIsReachable` Collateral past its unlock can still read as waiting until a market action matures it.	✅	🎯
MC-RC-31 `rcMcOnRepayNotBlockedByFreeze` A borrower can still repay while the market is frozen.	✅	🎯
MC-RC-32 `rcMcOnRepayNotBlockedByBan` A banned borrower can still repay their debt.	✅	🎯
MC-RC-33 `rcMcOnWithdrawCollateralDebtFreeExitNotBlocked` A debt-free borrower can still withdraw collateral under both a freeze and a ban.	✅	🎯
MC-RC-34 `rcMcOnSupplyCollateralNotBlockedByBan` A banned address can still deposit collateral into the market.	✅	🎯
MC-RC-35 `rcMcOnWithdrawCollateralClearsWholeHoldingIsReachable` A withdrawal can carry a borrower's whole inactive holding away in one call and leave the timer cleared.	✅	🎯

Reverts

Properties	Status	Mutations
MC-RV-01 `rvMcOnRevertsWhenCallerNotCore` Only the core lending contract can drive this market's policy hook.	✅	🎯
MC-RV-02 `rvMcOnRevertsOnForeignMarketId` A hook call for any market other than the bound one fails.	✅	🎯🎯
MC-RV-03 `rvMcOnRevertsWhenPaused` While the controller is paused, every core action on the market fails.	✅	🎯

MC-RV-04 `rvMcOnPassiveEventRevertsIff` The controller blocks a lender withdrawal or interest accrual exactly when the caller, market or pause check rejects it.	✅	🎯🎯
MC-RV-05 `rvMcOnBorrowRevertsWhenFrozen` While the market is frozen, every new borrow fails.	✅	🎯
MC-RV-06 `rvMcOnSettleRevertsOnRootBadDebt` No settlement proceeds while the riskiest queued borrower is underwater.	✅	🎯
MC-RV-07 `rvMcSetActivationConfigRevertsIff` Installing an activation config fails exactly when any of its fields falls outside its allowed range.	✅	🎯
MC-RV-08 `rvMcSetInstantActivationEnableNeverReverts` The manager can always grant instant activation to a caller, even a banned one.	✅	🎯
MC-RV-09 `rvMcSetInstantActivationDisableNeverReverts` The manager can always revoke instant activation, even from a caller that never had it.	✅	🎯
MC-RV-10 `rvMcSetSupplyCapNeverReverts` The manager can always set the supply cap, including below the assets already supplied.	✅	🎯
MC-RV-11 `rvMcSetSettlementConfigRevertsIff` Installing a settlement policy fails exactly when either of its percentages exceeds 100 percent.	✅	🎯
MC-RV-12 `rvMcSetSettlementLtvRevertsIff` Setting a borrower's own settlement target fails exactly when it reaches 100 percent.	✅	🎯
MC-RV-13 `rvMcFreezeNeverReverts` The freezer can always freeze the market, even when it is already frozen.	✅	🎯
MC-RV-14 `rvMcUnfreezeNeverReverts` The unfreezer can always lift a freeze, even while the controller is paused.	✅	🎯
MC-RV-15 `rvMcPauseForRevertsIff` Pausing the market fails exactly when it is already paused, the duration is zero, or the clock overflows.	✅	🎯
MC-RV-16 `rvMcResumeRevertsIff` Resuming fails exactly when no pause is in force.	✅	🎯
MC-RV-17 `rvMcIsPausedNeverReverts` Reading whether the market is paused never fails.	✅	🎯
MC-RV-18 `rvMcCurrentInflowNeverReverts` Reading the current level of the collateral inflow throttle never fails.	✅	🎯
MC-RV-19 `rvMcCurrentActivationDelayNeverReverts` Quoting the base activation wait for a collateral inflow of any size never fails.	✅🔻	🎯
MC-RV-20 `rvMcMaxDepositNeverReverts` Reading how much room is left under the supply cap never fails.	✅	🎯
MC-RV-21 `rvMcMaxWithdrawableAssetsNeverReverts` Reading how much a lender can withdraw right now never fails.	✅	🎯

MC-RV-22 `rvMcSettlementLtvViewNeverReverts` Reading a borrower's settlement target and the debt slice it allows never fails while the market oracle answers.	✅	🎯
MC-RV-23 `rvMcHasBadDebtNeverReverts` Reading whether the market carries bad debt never fails while the market oracle answers.	✅	🎯
MC-RV-24 `rvMcNextSettlementBorrowerNeverReverts` Reading which borrower settles next, and what a given budget could cut, never fails.	✅🔻	🎯
MC-RV-25 `rvMcSettlementForNeverReverts` Pricing a settlement cut against a borrower within a spendable supply budget never fails.	⏱️	🎯🎯
MC-RV-26 `rvMcSettlementBorrowersNeverReverts` Paging through the settlement queue never fails, even past its end.	✅	🎯
MC-RV-27 `rvMcPositionOfNeverReverts` Reading a borrower's full position summary never fails while the market oracle answers.	✅	🎯
MC-RV-28 `rvMcOnWithdrawRevertsWhenFrozenWithDebt` A frozen market refuses a collateral withdrawal while the position still carries debt.	✅	🎯
MC-RV-29 `rvMcOnBorrowRevertsWhenActorBanned` A borrow fails when the guardian has banned its caller, its receiver, or the borrower.	✅	🎯🎯
MC-RV-30 `rvMcOnBorrowRevertsAboveShareRateCeiling` A borrow fails when it would push the market's borrow shares past the rate ceiling or draw zero assets.	✅	🎯
MC-RV-31 `rvMcOnBorrowRevertsAboveActiveCapacity` A borrow fails unless the borrower's borrowable collateral clears the activation minimum and covers the resulting debt.	✅🔻	🎯🎯
MC-RV-32 `rvMcOnWithdrawRevertsWhenRemainingActiveInsufficient` A borrower cannot withdraw collateral their remaining debt still needs at the market's liquidation threshold.	⏱️	🎯
MC-RV-33 `rvMcOnSettleIneligibleBorrowerReverts` A settlement can cut only the riskiest queued borrower or one who opted in.	✅	🎯
MC-RV-34 `rvMcOnFullRepayRemovesFromQueue` A borrower who repays in full leaves the settlement queue.	✅	🎯
MC-RV-35 `rvMcOnSupplyCollateralInstantSurvivesCallerBan` A banned caller with the instant activation grant can still deposit collateral that is active at once.	✅	🎯
MC-RV-36 `rvMcOnSupplyCollateralInstantSurvivesFreeze` In a frozen market a caller with the instant activation grant can still deposit collateral that is active at once.	✅	🎯
MC-RV-37 `rvMcOnPartialRepayKeepsQueued` After a partial repay the borrower stays queued exactly while collateral remains.	✅	🎯
MC-RV-38 `rvMcOnRepayDustRootStaysNext` A borrower who repays down to one share can still be the next in line for settlement.	✅	🎯
MC-RV-39 `rvMcOnWithdrawCollateralDebtFreeFullExitRevertsUnderLock`		

A borrower with no debt cannot withdraw their whole collateral while a fresh supply is still locked.	✅	🎯🎯
MC-RV-40 `rvMcOnSupplyRevertsIff` Only the caller, market and pause checks or the supply cap can stop a lender deposit.	✅	🎯🎯🎯
MC-RV-41 `rvMcOnRepayRevertsIff` Only the caller, market and pause checks, the share rate ceiling or repaying more than is owed can stop a repayment.	✅	🎯
MC-RV-42 `rvMcOnLiquidateRevertsIff` Only the caller, market and pause checks or seizing or repaying more than the borrower holds can stop a liquidation.	✅	🎯
MC-RV-43 `rvMcOnBorrowRevertsIff` Only the caller, market and pause checks, a freeze or ban, the rate ceiling or collateral capacity can stop a borrow.	✅🔻	🎯🎯
MC-RV-44 `rvMcOnSupplyCollateralRevertsIff` Only the usual checks, the deposit floor, unresolved dust or an overfull activating lot can stop a collateral deposit.	✅	🎯
MC-RV-45 `rvMcOnWithdrawCollateralRevertsIff` Only the usual checks, a freeze with open debt, the position size, the lock or uncovered debt can stop a collateral withdrawal.	✅🔻	🎯🎯
MC-RV-46 `rvMcOnSettleRevertsAboveSettleableSupply` Settling the riskiest borrower past their allowed slice fails unless the lender's illiquid supply funds it.	⏱️	🎯
MC-RV-47 `rvMcOnSettleRevertsAcrossShareRateCeiling` Settling fails when the cut would carry a market from under the borrow share rate ceiling to over it.	✅	🎯
MC-RV-48 `rvMcOnWithdrawCollateralRevertsUnderActivatingLock` While the activating lock stands, a withdrawal that reaches past the active collateral is refused.	✅	🎯
MC-RV-49 `rvMcOnWithdrawCollateralRevertsWhenInactiveNotCleared` Once the lock has elapsed, a withdrawal that leaves any of the inactive holding behind is refused.	✅	🎯

Access Control

Properties	Status	Mutations
MC-AC-01 `acMcOnlyManagerMovesActivationConfig` Only a manager can change the collateral activation delays and limits.	✅	🎯🎯
MC-AC-02 `acMcOnlyManagerMovesSettlementConfig` Only a manager can change the settlement policy that sizes debt cuts.	✅	🎯
MC-AC-03 `acMcOnlyManagerMovesSupplyCap` Only a manager can raise or lower the market's supply cap.	✅	🎯
MC-AC-04 `acMcOnlyManagerFlipsInstantActivation` Only a manager can grant or revoke a caller's exemption from the collateral activation	✅	🎯

delay.		
MC-AC-05 `acMcOnlyFreezerFreezes` Only a freezer can put the market into the frozen state.	✅	🎯
MC-AC-06 `acMcOnlyUnfreezerUnfreezes` Only an unfreezer can lift the market out of the frozen state.	✅	🎯
MC-AC-07 `acMcOnlyPauserPauses` Only a pauser can put the controller into a paused state.	✅	🎯
MC-AC-08 `acMcOnlyResumerResumes` Only a resumer can lift an active pause before it expires.	✅	🎯
MC-AC-09 `acMcOnlyCoreMovesActivationPosition` Only the core can change a borrower's collateral that is not yet active, or its timer.	✅	🎯
MC-AC-10 `acMcOnlyCoreMovesInflow` Only the core can change the recorded collateral inflow that sets activation delays.	✅	🎯
MC-AC-11 `acMcOnlyCoreMovesSettlementHeap` Only the core can change the settlement queue of borrowers.	✅	🎯
MC-AC-12 `acMcOnlyDefaultAdminMovesControllerRoles` Only a role's admin can grant or revoke it, apart from a holder giving up their own.	✅	🎯
MC-AC-13 `acMcRoleAdminNeverMoves` Which role may grant and revoke a controller role never changes.	✅	🎯
MC-AC-14 `acMcOnlyPauseAuthoritiesMoveResumeStamp` Only a pauser or a resumer can move the moment a pause ends.	✅	🎯

Reentrancy

Properties	Status	Mutations
MC-RE-01 `reMcIssuesNoStateChangingCall` The controller issues no state changing call and no delegatecall of any kind.	✅	🎯
MC-RE-02 `reMcHookReachesTheCoreAndTheGuardian` A collateral hook can read both the core and the guardian registry before it returns.	✅	🎯

LidoLend

- Every configuration
  - Loops running up to 2 iterations
- Every conf except the three below
  - Controller, oracle, interest model and tokens replaced by CVL models
  - Borrower callbacks summarized away
- Every conf except `accrue_interest.conf` and the three below
  - One symbolic market, the per-market mirrors carrying no market key
  - Position holders up to 7 modeled accounts plus the zero slot
- `reentrancy.conf`

    - Callbacks, token moves and the oracle running unresolved; the interest model and the controller trusted CVL models
    - STATICCALL not watched, so read-only reentrancy is not covered
- `oracle_outage.conf`
    - No valid state and no storage mirrors
    - The oracle a reverting mock; the interest model and the controller hook NONDET
- `create_market.conf`
    - No market pinned and no storage mirrors

Valid State

Properties	Status	Mutations
LL-VS-01 `feeCapped` A market's fee on accrued interest can never exceed the protocol's 25 percent cap.	✅	🎯
LL-VS-02 `lastUpdateNotInFuture` A market's interest clock never runs ahead of the current block time.	✅	🎯
LL-VS-03 `lastUpdateMinBound` A live market's interest clock always holds a plausible block time, never a near zero value.	✅	🎯
LL-VS-04 `borrowLeSupplyAssets` A market never has more assets borrowed than supplied, so utilization stays at or below 100 percent.	✅	🎯
LL-VS-05 `supplySharesSumCorrect` A market's total supply shares always equal the sum of the lenders' shares.	✅	🎯
LL-VS-06 `borrowSharesSumCorrect` A market's total borrow shares always equal the sum of the borrowers' shares.	✅	🎯
LL-VS-07 `borrowerCollateralizedUnlessSettlement` In a market without settlement, any borrower carrying debt still holds collateral.	✅	🎯
LL-VS-08 `zeroAddressNeverAuthorizes` Nobody is ever authorized to act on behalf of the zero address.	✅	🎯
LL-VS-09 `tokenBackingSolvency` The pool always holds enough loan and collateral tokens to cover idle liquidity and the collateral entrusted to it.	⏱	🎯

Variable Transitions

Properties	Status	Mutations
LL-VT-01 `vtLlLastUpdateMonotonic` A market's interest clock only ever moves forward.	✅	🎯
LL-VT-02 `vtLlInitializedLatchesOn` Once initialized, the pool can never be returned to an uninitialized state.	✅	🎯
LL-VT-03 `vtLlNonceMonotonic` Apart from signature based authorization, no call moves a signer's nonce.	✅	🎯

LL-VT-04 `vtLlCoreBalanceOutflowOnlyViaExits` Only withdraw, borrow, collateral withdrawal, liquidation, settlement and flash loans can reduce the pool's token balance.	✅	🎯
LL-VT-05 `vtLlMarketWiringImmutable` A market's tokens, oracle, rate model, liquidation threshold, controller and settlement switch never change after creation.	✅	🎯

State Transitions

Properties	Status	Mutations
LL-ST-01 `stLlBorrowGrowthIsFresh` Total debt grows only in a call that leaves interest up to date.	✅	🎯
LL-ST-02 `stLlNewDebtNeedsCollateral` A borrower's debt grows only while that same position holds collateral.	✅	🎯 🎯
LL-ST-03 `stLlFeeChangeIsFresh` A market's fee changes only in a call that leaves interest up to date.	✅	🎯
LL-ST-04 `stLlSupplyShrinkIsAccounted` The supply pool shrinks only when supply shares are burned or debt is written off.	⏱	🎯
LL-ST-05 `stLlSupplyShareMintAddsAssets` Minting supply shares always adds assets to the supply pool.	✅	🎯
LL-ST-06 `stLlSupplyShareBurnRemovesAssets` Burning supply shares never adds assets to the supply pool.	✅	🎯
LL-ST-07 `stLlBorrowShareMintAddsDebt` Minting debt shares never cancels part of the market's outstanding debt.	✅	🎯
LL-ST-08 `stLlAccrualFavorsSuppliers` Accruing interest credits suppliers at least as much as the outstanding debt grows.	✅	🎯
LL-ST-09 `stLlAccountingRefreshesClock` Once time has passed, any move in the market totals restamps the interest clock.	✅	🎯
LL-ST-10 `stLlSupplyShareMintIsFunded` A supply that mints shares always brings loan tokens into the pool.	✅	🎯
LL-ST-11 `stLlUserSupplyMintGrowsTotal` Crediting shares to a supplier always grows the market's total supply shares.	✅	🎯
LL-ST-12 `stLlUserSupplyBurnShrinksTotal` Burning a supplier's shares always shrinks the market's total supply shares.	✅	🎯
LL-ST-13 `stLlUserDebtMintGrowsTotal` Crediting debt shares to a borrower always grows the market's total debt shares.	✅	🎯
LL-ST-14 `stLlUserDebtBurnShrinksTotal` Burning a borrower's debt shares always shrinks the market's total debt shares.	✅	🎯

High Level

Properties	Status	Mutations

LL-HL-01 `hlLlSupplyCreditsExactShares` A supplier's position grows by exactly the shares the deposit reports.	✅	🎯
LL-HL-02 `hlLlWithdrawDebitsExactShares` A supplier's position shrinks by exactly the shares the withdrawal reports.	✅	🎯
LL-HL-03 `hlLlBorrowCreditsExactDebt` A borrower's debt grows by exactly the debt shares the borrow reports.	✅	🎯
LL-HL-04 `hlLlRepayDebitsExactDebt` A borrower's debt shrinks by exactly the debt shares the repayment reports.	✅	🎯
LL-HL-05 `hlLlSupplyCollateralLeavesLoanTotals` Posting collateral leaves the market's supply and borrow totals unchanged.	✅	🎯
LL-HL-06 `hlLlWithdrawCollateralLeavesBorrowShares` Taking collateral back leaves the market's total debt shares unchanged.	✅	🎯
LL-HL-07 `hlLlFlashLoanLeavesMarkets` A flash loan returns every borrowed token and leaves the pool's supply total unchanged.	✅	🎯
LL-HL-08 `hlLlAccrueKeepsBorrowShares` Accruing interest never mints or burns debt shares.	✅	🎯
LL-HL-09 `hlLlSupplyLeavesOtherPositions` A deposit leaves every other user's shares, debt and collateral untouched.	✅	🎯
LL-HL-10 `hlLlSettleReturnsExactDeltas` Settlement reports exactly the amounts it removes from the books, with the market's debt floored at zero.	✅	🎯🎯🎯🎯
LL-HL-11 `hlLlSettleSeizesAtPar` Settlement seizes collateral at par with the debt it cancels, to within one unit.	✅	🎯
LL-HL-12 `hlLlBorrowLeavesPositionHealthy` A borrow leaves the borrower's position healthy at the current oracle price.	✅	🎯🎯
LL-HL-13 `hlLlWithdrawCollateralLeavesPositionHealthy` Pulling collateral out never leaves the borrower's own position under water.	✅	🎯🎯
LL-HL-14 `hlLlLiquidateRevertsOnHealthyBorrower` Liquidating a borrower whose collateral still covers their debt always fails.	⏱	🎯
LL-HL-15 `hlLlLiquidateDebitsSharesLockstep` A liquidation burns the borrower's debt shares and the market's total in lockstep and seizes what it reports.	✅	🎯🎯🎯🎯
LL-HL-16 `hlLlLiquidateLeavesOtherPositions` A liquidation leaves every position other than the borrower's untouched.	✅	🎯
LL-HL-17 `hlLlAccrueGrowsSupplyAssets` Accruing interest never shrinks the pot the suppliers own.	✅	🎯
LL-HL-18 `hlLlAccrueAddsEqualValue` Accrued interest credits suppliers exactly what it charges borrowers.	✅	🎯🎯
LL-HL-19 `hlLlSupplyWithdrawNoProfit` Depositing and immediately withdrawing the same shares never returns more tokens than	✅	🎯

went in.		
LL-HL-20 `hlLlBorrowRepayNoProfit` Borrowing and immediately repaying the same debt shares never leaves the borrower ahead.	✅ 🔻	🎯
LL-HL-21 `hlLlSupplyPullsExactAssets` A deposit moves exactly the assets it reports from the payer into the pool.	✅	🎯
LL-HL-22 `hlLlWithdrawLeavesOtherPositions` A withdrawal leaves every other user's shares, debt and collateral untouched.	✅	🎯
LL-HL-23 `hlLlBorrowLeavesOtherPositions` A borrow leaves every other user's shares, debt and collateral untouched.	✅	🎯
LL-HL-24 `hlLlRepayLeavesOtherPositions` A repayment leaves every other user's shares, debt and collateral untouched.	✅	🎯
LL-HL-25 `hlLlSupplyCollateralLeavesOtherPositions` Posting collateral leaves every other user's shares, debt and collateral untouched, the fee recipient included.	✅	🎯
LL-HL-26 `hlLlWithdrawCollateralLeavesOtherPositions` Taking collateral back leaves every other user's shares, debt and collateral untouched.	✅	🎯
LL-HL-27 `hlLlAccrueRaisesSupplySharePrice` Accruing interest never makes a supply share cheaper.	✅	🎯 🎯
LL-HL-28 `hlLlSupplySharePriceOnlyFallsOnSeizure` With interest already up to date, only a liquidation can make a supply share cheaper.	✅	🎯 🎯
LL-HL-29 `hlLlLiquidateNoBadDebtRaisesSupplySharePrice` A liquidation that leaves the borrower some collateral never makes a supply share cheaper.	✅	🎯
LL-HL-30 `hlLlBorrowSharePriceOnlyRisesOnAccrual` With interest already up to date, only repayment, liquidation and settlement can make a debt share costlier.	⏱	🎯
LL-HL-31 `hlLlRepayNeverRaisesDebtSharePrice` A repayment never makes the remaining debt shares more expensive.	✅	🎯
LL-HL-32 `hlLlLiquidateNeverRaisesDebtSharePrice` A liquidation that leaves the market carrying debt never raises what a debt share owes.	⏱	🎯
LL-HL-33 `hlLlLiquidateSeizureBoundedByIncentive` Liquidating a comfortably collateralized borrower leaves them collateral and takes at most the same fraction of it as of their debt.	⏱	🎯
LL-HL-34 `hlLlOthersStayHealthy` No operation other than liquidation or settlement pushes an untouched, healthy borrower under water.	⏱	🎯
LL-HL-35 `hlLlLiquidateLeavesOthersHealthy` A liquidation that leaves the borrower collateral cannot push a healthy bystander under water.	⏱	🎯
LL-HL-36 `hlLlLastBorrowLiquidationLeavesOthersHealthy` A liquidation clearing the market's last debt still leaves a healthy bystander healthy.	⏱	🎯

LL-HL-37 `hlLlSupplyBookValueBounded` A deposit never buys more book value than the assets it pays in.	⏱️	🎯
LL-HL-38 `hlLlWithdrawBookValueBounded` A withdrawal never pays out more than the position was worth beforehand.	✅	🎯
LL-HL-39 `hlLlBorrowDebtValueBounded` A borrow always books at least as much debt as the assets it hands out.	⏱️	🎯
LL-HL-40 `hlLlRepayCostBounded` Clearing a debt costs at least what the position's debt shares were worth.	⏱️	🎯
LL-HL-41 `hlLlSupplyWithdrawAnyModeNoProfit` Depositing and immediately withdrawing never turns a profit, whether the amount is set in assets or in shares.	✅	🎯🎯
LL-HL-42 `hlLlBorrowRepayAnyModeNoProfit` Borrowing and immediately repaying the same debt never leaves the borrower ahead.	⏱️	🎯
LL-HL-43 `hlLlSettleBorrowAnchoredNeverRaisesDebtSharePrice` A settlement cancels at most one unit of debt beyond the book and otherwise never makes remaining debt shares costlier.	✅	🎯
LL-HL-44 `hlLlSettleBorrowAnchoredLeavesOthersHealthy` A settlement cannot push a healthy borrower it never touched into liquidation.	⏱️	🎯
LL-HL-45 `hlLlBorrowDebtValueBoundedAssetsMode` A borrow for a named amount hands over exactly that amount and books at least as much new debt.	⏱️	🎯
LL-HL-46 `hlLlFreeLiquidityFallsOnlyOnExitOrDraw` Only a lender withdrawal, a borrow or a settlement can reduce the cash a market has free.	✅	🎯
LL-HL-47 `hlLlWithdrawPaysAtMostFreeLiquidity` A withdrawal pays out at most the market's free cash and reduces it by exactly the amount paid.	✅	🎯
LL-HL-48 `hlLlBorrowDrawsAtMostFreeLiquidity` A borrow draws at most the market's free cash and reduces it by exactly the amount drawn.	✅	🎯
LL-HL-49 `hlLlLiquidateBadDebtTailClearsDebt` A liquidation that seizes all of a borrower's collateral leaves no debt standing against them.	✅	🎯
LL-HL-50 `hlLlLiquidateSocializesAtMostBorrowerDebt` A liquidation never charges lenders more than the liquidated borrower owed.	⏱️	🎯
LL-HL-51 `hlLlFeeWindowSettlesToRecipientOfRecord` When the fee recipient changes, the next interest accrual pays the whole pending fee to the new recipient.	✅	🎯
LL-HL-52 `hlLlSettleFreeLiquidityFallsByAtMostTheClampWei` A settlement gives up at most one unit of free cash, and none when the book covers the cancellation.	✅	🎯
LL-HL-53 `hlLlSettleSeizurePricedDownAgainstCancelUp` A settlement rounds the collateral it seizes down and the debt it cancels up.	✅	🎯

LL-HL-54 `hlLlSettleBurnsSupplySharesAtSupplyRate` A settlement burns the supply shares its canceled debt is worth, rounded up.	✅	🎯🎯
LL-HL-55 `hlLlSettleDoesNotRaiseCoveredBorrowerRiskKey` A settlement never raises the risk key of a borrower whose collateral covers the debt.	✅	🎯

Unit Tests

Properties	Status	Mutations
LL-UT-01 `utLlSetOwnerSetsNewOwner` A transfer of ownership installs exactly the named new owner.	✅	🎯🎯
LL-UT-02 `utLlSetFeeRecipientSetsNewRecipient` The owner installs exactly the named account as the fee recipient.	✅	🎯🎯
LL-UT-03 `utLlSetFeeSetsNewFee` The owner sets a market's fee to exactly the requested value.	✅	🎯
LL-UT-04 `utLlSetAuthorizationSetsFlag` The caller's authorization change records exactly the requested permission for the named account.	✅	🎯
LL-UT-05 `utLlSupplyCollateralCreditsExactCollateral` A collateral deposit credits the beneficiary with exactly the assets supplied.	✅	🎯🎯
LL-UT-06 `utLlWithdrawCollateralDebitsExactCollateral` A collateral withdrawal debits the position by exactly the assets requested.	✅	🎯
LL-UT-07 `utLlLiquidateNeverGrowsBorrowerDebt` Liquidation never increases the borrower's debt, whoever calls it.	✅	🎯
LL-UT-08 `utLlSetOwnerLeavesFeeRecipient` A transfer of ownership leaves the fee recipient unchanged.	✅	🎯
LL-UT-09 `utLlSetFeeRecipientLeavesOwner` Changing the fee recipient leaves the owner unchanged.	✅	🎯
LL-UT-10 `utLlSetAuthorizationLeavesOtherRows` An authorization change leaves every other permission unchanged.	✅	🎯
LL-UT-11 `utLlSupplyGrowsTotalsExactly` A supply grows both supply totals by exactly the shares and assets it reports.	✅	🎯
LL-UT-12 `utLlWithdrawShrinksTotalsExactly` A withdrawal shrinks both supply totals by exactly the shares and assets it reports.	✅	🎯
LL-UT-13 `utLlBorrowGrowsDebtTotalsExactly` A borrow grows both debt totals by exactly the shares and assets it reports.	✅	🎯
LL-UT-14 `utLlRepayShrinksDebtTotalsExactly` Repayment cuts the borrower's and the market's debt by exactly the amounts it reports.	✅	🎯
LL-UT-15 `utLlWithdrawMovesExactTokens` A withdrawal pays the receiver exactly the amount it reports, in full when an amount is requested.	✅	🎯
LL-UT-16 `utLlBorrowMovesExactTokens`		

A borrow hands the receiver exactly the amount it reports, in full when an amount is requested.	✅	🎯
LL-UT-17 `utLlRepayMovesExactTokens` Repayment pulls exactly the amount it reports from the caller, in full when an amount is requested.	✅	🎯
LL-UT-18 `utLlSupplyCollateralMovesExactTokens` A collateral deposit pulls exactly the deposited collateral from the caller into the pool.	✅	🎯
LL-UT-19 `utLlWithdrawCollateralMovesExactTokens` A collateral withdrawal pays the receiver exactly the requested collateral out of the pool.	✅	🎯
LL-UT-20 `utLlLiquidateMovesExactTokens` Liquidation takes in exactly the repaid loan tokens and hands out exactly the seized collateral.	✅	🎯
LL-UT-21 `utLlSettleMovesExactTokens` Settlement pays the receiver exactly the seized collateral and moves no loan tokens.	✅	🎯
LL-UT-22 `utLlAccrueIdleMarketMovesOnlyClock` A market with no rate model earns no interest, though accruing still updates its clock.	✅	🎯
LL-UT-23 `utLlAccrueZeroDebtLeavesTotals` With nothing borrowed, accruing interest changes no total and mints no fee shares.	✅	🎯
LL-UT-24 `utLlCreateControlledMarketBindsItsWiring` Creating a controlled market binds its parameters, controller and settlement switch.	✅	🎯
LL-UT-25 `utLlCreateUncontrolledMarketBindsNoController` Creating a market without a controller leaves it with no controller and no settlement.	✅	🎯

Accrue Interest

Properties	Status	Mutations
LL-AI-01 `aiLlSupplyAccrualIdempotent` Supplying already brings interest current first, so accruing beforehand changes nothing.	✅	🎯
LL-AI-02 `aiLlWithdrawAccrualIdempotent` Withdrawing already brings interest current first, so accruing beforehand changes nothing.	✅	🎯
LL-AI-03 `aiLlBorrowAccrualIdempotent` Borrowing already brings interest current first, so accruing beforehand changes nothing.	✅	🎯
LL-AI-04 `aiLlRepayAccrualIdempotent` Repaying already brings interest current first, so accruing beforehand changes nothing.	✅	🎯
LL-AI-05 `aiLlSetFeeAccrualIdempotent` Changing a market fee brings interest current at the old fee first, so accruing beforehand changes nothing.	✅	🎯
LL-AI-06 `aiLlWithdrawCollateralAccrualIdempotent` Withdrawing collateral brings interest current first, so accruing beforehand changes nothing.	✅	🎯
LL-AI-07 `aiLlSettleAccrualIdempotent` Settling a position already brings interest current first, so accruing beforehand changes	✅	🎯

nothing.		
LL-AI-08 `aiLlLiquidateAccrualIdempotent` Liquidating already brings interest current first, so accruing beforehand changes nothing.	✅	🎯
LL-AI-09 `aiLlAccrueTwiceIdempotent` Interest accrues at most once per block, so a second accrual changes nothing.	✅	🎯

Reachability

Properties	Status	Mutations
LL-RC-01 `rcLlSupplyIsReachable` A supplier can deposit assets and be credited freshly minted shares.	✅	🎯
LL-RC-02 `rcLlSupplySharesAnchoredIsReachable` A supplier can name an exact share amount and be minted exactly that.	✅	🎯
LL-RC-03 `rcLlSupplyBootstrapIsReachable` A first deposit into an empty market can mint its whole share supply to the depositor.	✅	🎯
LL-RC-04 `rcLlSupplyControlledWithCallbackIsReachable` A deposit into a market governed by the controller can complete and call the depositor back.	✅	🎯
LL-RC-05 `rcLlWithdrawIsReachable` A lender can withdraw assets and burn the matching supply shares.	✅	🎯
LL-RC-06 `rcLlWithdrawByDelegateIsReachable` An authorized delegate can withdraw a lender's supply on their behalf.	✅	🎯
LL-RC-07 `rcLlWithdrawAllLeavesAssetDustIsReachable` The sole lender can burn every share and still leave asset dust in the market.	✅	🎯
LL-RC-08 `rcLlBorrowIsReachable` A borrower can draw assets against collateral and take on debt shares.	✅	🎯
LL-RC-09 `rcLlRepayIsReachable` A borrower can repay assets and shrink the outstanding debt.	✅	🎯
LL-RC-10 `rcLlRepayAllLeavesAssetResidueIsReachable` A full repayment can zero all debt shares and still leave borrowed assets on the books.	✅	🎯
LL-RC-11 `rcLlSupplyCollateralIsReachable` A user can post collateral and be credited exactly the amount deposited.	✅	🎯
LL-RC-12 `rcLlWithdrawCollateralWithLiveDebtIsReachable` A borrower with live debt can still pull out surplus collateral.	✅	🎯
LL-RC-13 `rcLlWithdrawCollateralFullExitIsReachable` A debt free holder can pull its collateral out down to zero.	✅	🎯🎯
LL-RC-14 `rcLlSettleBorrowAnchoredIsReachable` A settlement can cancel debt shares of the borrower it names.	✅	🎯
LL-RC-15 `rcLlLiquidateSeizeAnchoredIsReachable` A liquidator can name the collateral to seize and cut the borrower's debt with it.	✅	🎯

LL-RC-16 `rcLlLiquidateRepaidAnchoredIsReachable` A liquidator can name the debt shares to repay and seize collateral for them.	✅	🎯
LL-RC-17 `rcLlLiquidateBadDebtIsReachable` A liquidation can wipe out a hopeless position and push the loss onto lenders.	✅	🎯🎯
LL-RC-18 `rcLlAccrueInterestGrowsDebtIsReachable` Accrued interest can grow the outstanding debt and the lenders' assets together.	✅	🎯
LL-RC-19 `rcLlAccrueInterestMintsFeeSharesIsReachable` Accrual under a fee can mint new shares to the fee recipient.	✅	🎯
LL-RC-20 `rcLlSupplyWithdrawRoundTripIsReachable` A supplier can enter and fully exit within a single block.	✅	🎯
LL-RC-21 `rcLlBorrowSharesAnchoredIsReachable` A borrower can name an exact debt share amount and be issued exactly that.	✅	🎯
LL-RC-22 `rcLlWithdrawAllWithLiveDebtIsReachable` A lender can exit its whole position while debt and other lenders remain in the market.	✅	🎯
LL-RC-23 `rcLlSettleOnHealthyBorrowerIsReachable` A settlement can cancel debt on a borrower that is still healthy.	✅	🎯
LL-RC-24 `rcLlSettleOnUnhealthyBorrowerIsReachable` A settlement can cancel debt on a borrower that is under water.	✅	🎯
LL-RC-25 `rcLlRepayByUnauthorizedThirdPartyIsReachable` An account with no authorization can repay another account's debt.	✅	🎯
LL-RC-26 `rcLlWithdrawToExactZeroLiquidityIsReachable` A withdrawal can take a market's free liquidity down to exactly zero while debt remains.	✅	🎯
LL-RC-27 `rcLlSettleBurnsWholeSupplyPositionIsReachable` A settlement can burn the entire supply position it is charged against.	✅	🎯
LL-RC-28 `rcLlSettleDebtFloorAbsorbsAWeiIsReachable` A settlement can cancel one unit more debt than the market's books record.	✅	🎯
LL-RC-29 `rcLlCreateSettleableMarketIsReachable` The owner can create a market bound to a controller with settlement switched on.	✅	🎯

Reverts

Properties	Status	Mutations
LL-RV-01 `rvLlSetOwnerRevertsIff` Handing over ownership fails exactly when the caller is not the owner or names the current owner.	✅	🎯
LL-RV-02 `rvLlSetFeeRevertsWhenNotOwner` Only the owner can change a market's fee.	✅	🎯
LL-RV-03 `rvLlSetFeeRevertsWhenFeeUnchanged` Setting a market fee to the value it already holds is rejected.	✅	🎯
LL-RV-04 `rvLlSetFeeRevertsWhenAboveMaxFee` A market fee can never be set above the protocol's hard cap.	✅	🎯

LL-RV-05 `rvLlSetFeeSucceedsOnValidInput` The owner can always set a different market fee within the cap.	✅	🎯
LL-RV-06 `rvLlSetFeeRecipientRevertsIff` Changing the fee recipient fails exactly when the caller is not the owner or names the current recipient.	✅	🎯
LL-RV-07 `rvLlSupplyRevertsWhenInconsistentInput` A supply must name either an asset amount or a share amount, never both or neither.	✅	🎯
LL-RV-08 `rvLlSupplyRevertsWhenZeroOnBehalf` Nobody can supply on behalf of the zero address.	✅	🎯
LL-RV-09 `rvLlWithdrawRevertsWhenInconsistentInput` A withdrawal must name either an asset amount or a share amount, never both or neither.	✅	🎯
LL-RV-10 `rvLlWithdrawRevertsWhenZeroReceiver` A withdrawal can never pay out to the zero address.	✅	🎯
LL-RV-11 `rvLlWithdrawRevertsWhenUnauthorized` Only the position holder or an account it authorized can withdraw its supply.	✅	🎯
LL-RV-12 `rvLlBorrowRevertsWhenInconsistentInput` A borrow must name either an asset amount or a share amount, never both or neither.	✅	🎯
LL-RV-13 `rvLlBorrowRevertsWhenZeroReceiver` A borrow can never pay out to the zero address.	✅	🎯
LL-RV-14 `rvLlBorrowRevertsWhenUnauthorized` Only the position holder or an account it authorized can borrow against it.	✅	🎯
LL-RV-15 `rvLlRepayRevertsWhenInconsistentInput` A repayment must name either an asset amount or a share amount, never both or neither.	✅	🎯
LL-RV-16 `rvLlRepayRevertsWhenZeroOnBehalf` Nobody can repay on behalf of the zero address.	✅	🎯
LL-RV-17 `rvLlSupplyCollateralRevertsWhenZeroAssets` Posting collateral fails when the amount is zero.	✅	🎯
LL-RV-18 `rvLlSupplyCollateralRevertsWhenZeroOnBehalf` Nobody can post collateral on behalf of the zero address.	✅	🎯
LL-RV-19 `rvLlWithdrawCollateralRevertsWhenZeroAssets` Withdrawing collateral fails when the amount is zero.	✅	🎯
LL-RV-20 `rvLlWithdrawCollateralRevertsWhenZeroReceiver` A collateral withdrawal can never send the collateral to the zero address.	✅	🎯
LL-RV-21 `rvLlWithdrawCollateralRevertsWhenUnauthorized` Only the position holder or an account it authorized can pull its collateral out.	✅	🎯
LL-RV-22 `rvLlSettleRevertsWhenSettlementDisabled` A market that has not enabled settlement can never be settled.	✅	🎯
LL-RV-23 `rvLlSettleRevertsWhenZeroReceiver` A settlement can never send the seized collateral to the zero address.	✅	🎯

LL-RV-24 `rvLlSettleRevertsWhenUnauthorized` Only the position holder or an account it authorized can settle on its behalf.	✅	🎯
LL-RV-25 `rvLlSettleRevertsOnBorrowAnchoredNoSupply` Cancelling a borrower's debt fails when the account absorbing the loss holds no supply.	✅	🎯🎯
LL-RV-26 `rvLlSettleRevertsWhenBorrowAnchoredExceedsDebt` A settlement can never cancel more debt than the borrower actually owes.	✅	🎯
LL-RV-27 `rvLlLiquidateRevertsWhenInconsistentInput` A liquidation must name either the collateral to seize or the debt shares to repay, never both or neither.	✅	🎯
LL-RV-28 `rvLlFlashLoanRevertsIff` A flash loan fails exactly when it asks for a zero amount.	✅	🎯
LL-RV-29 `rvLlSetAuthorizationRevertsIff` Granting or revoking an authorization fails exactly when it would not change the current permission.	✅	🎯
LL-RV-30 `rvLlAccrueInterestRevertsWhenHookInProgress` Accruing interest fails while one of the market's controller hooks is still running.	✅	🎯
LL-RV-31 `rvLlSettleRevertsOnZeroSlice` A settlement that cancels no debt shares is refused.	✅	🎯
LL-RV-32 `rvLlCreateControlledMarketRevertsIff` Controlled creation fails exactly when the caller, controller, threshold or id is wrong.	✅	🎯

Access Control

Properties	Status	Mutations
LL-AC-01 `acLlOnlyOwnerOrInitLatchMovesOwner` Only the current owner can hand over ownership, apart from the one time initial setup.	✅	🎯
LL-AC-02 `acLlOnlyOwnerMovesFeeRecipient` Only the owner can change the address that receives protocol fees.	✅	🎯
LL-AC-03 `acLlOnlyOwnerMovesMarketFee` Only the owner can change a market's interest fee.	✅	🎯
LL-AC-04 `acLlUnauthorizedCannotReduceSupplyShares` Only the lender or an account they authorized can reduce their supply shares.	✅	🎯
LL-AC-05 `acLlUnauthorizedCannotIncreaseBorrowShares` Only the borrower or an account they authorized can add to their debt.	✅	🎯
LL-AC-06 `acLlAuthorizationRowMovesOnlyByAuthorizer` Only the account itself grants or revokes permission over its positions.	✅	🎯
LL-AC-07 `acLlSeizureOnlyByLiquidateOrSettle` Collateral leaves a user without their consent only through liquidation or settlement.	✅	🎯
LL-AC-08 `acLlDebtFreeCollateralNeverSeized` A user with no debt never loses collateral to a caller they did not authorize.	✅	🎯

Reentrancy

Properties	Status	Mutations
LL-RE-01 `reLlHookedActionsRevertWhileHookInProgress` A controlled market refuses every hooked action while one of those hooks is running.	✅	🎯
LL-RE-02 `reLlHooklessMarketActionRevertsWhileHookInProgress` The gate is read before the market's controller, so a market without one is refused too.	✅	🎯
LL-RE-03 `reLlHookLatchClearedByReturn` A call that begins with the hook latch down always returns with it down.	✅	🎯
LL-RE-04 `reLlFlashLoanRunsWhileHookInProgress` A flash loan can still complete while one of the controller hooks is running.	✅	🎯
LL-RE-05 `reLlNoStorageTouchAfterUntrustedCall` The core never touches storage, calls a token or a callback, then touches storage again.	✅	🎯
LL-RE-06 `reLlSupplyCallsOutAfterWritingItsPosition` A supply carrying callback data can call out twice after touching its own storage.	✅	🎯
LL-RE-07 `reLlMakesNoDelegateCall` The core executes no delegatecall, so no foreign code runs against its storage.	✅	🎯

Oracle Outage

Properties	Status	Mutations
LL-OO-01 `ooLlWithdrawNeverReadsTheOracle` A lender withdrawal never reaches the market oracle on any of the core's own paths.	✅	🎯
LL-OO-02 `ooLlRepayNeverReadsTheOracle` A debt repayment never reaches the market oracle on any of the core's own paths.	✅	🎯
LL-OO-03 `ooLlDebtFreeWithdrawCollateralNeverReadsTheOracle` A debt free collateral withdrawal never reaches the market oracle.	✅	🎯
LL-OO-04 `ooLlSupplyNeverReadsTheOracle` A supply of loan assets never reaches the market oracle on any of the core's own paths.	✅	🎯
LL-OO-05 `ooLlSupplyCollateralNeverReadsTheOracle` A collateral supply never reaches the market oracle on any of the core's own paths.	✅	🎯
LL-OO-06 `ooLlWithdrawCanLandWhileTheOracleReverts` A lender can withdraw assets while the market oracle reverts on every call.	✅	🎯
LL-OO-07 `ooLlRepayCanLandWhileTheOracleReverts` A borrower can repay debt while the market oracle reverts on every call.	✅	🎯
LL-OO-08 `ooLlDebtFreeWithdrawCollateralCanLandWhileTheOracleReverts` A debt free borrower can withdraw collateral while the oracle reverts.	✅	🎯
LL-OO-09 `ooLlBorrowAlwaysFailsWhileTheOracleReverts` A borrow always fails while the market oracle reverts on every call.	✅	🎯
LL-OO-10 `ooLlIndebtedWithdrawCollateralAlwaysFailsWhileTheOracleReverts` A collateral withdrawal that leaves debt fails while the oracle reverts.	✅	🎯

LL-OO-11 `ooLlLiquidateAlwaysFailsWhileTheOracleReverts` A liquidation always fails while the market oracle reverts on every call.	✅	🎯
LL-OO-12 `ooLlSettleAlwaysFailsWhileTheOracleReverts` A settlement always fails while the market oracle reverts on every call.	✅	🎯
LL-OO-13 `ooLlBorrowCanCompleteAndReadTheOracleWhileItAnswers` A borrow can complete while the oracle answers, with the read recorded.	✅	🎯

LidoLendERC4626

- The vault running its real bytecode; the core, the oracle, the interest model and the tokens linked as CVL models
- Core reads answered by a stub that does not accrue, so the pending-interest regime is uncovered
- Token ledgers holding up to 7 modeled accounts each, decimals drawn between 6 and 18
- Loops running up to 3 iterations
- The vault taken as deployed and initialized, the wiring step itself out of scene
- The vault's share supply capped at uint128 in the conversion, preview and never-revert laws, so supply above that band is uncovered

Valid State

Properties	Status	Mutations
VA-VS-01 `assetIsLoanToken` The vault's asset is always the loan token of its market.	✅	🎯
VA-VS-02 `marketIdCanonical` The vault's market id always matches the market parameters it stores.	✅	🎯🎯
VA-VS-03 `offsetBounded` The vault's virtual share offset never exceeds eighteen decimals.	✅	🎯
VA-VS-04 `managedSharesBackedByCorePosition` The vault never counts more managed shares than its supply position in the core actually holds.	✅	🎯

Variable Transitions

Properties	Status	Mutations
VA-VT-01 `vtVaMarketIdentityImmutable` The vault's controller, market and token pair never change after it is set up.	✅	🎯🎯
VA-VT-02 `vtVaMarketParamsImmutable` The vault's oracle, rate model, loan-to-value limit and decimals offset never change after setup.	✅	🎯
VA-VT-03 `vtVaManagedCounterTracksCoreSupplyMoves` Only the vault's own deposits and exits move the core supply it manages, and by exactly the amount they moved.	✅	🎯

State Transitions

Properties	Status	Mutations
VA-ST-01 `stVaAllowanceSpendOnly` An allowance rises only when its own holder is the caller.	✅	🎯
VA-ST-02 `stVaThirdPartyDebitIsAuthorized` Shares leave a holder only by the holder's own act or within an allowance covering the whole debit.	✅	🎯

High Level

Properties	Status	Mutations
VA-HL-01 `hlVaDepositLeavesOtherBalances` A deposit leaves every share balance other than the receiver's untouched.	✅	🎯
VA-HL-02 `hlVaRedeemLeavesOtherBalances` A redeem leaves every share balance other than the owner's untouched.	✅	🎯
VA-HL-03 `hlVaSettleSpendsAllowanceExactly` Settling for someone else spends exactly the burned shares of their approval, and unlimited approval stays unlimited.	✅	🎯
VA-HL-04 `hlVaPreviewRoundingFavorsVault` An exit quote never costs fewer shares or pays more assets than the matching entry quote.	✅	🎯
VA-HL-05 `hlVaWithdrawBurnsAtMostPreview` A withdraw burns exactly the shares its quote advertised, with no repricing during the call.	✅	🎯🎯🎯
VA-HL-06 `hlVaMintRedeemNoProfit` Minting shares and redeeming them straight back never returns more assets than the mint cost.	⏱️	🎯🔻
VA-HL-07 `hlVaSupplyMovesOnlyOnFlows` Only deposits and mints create shares, and only the withdraw, redeem and settlement paths burn them.	✅	🎯
VA-HL-08 `hlVaSettleBurnsAtMostRequestedShares` A settlement burns at most the shares the caller asked for, all taken from the named owner.	✅🔻	🎯🎯
VA-HL-09 `hlVaSettleBurnsProRataOfConsumedBudget` A settlement burns the requested shares scaled by the position its cuts actually consumed, rounded up.	✅🔻	🎯
VA-HL-10 `hlVaFallbackExitBurnsProRataOfConsumedBudget` With no free liquidity, a fallback exit burns the request scaled by the position its sweep consumed, rounded up.	⏱️	🎯
VA-HL-11 `hlVaSettleDoesNotDiluteRemainingHolders` A settlement never dilutes the holders who stay, since the backing per remaining share never falls.	✅🔻	🎯
VA-HL-12 `hlVaSettlePaysSeizedCollateralToReceiver` A settlement pays every token of seized collateral to the receiver the caller named, and the vault keeps none.	✅	🎯

VA-HL-13 `hlVaFallbackExitPaysSeizedCollateralToReceiver` A fallback exit pays the collateral it seizes to the caller's named receiver, and the vault keeps none.	✅	🎯🔻
VA-HL-14 `hlVaSettleTouchesOnlyNamedBorrowers` A settlement cuts only the borrowers the caller listed, leaving every other borrower's debt and collateral untouched.	✅	🎯
VA-HL-15 `hlVaFallbackExitSettleLegRespectsIterationCap` A fallback exit capped at one round never settles two different borrowers.	✅	🎯🔻
VA-HL-16 `hlVaFallbackExitDoesNotDiluteRemainingHolders` With no free liquidity, a fallback exit still never dilutes the holders who stay.	✅🔻	🎯🔻
VA-HL-17 `hlVaSettleCutsTheNamedBorrower` The collateral a settlement reports as seized comes out of the named borrower's own position.	✅	🎯
VA-HL-18 `hlVaDepositMintsAtLeastPreview` A deposit mints the receiver at least the shares its quote advertised.	✅	🎯
VA-HL-19 `hlVaMintChargesAtMostPreview` A mint charges the caller no more assets than its quote advertised.	✅	🎯
VA-HL-20 `hlVaRedeemPaysAtLeastPreview` A redeem pays the receiver at least the assets its quote advertised.	✅	🎯
VA-HL-21 `hlVaMaxRedeemIsWholeBalanceWhenLiquidityCovers` When positive free liquidity covers an owner's whole position, the vault quotes their exact share balance as redeemable.	✅	🎯
VA-HL-22 `hlVaFallbackExitDeliversPreviewedLoanAssetsWhenLiquidCovers` When free liquidity covers the whole request, a fallback exit reports paying exactly the quoted loan assets.	✅	🎯🔻
VA-HL-23 `hlVaFallbackExitDeliversPreviewedLoanAssetsWhenLiquidPartial` When free liquidity is short of the request, a fallback exit reports paying out exactly the free liquidity.	✅	🎯🔻
VA-HL-24 `hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidCovers` A fallback exit that free liquidity fully covers burns exactly the shares asked for and reports that burn honestly.	✅	🎯🔻🎯🔻
VA-HL-25 `hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial` When free liquidity is short, a fallback exit burns at most the requested shares across both legs and reports it honestly.	⏱️	🎯🔻
VA-HL-26 `hlVaFallbackExitBurnsAtMostRequestedSharesWhenNoLiquidity` With no free liquidity, a fallback exit's settlement leg alone burns at most the requested shares and reports it honestly.	✅🔻	🎯🔻
VA-HL-27 `hlVaMaxRedeemIsConvertedLiquidityWhenShort` When free liquidity falls short, the redeemable quote is that liquidity in shares, not the owner's balance.	✅	🎯🎯
VA-HL-28 `hlVaTotalAssetsPricesTheManagedCounter` The share price counts only the core supply the vault itself made, so outside top ups cannot inflate it.	✅	🎯🎯

VA-HL-29 `hlVaFallbackExitSkipsRedeemLegWhenIlliquid` With no free liquidity, a fallback exit pays out no loan tokens at all.	✅	🎯

ERC4626

Properties	Status	Mutations
ERC4626-01 `assetReturnsConfiguredToken` The vault names the loan token of its own market as its underlying asset.	✅	🎯
ERC4626-02 `assetNeverReverts` Reading the vault's underlying token never fails.	✅	🎯
ERC4626-03 `totalAssetsNeverReverts` Reading the vault's total assets never fails, even with nothing supplied yet.	✅	🎯🎯
ERC4626-04 `convertToSharesSameForAnyCaller` Every caller gets the same share quote for the same amount of assets.	✅	🎯
ERC4626-05 `convertToSharesNeverReverts` Quoting shares for a realistic amount of assets never fails.	✅	🎯
ERC4626-06 `sharesToAssetsRoundTripNeverExceeds` Converting assets into shares and back never returns more assets than were put in.	✅🔻	🎯
ERC4626-07 `convertToAssetsSameForAnyCaller` Every caller gets the same asset quote for the same number of shares.	✅	🎯
ERC4626-08 `convertToAssetsNeverReverts` Quoting assets for a realistic number of shares never fails.	✅	🎯
ERC4626-09 `assetsToSharesRoundTripNeverExceeds` Converting shares into assets and back never returns more shares than were converted.	✅	🎯
ERC4626-10 `depositAboveMaxLimitReverts` A deposit larger than the advertised deposit limit always fails.	✅	🎯
ERC4626-11 `maxDepositNeverReverts` The vault adds no failure of its own when it reports the deposit limit.	✅	🎯
ERC4626-12 `mintAboveMaxLimitReverts` A mint of more shares than the advertised mint limit always fails.	✅	🎯
ERC4626-13 `maxMintNeverReverts` Reading the vault's advertised mint limit never fails.	✅	🎯🎯
ERC4626-14 `withdrawAboveMaxLimitReverts` A withdrawal of more assets than the advertised withdraw limit always fails.	✅	🎯
ERC4626-15 `maxWithdrawNeverReverts` Reading how much a holder may withdraw never fails.	✅	🎯
ERC4626-16 `maxRedeemNeverReverts` Reading how many shares a holder may redeem never fails.	✅🔻	🎯
ERC4626-17 `redeemAboveMaxLimitReverts` A redemption of more shares than the advertised redeem limit always fails.	✅	🎯

ERC4626-18 `previewDepositIncludesFees` A deposit quote never promises more shares than the ideal conversion would give.	✅	🎯
ERC4626-19 `previewMintSameForAnyCaller` Every caller gets the same quote for the assets a mint costs.	✅	🎯
ERC4626-20 `previewMintIncludesFees` A mint quote never understates the assets the minter must pay.	✅	🎯
ERC4626-21 `previewWithdrawSameForAnyCaller` Every caller gets the same quote for the shares a withdrawal burns.	✅	🎯
ERC4626-22 `previewRedeemIncludesFees` A redemption quote never overstates the assets an exiting holder receives.	✅	🎯
ERC4626-23 `depositTransfersAndMintsCorrectly` A deposit charges the caller the named assets and mints the receiver the reported shares.	✅	🎯🎯
ERC4626-24 `mintTransfersAndMintsCorrectly` A mint gives the receiver the requested shares and charges the caller the reported assets.	✅	🎯🎯
ERC4626-25 `mintDecreasesAllowanceByAssetsSpent` A mint spends exactly the charged assets from the caller's token allowance.	✅	🎯
ERC4626-26 `withdrawBurnsAndTransfersCorrectly` A withdrawal burns the reported shares from the owner and pays the receiver the named assets.	✅	🎯🎯
ERC4626-27 `redeemBurnsAndTransfersCorrectly` A redemption burns the named shares from the owner and pays the receiver the reported assets.	✅	🎯
ERC4626-28 `withdrawDecrementsShareAllowanceCorrectly` A third party's withdrawal spends exactly the shares burned from its allowance, and never an unlimited one.	✅	🎯🎯
ERC4626-29 `redeemDecrementsShareAllowanceCorrectly` A third party's redemption spends exactly the shares redeemed from its allowance, and never an unlimited one.	✅	🎯🎯
ERC4626-30 `donationCannotMoveSharePrice` Sending tokens straight to the vault cannot move the price of its shares.	✅	🎯
ERC4626-31 `depositAtAdvertisedMaxDoesNotRevert` A deposit of exactly the advertised deposit limit is never refused, provided it buys at least one share.	⏱	🎯🎯
ERC4626-32 `mintAtAdvertisedMaxDoesNotRevert` A mint of exactly the advertised mint limit is never refused, provided its assets buy a share.	✅	🎯
ERC4626-33 `withdrawAtAdvertisedMaxDoesNotRevert` A withdrawal of exactly the advertised withdrawable amount is never refused.	⏱	🎯
ERC4626-34 `redeemAtAdvertisedMaxDoesNotRevert` A redemption of exactly the advertised redeemable share count is never refused.	⏱	🎯

Reachability

Properties	Status	Mutations
VA-RC-01 `rcVaDepositIsReachable` A deposit can mint shares to the receiver and charge the depositor its loan tokens.	✅	🎯
VA-RC-02 `rcVaWithdrawIsReachable` A lender can withdraw loan tokens to a receiver, burning their own shares.	✅	🎯
VA-RC-03 `rcVaRedeemIsReachable` A lender can redeem shares for a positive payout of loan tokens.	✅	🎯
VA-RC-04 `rcVaSettleIsReachable` A lender can settle against chosen borrowers, burning exactly the shares the call reports.	✅	🎯
VA-RC-05 `rcVaFallbackExitIsReachable` A fallback exit can burn exactly the shares it reports from the owner's balance.	✅	🎯
VA-RC-06 `rcVaFirstDepositIsReachable` The first deposit into an empty vault can create the whole initial share supply.	✅	🎯
VA-RC-07 `rcVaFirstMintIsReachable` The first entry through mint can also seed an empty vault, charging assets for the shares.	✅	🎯
VA-RC-08 `rcVaDepositWithExistingSharesIsReachable` A deposit into a vault that already has shareholders can credit the receiver new shares.	✅	🎯
VA-RC-09 `rcVaDepositOneWeiIsReachable` A deposit of a single unit can still mint shares and charge the depositor exactly that unit.	✅	🎯
VA-RC-10 `rcVaWithdrawMaxIsReachable` A lender can withdraw the full amount the vault advertises as withdrawable.	✅	🎯
VA-RC-11 `rcVaRedeemFullBalanceIsReachable` A lender can redeem their entire share balance and receive a positive payout.	✅	🎯
VA-RC-12 `rcVaFallbackExitFullBalanceIsReachable` A lender can exit their entire share balance through the fallback exit.	✅	🎯
VA-RC-13 `rcVaSettleSingleBorrowerIsReachable` A lender can settle against a single named borrower and still burn shares.	✅	🎯
VA-RC-14 `rcVaSettleMultipleBorrowersIsReachable` A lender can name several borrowers in one settle and still burn shares.	✅	🎯
VA-RC-15 `rcVaFallbackExitLiquidLegIsReachable` The fallback exit can pay the receiver loan tokens out of the market's free liquidity.	✅	🎯
VA-RC-16 `rcVaFallbackExitSingleIterationIsReachable` A fallback exit limited to one settlement round can still seize collateral or cancel debt.	✅	🎯
VA-RC-17 `rcVaDepositThenRedeemIsReachable` A lender can deposit for shares and redeem them back for assets in the same transaction.	✅	🎯
VA-RC-18 `rcVaDepositThenSettleIsReachable` Shares just deposited can immediately fund a settle cut against borrowers.	✅	🎯
VA-RC-19 `rcVaWithdrawOnBehalfIsReachable` An approved third party can withdraw for the owner, spending the owner's allowance.	✅	🎯

VA-RC-20 `rcVaRedeemOnBehalfIsReachable` An approved third party can redeem for the owner, spending the owner's allowance.	✅	🎯
VA-RC-21 `rcVaSettleOnBehalfIsReachable` An approved third party can settle the owner's shares, spending the owner's allowance.	✅	🎯
VA-RC-22 `rcVaMintWithExistingSharesIsReachable` A mint into a vault that already has shareholders can charge assets and credit the requested shares.	✅	🎯
VA-RC-23 `rcVaFallbackExitPartialFillIsReachable` A fallback exit can burn fewer shares than requested instead of reverting.	✅	🎯🔻
VA-RC-24 `rcVaFallbackExitDefaultIterationCapSettles` A fallback exit that asks for zero settlement rounds can still cut a borrower.	✅	🎯
VA-RC-25 `rcVaFallbackExitEmptySweepKeepsLiquidPayout` A fallback exit can keep its liquid payout when the settlement sweep cuts nobody.	✅	🎯

Reverts

Properties	Status	Mutations
VA-RV-01 `rvVaSettleRevertsOnEmptyBorrowers` Settling against an empty borrower list fails rather than doing nothing.	✅	🎯
VA-RV-02 `rvVaSettleRevertsOnZeroShares` A settle of zero shares fails instead of returning an empty result.	✅	🎯
VA-RV-03 `rvVaSettleRevertsOnZeroReceiver` A settle paying the zero address fails, so seized collateral is never sent into the void.	✅	🎯
VA-RV-04 `rvVaSettleRevertsAboveOwnerBalance` Nobody can settle more shares than the share owner actually holds.	✅	🎯
VA-RV-05 `rvVaSettleRevertsOnZeroSettlingShares` A settle whose slice is too small to fund any cut fails.	✅	🎯
VA-RV-06 `rvVaSettleRevertsOnMissingAllowance` A third party settling for an owner fails when its allowance does not cover the burned shares.	✅	🎯
VA-RV-07 `rvVaFallbackExitRevertsOnZeroShares` A fallback exit of zero shares fails instead of returning an empty result.	✅	🎯
VA-RV-08 `rvVaFallbackExitRevertsOnZeroReceiver` A fallback exit paying the zero address fails, so the exiting assets are never sent into the void.	✅	🎯
VA-RV-09 `rvVaFallbackExitRevertsAboveOwnerBalance` Nobody can fallback exit more shares than the share owner actually holds.	✅	🎯
VA-RV-10 `rvVaWithdrawRevertsOnMissingAllowance` A third party withdrawing for an owner fails when its allowance does not cover the shares charged.	✅	🎯
VA-RV-11 `rvVaRedeemRevertsAboveOwnerBalance` Redeeming more shares than the owner holds fails, and a redeem that goes through debits	✅	🎯

exactly those shares.		
VA-RV-12 `rvVaDecimalsNeverReverts` Reading the number of decimals the vault's shares use never fails.	✅	🎯
VA-RV-13 `rvVaPreviewDepositNeverReverts` Quoting a deposit never fails for asset amounts in the supported range.	✅	🎯
VA-RV-14 `rvVaPreviewMintNeverReverts` Quoting a mint never fails for share amounts in the supported range.	✅	🎯
VA-RV-15 `rvVaPreviewWithdrawNeverReverts` Quoting a withdrawal never fails for asset amounts in the supported range.	✅	🎯
VA-RV-16 `rvVaPreviewRedeemNeverReverts` Quoting a redemption never fails for share amounts in the supported range.	✅	🎯
VA-RV-17 `rvVaFallbackExitDoesNotRevertWhenLiquidityCovers` An authorized exit of shares the market's free liquidity fully covers is never refused.	⏱	🎯
VA-RV-18 `rvVaFallbackExitRevertsOnZeroSettlingShares` A fallback exit whose slice funds no settlement budget fails when free liquidity is gone.	✅	🎯
VA-RV-19 `rvVaDepositRevertsWhenItWouldMintNothing` A deposit that pays assets but would mint no shares fails, so nobody pays for nothing.	✅	🎯
VA-RV-20 `rvVaSettleRevertsWhenNothingIsSettled` Every settle that goes through burns shares and consumes the core supply the vault manages, never a silent no-op.	✅🔻	🎯
VA-RV-21 `rvVaFallbackExitRevertsWhenNothingIsSettled` Every fallback exit that goes through burns shares from the owner, never a silent no-op.	✅	🎯
VA-RV-22 `rvVaDepositRevertsWhenCoreMintsNothing` A funded deposit that goes through always buys core supply, so no share is backed by nothing.	✅	🎯

Access Control

Properties	Status	Mutations
VA-AC-01 `acVaAllowanceMovesOnlyByOwnerOrSpender` A share allowance changes only when the caller is its owner or its spender.	✅	🎯

LidoLend_LinkedController

- The core and the controller running their real bytecode, linked; GuardianSignals, the oracle, the interest model and the tokens linked as CVL models
- Settlement queue narrowed to 7 live nodes plus its sentinel
- Core positions holding up to 7 modeled accounts, an eighth holder deleted rather than analyzed
- The core's borrower callbacks summarized away
- Reentrancy rows watching the core's call into the controller only, STATICCALL not watched
- Loops running up to 3 iterations

Valid State

Properties	Status	Mutations
CL-VS-01 `heapKeyFreshCoreBacked` Every queued borrower's stored risk score matches its current debt per unit of collateral.	✅	🎯 🎯
CL-VS-02 `heapMembershipMatchesCoreLifecycle` The settlement queue holds exactly those borrowers who carry both debt and collateral.	✅	🎯
CL-VS-03 `activationBucketsBackedByCoreCollateral` A borrower's inactive collateral never exceeds the collateral the market holds for that borrower.	⏱	🎯

High Level

Properties	Status	Mutations
CL-HL-01 `hlLwcSupplyCollateralCreditMatchesCoreDelta` A collateral deposit is credited in full and stays inactive unless the depositor is trusted.	✅	🎯
CL-HL-02 `hlLwcLiquidateSeizureDrainsBucketsExactly` A liquidation spends a borrower's active collateral first, and drains the inactive buckets only by what the active balance could not cover.	✅	🎯
CL-HL-03 `hlLwcSettleSeizureDrainsBucketsExactly` A settlement spends a borrower's active collateral first, and drains the inactive buckets only by what the active balance could not cover.	✅ 🔻	🎯 🔻
CL-HL-04 `hlLwcBorrowLeavesDebtCoveredByActiveCollateral` A borrow leaves the debt covered by matured collateral at the market's liquidation threshold.	⏱	🎯
CL-HL-05 `hlLwcMaxDepositQuoteMatchesCoreHeadroom` The quoted deposit headroom is the real room left under the supply cap, zero while paused.	✅	🎯
CL-HL-06 `hlLwcMaxDepositIsSuppliableEndToEnd` The market never refuses a deposit of exactly the headroom it quotes.	✅	🎯
CL-HL-07 `hlLwcMaxWithdrawableQuoteMatchesCoreFormula` The quoted withdrawable amount is the lender's own assets capped by idle liquidity, zero while paused.	✅	🎯
CL-HL-08 `hlLwcMaxWithdrawableIsWithdrawableEndToEnd` The market pays out a withdrawal of exactly the amount it quotes as withdrawable.	⏱	🎯
CL-HL-09 `hlLwcWithdrawCollateralDebitsCoreAndHoldsActiveFloor` A collateral withdrawal takes exactly the amount asked and keeps an indebted position at the activation minimum or above.	✅	🎯
CL-HL-10 `hlLwcWithdrawCollateralLeavesDebtCoveredByActiveCollateral` A collateral withdrawal leaves the remaining debt covered by matured collateral at the liquidation threshold.	⏱	🎯
CL-HL-11 `hlLwcSettleStrandsAtMostSubWeiDebt` A settlement that takes all the collateral leaves behind less than one loan asset of debt.	⏱	🎯 🔻
CL-HL-12 `hlLwcMaxWithdrawableQuoteAccrualInvariant`		

Booking interest does not move a lender's withdrawable quote, which already prices the pending interest.	✅	🎯🎯🔻

Reachability

Properties	Status	Mutations
CL-RC-01 `rcLwcSupplyCollateralReachesActivation` A collateral deposit can land in the activation pipeline that holds it before it counts.	✅	🎯
CL-RC-02 `rcLwcBorrowInsertsBorrowerIntoHeap` A borrow can enroll the borrower in the settlement queue.	✅	🎯🎯
CL-RC-03 `rcLwcFullRepayRemovesBorrowerFromHeap` A borrower who repays in full can drop out of the settlement queue.	✅	🎯
CL-RC-04 `rcLwcFullSeizureLiquidationClearsHeapAndDebt` A liquidation can clear a borrower's collateral, debt and queue entry in one call.	✅	🎯
CL-RC-05 `rcLwcSettleAtRootCutsDebt` A settlement aimed at the riskiest queued borrower can pass the policy gates and cut debt.	⏱	🎯
CL-RC-06 `rcLwcWithdrawCollateralReKeysHeap` Pulling collateral can change the borrower's risk score without dropping it from the settlement queue.	✅	🎯
CL-RC-07 `rcLwcSupplyUnderCapSucceeds` A deposit that fits under a live supply cap can go through.	✅	🎯
CL-RC-08 `rcLwcLiveQueueMemberFeasible` A borrower can sit in the settlement queue with a live risk score against real collateral.	✅	🎯
CL-RC-09 `rcLwcBorrowAtHeapCap` A borrower already queued can still borrow when the settlement queue holds the scene's seven live nodes.	✅	🎯
CL-RC-10 `rcLwcBorrowMintsDebtFeasible` A borrower can take on new debt in this market.	✅	🎯
CL-RC-11 `rcLwcSupplyMaxDepositFeasible` A lender can deposit exactly the headroom the market quotes.	✅	🎯
CL-RC-12 `rcLwcMaxWithdrawableQuoteLiveUnderProjection` A lender can be quoted a positive withdrawable amount while interest is still pending.	✅	🎯
CL-RC-13 `rcLwcWithdrawPaysOutThroughNoOpHook` A lender can withdraw and be paid out while the market is running.	✅	🎯
CL-RC-14 `rcLwcAccrueInterestStampsClockThroughNoOpHook` Interest can be booked on the market while it is running.	✅	🎯
CL-RC-15 `rcLwcMaturationUnblocksCoreBorrow` Once its waiting period ends, freshly matured collateral can back a new borrow.	✅	🎯
CL-RC-16 `rcLwcSettleCutsDebtWithIdleLiquidityPresent` A settlement can cut debt even while the market still holds idle liquidity.	⏱	🎯
CL-RC-17 `rcLwcAccrualLiftsFeeRecipientQuote`		

Booking interest can raise the withdrawable quote of the account that collects the protocol fee.	⏱️	🎯

Reverts

Properties	Status	Mutations
CL-RV-01 `rvLwcPausedBlocksAllHookedCoreMutators` A paused market blocks every deposit, withdrawal, borrow, repay, collateral move, liquidation, settlement and interest accrual.	✅	🎯
CL-RV-02 `rvLwcFrozenBlocksCoreBorrow` A frozen market lets nobody take on new debt.	✅	🎯
CL-RV-03 `rvLwcFrozenBlocksDebtBearingCoreWithdrawCollateral` A frozen market blocks a borrower carrying debt from pulling collateral out.	✅	🎯
CL-RV-04 `rvLwcCoreSupplyRevertsOverCap` A deposit that would push supplied assets past the cap fails.	✅	🎯
CL-RV-05 `rvLwcCoreSettleRevertsWhileRootBadDebt` No settlement goes through while the riskiest queued borrower carries bad debt.	✅	🎯
CL-RV-06 `rvLwcBannedCannotBorrowEndToEnd` A ban on the caller, the borrower or the receiver blocks the borrow.	✅	🎯
CL-RV-07 `rvLwcShareAnchoredBorrowRoundingToZeroReverts` A borrow priced in shares that rounds down to zero assets is refused.	✅	🎯
CL-RV-08 `rvLwcCoreSettleRevertsForIneligibleBorrower` A settlement of a borrower the controller bars unwinds the core call entirely.	✅	🎯

Reentrancy

Properties	Status	Mutations
CL-RE-01 `reLwcLatchIsRaisedWhenTheCoreCallsTheController` A collateral deposit can hand control to the real controller with the hook latch raised.	✅	🎯🎯
CL-RE-02 `reLwcControllerNeverGetsControlWithTheLatchDown` A collateral deposit never hands control to the controller with the hook latch down.	✅	🎯

Oracle Outage

Properties	Status	Mutations
CL-OO-01 `ooLwcWithdrawReadsNoOracle` A lender withdrawal through the real controller never reads the market oracle.	✅	🎯
CL-OO-02 `ooLwcRepayReadsNoOracle` A repay with no callback through the real controller never reads the market oracle.	✅	🎯
CL-OO-03 `ooLwcDebtFreeCollateralExitReadsNoOracle` A debt free collateral exit through the real controller reads no oracle.	✅	🎯
CL-OO-04 `ooLwcDebtBearingCollateralExitReadsTheOracleTwice`		

A collateral exit with debt can read the oracle in the controller and in the core.	✅	🎯
CL-OO-05 `ooLwcMaxWithdrawableQuoteReadsNoOracle` The quoted withdrawable amount is produced without reading the market oracle.	✅	🎯
CL-OO-06 `ooLwcMaxDepositQuoteReadsNoOracle` The quoted deposit room is produced without reading the market oracle.	✅	🎯

LidoLendERC4626_LinkedCore

- The vault and the core running their real bytecode; the controller, oracle, interest model and tokens linked as CVL models
- One symbolic market, the per-market mirrors carrying no market key
- Core positions holding up to 7 modeled accounts plus the zero slot, vault share balances up to 7
- Fresh accrual pinned in the rate laws, the deposit delta law and the witnesses that move the vault's position, so pending interest is uncovered there; the frame, token, liquidity, totalAssets and oracle outage rows run unpinned
- The core's borrower callbacks and EIP-712 grant path summarized away, so reentrancy is unanalyzed
- Loops running up to 3 iterations

High Level

Properties	Status	Mutations
VC-HL-01 `hlVcVaultNeverAuthorizesInCore` No ordinary vault call changes who may act for the vault inside the market.	✅	🎯
VC-HL-02 `hlVcDepositSuppliesExactAssetsToCore` A deposit adds exactly its assets to the market's supply side.	✅	🎯
VC-HL-03 `hlVcVaultMovesOnlyItsOwnCorePosition` No ordinary vault call moves a supply position in the market other than the vault's own and the fee recipient's.	✅	🎯🎯
VC-HL-04 `hlVcDepositCreditCappedByPreDepositRate` A deposit buys no more market shares than the price before it allows.	✅	🎯
VC-HL-05 `hlVcDepositCreditReachesPreDepositRateFloor` A deposit is credited every whole market share the pre-deposit price pays for.	✅	🎯
VC-HL-06 `hlVcWithdrawBurnCoversPreWithdrawRate` A withdrawal burns at least the market shares the pre-withdrawal price charges, so leaving never dilutes the holders who stay.	✅	🎯
VC-HL-07 `hlVcWithdrawBurnCappedByPreWithdrawRateCeiling` A withdrawal burns no more market shares than the pre-withdrawal price rounded up, so the leaver is not overcharged.	✅🔻	🎯
VC-HL-08 `hlVcDepositPassesAssetsThroughToCore` A deposit moves the loan token from the depositor straight to the market, with none kept by the vault.	✅	🎯
VC-HL-09 `hlVcTotalAssetsAccrualInvariant` The vault's reported assets already include pending interest, so booking that interest moves nothing.	✅	🎯

VC-HL-10 `hlVcTotalAssetsStableAcrossOwnFeeCredit` The vault's reported assets hold steady across an accrual even when the vault is the market's fee recipient.	✅	🎯

Reachability

Properties	Status	Mutations
VC-RC-01 `rcVcDepositReachesCorePosition` A lender can deposit into the vault and have it land on the vault's own market position.	✅	🎯
VC-RC-02 `rcVcCoreSideDonationMovesSharePrice` A supply made into the market on the vault's behalf can lift its share price.	✅	🎯🎯
VC-RC-03 `rcVcWithdrawDebitsCorePosition` A lender can exit the vault by drawing on its position in the market.	✅	🎯
VC-RC-04 `rcVcVaultExitsWithoutCoreGrant` The vault can exit its own market position while holding no permission grant in the market.	✅	🎯
VC-RC-05 `rcVcAccrualLandsInterestAndFeeCredit` A market accrual can add interest to the supply side and credit the fee recipient.	✅	🎯

Reverts

Properties	Status	Mutations
VC-RV-01 `rvVcWithdrawAboveCoreLiquidityReverts` A withdrawal larger than the market's free liquidity fails.	✅	🎯🎯🎯

Oracle Outage

Properties	Status	Mutations
VC-OO-01 `ooVcWithdrawNeverReadsTheOracle` A vault withdraw never reads the market oracle on its way through the core.	✅	🎯
VC-OO-02 `ooVcRedeemNeverReadsTheOracle` A vault redeem never reads the market oracle on its way through the core.	✅	🎯
VC-OO-03 `ooVcLiquidFallbackExitNeverReadsTheOracle` A fallback exit covered by free liquidity never reads the market oracle.	✅	🎯
VC-OO-04 `ooVcSettleCanReadTheOracle` A vault settlement can read the market oracle, as its design requires.	✅	🎯

Erc4626ExitBook_LinkedVaultCore

- The book running its real bytecode; the vault and the core compiled in beside it for their state, and the controller and the tokens linked as CVL models
- Every vault call the book makes resolving into a CVL ERC4626 model: previews and redeemed amounts are frozen draws, and a redemption moves the vault's asset over the shared token ledger
- One vault anchored to its real storage; any other allow-listed vault is a free model row

- Token ledgers and the vault's share ledger holding up to 7 accounts each
- Loops running up to 3 iterations

Valid State

Properties	Status	Mutations
EE-VS-01 `eeServiceFeeCapped` The protocol's cut of a fill's priority fee never exceeds 25 percent of it.	✅	🎯
EE-VS-02 `eeServiceFeeHasRecipient` A nonzero service fee always has an address on file to collect it.	✅	🎯
EE-VS-03 `eeZeroIdNeverCreated` Id zero never belongs to a request, so it cannot be mistaken for one.	✅	🎯
EE-VS-04 `eeForwardIdsNeverCreated` No request is stored past the last id the book has issued.	✅	🎯
EE-VS-05 `eeIssuedIdsDense` Every id the book has issued still resolves to a request, with no gaps.	✅	🎯
EE-VS-06 `eeUncreatedAssetZero` An id the book never issued carries no payout token.	✅	🎯
EE-VS-07 `eeUncreatedAssetsTotalZero` An id the book never issued carries no requested amount.	✅	🎯
EE-VS-08 `eeUncreatedMinFillAssetsZero` An id the book never issued carries no minimum fill size.	✅	🎯🎯🎯
EE-VS-09 `eeUncreatedStartRateZero` An id the book never issued carries no opening priority fee rate.	✅	🎯
EE-VS-10 `eeUncreatedEndRateZero` An id the book never issued carries no final priority fee rate.	✅	🎯
EE-VS-11 `eeUncreatedRampStartZero` An id the book never issued carries no fee ramp start time.	✅	🎯
EE-VS-12 `eeUncreatedRampEndZero` An id the book never issued carries no fee ramp end time.	✅	🎯
EE-VS-13 `eeUncreatedCreatedAtZero` An id the book never issued carries no creation time.	✅	🎯
EE-VS-14 `eeUncreatedDeadlineZero` An id the book never issued carries no deadline.	✅	🎯
EE-VS-15 `eeUncreatedMaxShareBurnZero` An id the book never issued carries no share burn budget.	✅	🎯🎯
EE-VS-16 `eeUncreatedStatusNotCreated` An id the book never issued reads as unknown, never as open, filled or cancelled.	✅	🎯🎯
EE-VS-17 `eeRemainingImpliesOpen` Only an open request still has assets waiting to exit.	✅	🎯

EE-VS-18 `eeOpenImpliesRemaining` An open request always has assets still waiting to be filled.	✅	🎯🎯
EE-VS-19 `eeRemainingLeTotal` A request never has more left to fill than the lender asked to exit.	✅	🎯🎯
EE-VS-20 `eeCreatedAssetsTotalPositive` Every request in the book asks to exit at least one unit of assets.	✅	🎯
EE-VS-21 `eeUncreatedReceiverZero` An id the book never issued carries no payee.	✅	🎯
EE-VS-22 `eeStartRateLeEndRate` A request's priority fee starts no higher than the ceiling it climbs to.	✅	🎯
EE-VS-23 `eeEndRateCapped` A request's peak priority fee never exceeds 10 percent of the amount filled.	✅	🎯
EE-VS-24 `eeRampStartLeRampEnd` A request's fee ramp never ends before it begins.	✅	🎯
EE-VS-25 `eeRampEndsByDeadline` A request's fee ramp finishes climbing by its deadline, while it can still be filled.	✅	🎯
EE-VS-26 `eeDeadlineAfterCreation` A request's deadline always falls after its creation, so none is born already expired.	✅	🎯
EE-VS-27 `eeDeadlineWithinLifetime` A request never stays fillable longer than the maximum lifetime allowed from its creation.	✅	🎯
EE-VS-28 `eeCreatedAtNotInFuture` A request's creation time is never in the future.	✅	🎯
EE-VS-29 `eeZeroOwnerNoIds` The empty address never accumulates a request history of its own.	✅	🎯
EE-VS-30 `eeOwnerIndexIssued` A lender's request history names only ids the book actually issued.	✅	🎯
EE-VS-31 `eeOwnerIndexOwnership` A lender's request history lists only requests they own.	✅	🎯
EE-VS-32 `eeOwnerIndexStrictlyIncreasing` A lender's request history reads in placement order and never repeats an id.	✅	🎯
EE-VS-33 `eeZeroVaultNeverAllowed` The empty address is never allow-listed as a vault.	✅	🎯
EE-VS-34 `eeCreatedIffVaultBound` A request exists exactly when a vault is bound to redeem it.	✅	🎯
EE-VS-35 `eeVaultAssetBinding` A request pays out in the underlying asset of the vault backing it.	✅	🎯
EE-VS-36 `eeCreatedReceiverSet` Every request names a payee, so a fill never pays into the void.	✅	🎯
EE-VS-37 `eeOwnerIndexLenLeNextId` A lender's history never lists more requests than the book has issued.	✅	🎯

Variable Transitions

Properties	Status	Mutations
EE-VT-01 `vtEeNextIdStepsByOne` The request id counter never moves backward and issues at most one new id per call.	✅	🎯🎯
EE-VT-02 `vtEeReqOwnerFrozenOnceSet` A request's owner is fixed at placement and never changes.	✅	🎯
EE-VT-03 `vtEeClosedStatusTerminal` A request that has been filled or cancelled never changes status again.	✅	🎯
EE-VT-04 `vtEeAssetsRemainingNeverGrows` A request's outstanding amount never grows once the request is placed.	✅	🎯🎯
EE-VT-05 `vtEeTermsFrozenOnceCreated` A request's asset, size, minimum fill, share budget and fee rates are fixed at placement.	✅	🎯
EE-VT-06 `vtEeScheduleFrozenOnceCreated` A request's fee ramp timing, creation time, deadline and payee are fixed at placement.	✅	🎯
EE-VT-07 `vtEeVaultFrozenOnceCreated` The vault a request draws from is fixed at placement and never switches.	✅	🎯🎯
EE-VT-08 `vtEeOwnerIndexAppendOnly` An owner's request list only gains entries, and entries already in it are never rewritten.	✅	🎯
EE-VT-09 `vtEeOwnerIndexAppendsFreshId` An owner's list only grows by appending a newly placed request, and every new request is listed under its owner.	✅	🎯🎯

State Transitions

Properties	Status	Mutations
EE-ST-01 `stEeFilledClosesAtZero` A request is marked filled only when nothing is left to exit on it.	✅	🎯
EE-ST-02 `stEeCancelZeroesRemaining` Cancelling a request leaves nothing outstanding on it.	✅	🎯
EE-ST-03 `stEeMintOwnsNewId` A newly placed request belongs to the caller who placed it.	✅	🎯

High Level

Properties	Status	Mutations
EE-HL-01 `hlEeFillLeavesOtherRequests` Filling one request leaves every other request's outstanding amount untouched.	✅	🎯
EE-HL-02 `hlEePlaceLeavesExistingRequests` Placing a request leaves every already issued request's outstanding amount untouched.	✅	🎯
EE-HL-03 `hlEeCancelLeavesOtherRequests` Cancelling a request leaves every other request's status untouched.	✅	🎯

EE-HL-04 `hlEeRampRateWithinBand` The fee rate a fill charges stays inside the request's own band and under the 10 percent cap.	✅	🎯
EE-HL-05 `hlEeRampRateMonotoneInTime` A request's priority fee rate never falls as the request ages.	✅	🎯
EE-HL-06 `hlEeFillKeepsNoRemainder` A fill pays out everything it withdrew and leaves no assets stuck in the exit book.	✅	🎯
EE-HL-07 `hlEeFillTakesAssetsOutOfItsVault` A fill draws the assets it pays out from the vault the request named.	✅	🎯
EE-HL-08 `hlEePlaceBindsItsNamedVault` A placement books the full amount asked for against exactly the vault the lender named.	✅	🎯🎯

Unit Tests

Properties	Status	Mutations
EE-UT-01 `utEeRampRateHoldsAtStartRate` Before a request's fee ramp begins, its priority fee rate stays at the starting rate.	✅	🎯
EE-UT-02 `utEeRampRateInterpolatesLinearly` Once the ramp begins, a request's priority fee rate rises in a straight line to its end rate.	✅	🎯
EE-UT-03 `utEeRampRatePlateausAtEndRate` Once the ramp is over, a request's priority fee rate stays at its end rate.	✅	🎯
EE-UT-04 `utEeFillPaysReceiverFillAssetsMinusPriorityFee` A fill pays the receiver the amount filled minus the current priority fee.	✅	🎯
EE-UT-05 `utEeFillSplitsFillerRewardAndServiceCut` A fill pays the filler the priority fee less the service cut, and the fee recipient exactly that cut.	✅	🎯
EE-UT-06 `utEeOwnerRequestIdsReadsTheRequestedWindow` Reading an owner's request ids returns the window asked for, starting at the offset given.	✅	🎯

Reachability

Properties	Status	Mutations
EE-RC-01 `rcEePlaceExitRequestIsReachable` A lender can open a new exit request that belongs to them and awaits filling.	✅	🎯
EE-RC-02 `rcEePlaceMinFillAtFullSizeIsReachable` A lender can require their request be filled in one go, all or nothing.	✅	🎯🎯
EE-RC-03 `rcEePlaceOnLinkedVaultIsReachable` A lender can place an exit request backed by the protocol's own lending vault.	✅	🎯
EE-RC-04 `rcEePlaceOnUnlinkedVaultIsReachable` A lender can place an exit request backed by another allow-listed vault.	✅	🎯
EE-RC-05 `rcEePlaceOneAssetIsReachable` A lender can place an exit request for a single unit of the asset.	✅	🎯

EE-RC-06 `rcEePlaceMaxAssetsIsReachable` A lender can place a request for the largest amount a request record can hold.	✅	🎯
EE-RC-07 `rcEePlaceZeroRampIsReachable` A lender can place a request that offers fillers no priority fee at all.	✅	🎯
EE-RC-08 `rcEePlaceDeadlineAtLifetimeCapIsReachable` A lender can set a request's deadline exactly at the maximum allowed lifetime.	✅	🎯
EE-RC-09 `rcEeFillExitRequestIsReachable` A filler can partly fill an open request and leave the rest outstanding.	✅	🎯
EE-RC-10 `rcEeFillMinFillAtFullSizeIsReachable` An all-or-nothing request can be closed by one fill covering its whole size.	✅	🎯🎯
EE-RC-11 `rcEeFillOneAssetStepIsReachable` A fill can take a single unit and leave the request open.	✅	🎯
EE-RC-12 `rcEeFillPaysRequestReceiverIsReachable` Someone other than the owner can fill a request and pay its named receiver.	✅	🎯🎯
EE-RC-13 `rcEeFillPaysServiceFeeIsReachable` A fill under a positive service fee credits the fee recipient.	✅	🎯🎯
EE-RC-14 `rcEeCancelExitRequestIsReachable` An owner can cancel an open request, leaving nothing outstanding on it.	✅	🎯
EE-RC-15 `rcEeCancelPartiallyFilledIsReachable` An owner can still cancel a request a filler has already partly consumed.	✅	🎯
EE-RC-16 `rcEeSetServiceFeeAtCapIsReachable` The fee manager can set the service fee to its 25 percent cap, naming a live recipient.	✅	🎯
EE-RC-17 `rcEeSetServiceFeeDisableIsReachable` The fee manager can switch a positive service fee back off.	✅	🎯
EE-RC-18 `rcEeSetVaultAllowedIsReachable` The allow-list manager can add a vault that was not allowed before.	✅	🎯
EE-RC-19 `rcEeSetVaultDelistIsReachable` The allow-list manager can take a vault off the allowed list.	✅	🎯
EE-RC-20 `rcEePlaceThenCancelIsReachable` A lender can place an exit request and immediately cancel it, leaving nothing to fill.	✅	🎯
EE-RC-21 `rcEePlaceThenFillIsReachable` A request can be filled immediately after a lender places it.	✅	🎯
EE-RC-22 `rcEeAllowThenPlaceIsReachable` A vault allowed a moment ago can immediately back a new exit request.	✅	🎯
EE-RC-23 `rcEeRecoverErc20IsReachable` The recoverer can move stray tokens out of the book to a recipient.	✅	🎯
EE-RC-24 `rcEeRecoverEtherIsReachable` The recoverer can sweep the book's whole ether balance to a recipient.	✅	🎯
EE-RC-25 `rcEeCancelWhilePausedIsReachable` A lender can still cancel an open request while the book is paused.	✅	🎯

EE-RC-26 `rcEeLastRequestIdPositiveIsReachable` A lender's placement can raise the last issued request id by one.	✅	🎯
EE-RC-27 `rcEeOwnerRequestIndexViewsIsReachable` A lender can read a freshly placed request back as the last entry of their own list.	✅	🎯
EE-RC-28 `rcEePlaceRampedRatesIsReachable` A lender can place a request whose priority fee rate climbs over time.	✅	🎯
EE-RC-29 `rcEeFillFullCloseIsReachable` A single fill can take the whole remainder and close the request as filled.	✅	🎯🎯
EE-RC-30 `rcEeSetServiceFeeClearsRecipientIsReachable` The fee manager can clear the fee recipient and leave the fee at zero.	✅	🎯
EE-RC-31 `rcEePauseResumeIsReachable` A paused book can be started again, and its record reads answer while paused.	✅	🎯

Reverts

Properties	Status	Mutations
EE-RV-01 `rvEeSetVaultAllowedRevertsIff` Setting a vault's allow-list flag fails exactly when the caller lacks the role or the vault is zero.	✅	🎯
EE-RV-02 `rvEeSetServiceFeeRevertsIff` Setting the service fee fails exactly for an unauthorized caller, a fee above its cap, or a nonzero fee with no recipient.	✅	🎯
EE-RV-03 `rvEePlaceExitRequestRevertsIff` Placing an exit request fails exactly when a pause, allow-list, backing, deadline or size check rejects it.	✅	🎯
EE-RV-04 `rvEeFillRevertsWhenExpired` Filling an exit request after its deadline has passed always fails.	✅	🎯
EE-RV-05 `rvEeFillRevertsOnAmountGuards` A fill fails when the book is paused, the reward recipient is empty, or its size sits outside the request's window.	✅	🎯🎯
EE-RV-06 `rvEeCancelExitRequestRevertsIff` Cancelling an exit request fails exactly when the caller is not its owner or it is not open.	✅	🎯
EE-RV-07 `rvEeRecoverEtherRevertsOnGuards` Sweeping the book's ether fails when the caller is not the recoverer or the recipient is zero.	✅	🎯
EE-RV-08 `rvEeRecoverErc20RevertsIff` Recovering a stray ERC-20 balance fails exactly when the caller is not the recoverer or the recipient is zero.	✅	🎯
EE-RV-09 `rvEeRecoverErc721RevertsIff` Recovering a stray NFT fails exactly when the caller is not the recoverer or the recipient is zero.	✅	🎯
EE-RV-10 `rvEeRecoverErc1155RevertsIff` Recovering a stray ERC-1155 balance fails exactly when the caller is not the recoverer or the recipient is zero.	✅	🎯

EE-RV-11 `rvEeFeeRecipientNeverReverts` Reading the fee recipient always answers, and reports the address the fee manager last set.	✅	🎯
EE-RV-12 `rvEeServiceFeeNeverReverts` Reading the service fee always answers, and reports the rate the fee manager last set.	✅	🎯
EE-RV-13 `rvEeIsVaultAllowedNeverReverts` Reading a vault's allow flag always answers, and reports what the allow-list manager last set.	✅	🎯
EE-RV-14 `rvEeVaultOfNeverReverts` Reading a request's vault always answers, and reports the vault the placement named.	✅	🎯
EE-RV-15 `rvEeGetExitRequestNeverReverts` Reading a single request record by id never fails.	✅	🎯
EE-RV-16 `rvEeGetRequestsNeverReverts` Reading a batch of request records never fails.	✅	🎯🎯
EE-RV-17 `rvEeGetOwnerRequestIdsNeverReverts` Reading a page of one lender's request ids never fails.	✅	🎯

Access Control

Properties	Status	Mutations
EE-AC-01 `acEeOnlyRequestOwnerCancels` An exit request is cancelled only by the lender who owns it.	✅	🎯🎯
EE-AC-02 `acEeOwnerIndexMovesOnlyBySelf` An owner's list of request ids changes only when that owner is the caller.	✅	🎯
EE-AC-03 `acEeOnlyFeeManagerSetsFeeConfig` Only the fee manager can change the service fee or its recipient.	✅	🎯
EE-AC-04 `acEeOnlyAllowlistManagerSetsVaultAllowlist` Only the allow-list manager can change which vaults may back exit requests.	✅	🎯
EE-AC-05 `acEeOnlyRoleAdminMovesRoles` Only a role's admin can grant or revoke it, apart from a holder giving up their own.	✅	🎯🎯
EE-AC-06 `acEeOnlyRecovererSweepsEther` The book's ether balance falls only when the recoverer sweeps it.	✅	🎯

MorphoExitBook_LinkedCore

- The core running its real bytecode; the controller, oracle, interest model and tokens linked as CVL models
- Fresh accrual pinned in the pricing, preview and fill-success rules, so the pending-interest regime is uncovered there
- Token layer a CVL model whose transfers succeed, each token ledger holding up to 7 accounts
- Loops running up to 3 iterations

Valid State

Properties	Status	Mutations

ME-VS-01 `meServiceFeeCapped` The protocol's cut of a fill's priority fee never exceeds 25 percent of it.	✅	🎯
ME-VS-02 `meServiceFeeHasRecipient` Whenever a service fee is charged, someone is named to receive it.	✅	🎯🎯
ME-VS-03 `meZeroIdNeverCreated` Id zero never belongs to a request, so it cannot be mistaken for one.	✅	🎯
ME-VS-04 `meForwardIdsNeverCreated` No request exists beyond the last id the book has issued.	✅	🎯
ME-VS-05 `meIssuedIdsDense` Every id the book has issued resolves to a real request, with no gaps.	✅	🎯
ME-VS-06 `meUncreatedAssetZero` An id the book never issued carries no payout token.	✅	🎯
ME-VS-07 `meUncreatedAssetsRemainingZero` An id the book never issued carries nothing left to fill.	✅	🎯
ME-VS-08 `meUncreatedAssetsTotalZero` An id the book never issued carries no requested amount.	✅	🎯
ME-VS-09 `meUncreatedMinFillZero` An id the book never issued carries no minimum fill size.	✅	🎯
ME-VS-10 `meUncreatedStartRateZero` An id the book never issued carries no opening priority fee rate.	✅	🎯
ME-VS-11 `meUncreatedEndRateZero` An id the book never issued carries no final priority fee rate.	✅	🎯
ME-VS-12 `meUncreatedRampStartZero` An id the book never issued carries no fee ramp start time.	✅	🎯
ME-VS-13 `meUncreatedRampEndZero` An id the book never issued carries no fee ramp end time.	✅	🎯
ME-VS-14 `meUncreatedCreatedAtZero` An id the book never issued carries no creation time.	✅	🎯
ME-VS-15 `meUncreatedDeadlineZero` An id the book never issued carries no deadline.	✅	🎯
ME-VS-16 `meUncreatedMaxShareBurnZero` An id the book never issued carries no share burn budget.	✅	🎯
ME-VS-17 `meUncreatedStatusNotCreated` An id the book never issued reads as unknown, never as open, filled or cancelled.	✅	🎯
ME-VS-18 `meRemainingImpliesOpen` Only an open request still has assets waiting to exit.	✅	🎯🎯🎯
ME-VS-19 `meOpenImpliesRemaining` An open request always has assets still waiting to be filled.	✅	🎯
ME-VS-20 `meRemainingLeTotal` A request never has more left to fill than the lender asked to exit.	✅	🎯

ME-VS-21 `meCreatedAssetsTotalPositive` Every request in the book asks to exit at least one unit of assets.	✅	🎯
ME-VS-22 `meStartRateLeEndRate` A request's priority fee never starts above the rate its ramp ends at.	✅	🎯
ME-VS-23 `meEndRateCapped` A request's peak priority fee never exceeds 10 percent of the amount filled.	✅	🎯
ME-VS-24 `meRampStartLeRampEnd` A request's fee ramp never ends before it begins.	✅	🎯
ME-VS-25 `meRampEndsByDeadline` A request's fee ramp finishes climbing by its deadline, while it can still be filled.	✅	🎯
ME-VS-26 `meDeadlineAfterCreation` A request's deadline always falls after its creation, so none is born already expired.	✅	🎯
ME-VS-27 `meDeadlineWithinLifetime` A request stays fillable for at most the book's configured maximum lifetime.	✅	🎯
ME-VS-28 `meCreatedAtNotInFuture` A request's creation time is never in the future.	✅	🎯
ME-VS-29 `meZeroOwnerNoIds` The zero address never accumulates a request listing of its own.	✅	🎯
ME-VS-30 `meOwnerIndexIssued` An owner's listing only points at request ids the book actually issued.	✅	🎯
ME-VS-31 `meOwnerIndexOwnership` An owner's listing never shows anyone else's request.	✅	🎯
ME-VS-32 `meOwnerIndexStrictlyIncreasing` An owner's listing keeps placement order and never repeats a request.	✅	🎯
ME-VS-33 `meZeroMarketIdNeverAllowed` The empty market id is never on the allow-list.	✅	🎯
ME-VS-34 `meCreatedIffMarketBound` A request exists exactly when it is bound to a market.	✅	🎯
ME-VS-35 `meAllowedMarketHasParamsRow` A market is never allow-listed without its full description recorded.	✅	🎯
ME-VS-36 `meBoundMarketHasParamsRow` Every request still resolves to its market's description, even after that market is delisted.	✅	🎯
ME-VS-37 `meCreatedAssetIsMarketLoanToken` A request always pays out in the loan token of its own market.	✅	🎯
ME-VS-38 `meParamsRowMarketExistsOnCore` Every market the book registers really exists on the lending core.	✅	🎯
ME-VS-39 `meMinFillWithinTotal` A request's minimum fill never exceeds the total amount it asks to exit.	✅	🎯
ME-VS-40 `meOwnerIndexLenLeNextId`	✅	🎯

No owner's listing holds more requests than the book has ever issued.		
ME-VS-41 `meParamsRowCanonical` A stored market description always matches the id it is filed under.	✅	🎯
ME-VS-42 `meUncreatedReceiverZero` An id the book never issued carries no payee.	✅	🎯
ME-VS-43 `meCreatedReceiverSet` Every request names a payee, so a fill never pays into the zero address.	✅	🎯

Variable Transitions

Properties	Status	Mutations
ME-VT-01 `vtMeNextIdIncrementsByOne` The request id counter never moves backward and issues at most one new id per call.	✅	🎯🎯
ME-VT-02 `vtMeRequestOwnerImmutable` A request's owner is fixed at placement and never changes.	✅	🎯🎯
ME-VT-03 `vtMeClosedStatusTerminal` A request that has been filled or cancelled never changes status again.	✅	🎯
ME-VT-04 `vtMeAssetsRemainingNonIncreasing` A request's outstanding amount never grows once the request is placed.	✅	🎯🎯
ME-VT-05 `vtMeRequestTermsFrozen` A request's asset, size, minimum fill, share budget and fee rates are fixed at placement.	✅	🎯🎯🎯
ME-VT-06 `vtMeRequestScheduleFrozen` A request's schedule, payee and backing market are fixed at placement.	✅	🎯🎯🎯🎯
ME-VT-07 `vtMeOwnerIndexAppendOnly` An owner's request list only grows by one entry at a time and never rewrites earlier entries.	✅	🎯
ME-VT-08 `vtMeOwnerIndexAppendsFreshId` An owner's list grows only by appending a freshly placed id, and every new request is listed under its owner.	✅	🎯
ME-VT-09 `vtMeParamsRowLatches` A market registered with the exit book keeps its record forever, even after it is de-listed.	✅	🎯
ME-VT-10 `vtMeParamsRowValuesFrozen` A registered market's recorded parameters never change, so one market id always means one market.	✅	🎯
ME-VT-11 `vtMeCoreMarketStaysCreated` No exit book call makes the lending core forget that a market was created.	✅	🎯

State Transitions

Properties	Status	Mutations
ME-ST-01 `stMeRemainingMovesOnlyOpen` Only an open request can have its outstanding amount move.	✅	🎯

ME-ST-02 `stMeMintOwnsNewId` A newly placed request belongs to the caller who placed it.	✅	🎯
ME-ST-03 `stMeMintIndexesOwner` Placing a request adds exactly one entry to the caller's own request list.	✅	🎯

High Level

Properties	Status	Mutations
ME-HL-01 `hlMeFillLeavesOtherRequests` Filling one request leaves every other request's outstanding amount untouched.	✅	🎯
ME-HL-02 `hlMePlaceLeavesExistingRequests` Placing a request leaves every already issued request's outstanding amount untouched.	✅	🎯
ME-HL-03 `hlMeCancelLeavesOtherRequests` Cancelling a request leaves every other request's status untouched.	✅	🎯
ME-HL-04 `hlMeRampRateWithinBand` The fee rate a fill charges stays inside the request's own band and under the 10 percent cap.	✅	🎯
ME-HL-05 `hlMeRampRateMonotoneInTime` A request's priority fee rate never falls as the request ages.	✅	🎯
ME-HL-06 `hlMeFillKeepsNoRemainder` A fill pays out everything it withdrew and leaves no assets stuck in the exit book.	✅	🎯
ME-HL-07 `hlMeFillTouchesOnlyItsOwnMarket` A fill touches only the market its request was placed against, leaving other markets and their positions alone.	✅	🎯
ME-HL-08 `hlMeFillDebitsExactlyWithinShareBudget` A fill debits the request by exactly the amount filled and burns no more of the lender's supply than budgeted.	✅	🎯
ME-HL-09 `hlMePlaceBindsItsOwnMarket` Placing a request binds it to the market named, in that market's loan token, for the full amount asked.	✅	🎯
ME-HL-10 `hlMeFillTakesAssetsOutOfItsMarket` A fill draws the assets it pays out from the market the request was placed against.	✅	🎯

Unit Tests

Properties	Status	Mutations
ME-UT-01 `utMeRampRateHoldsAtStartRate` Before a request's fee ramp begins, its priority fee rate stays at the starting rate.	✅	🎯
ME-UT-02 `utMeRampRateInterpolatesLinearly` Once the ramp begins, a request's priority fee rate rises in a straight line to its end rate.	✅	🎯
ME-UT-03 `utMeRampRatePlateausAtEndRate` Once the ramp is over, a request's priority fee rate stays at its end rate.	✅	🎯

ME-UT-04 `utMeFillPaysReceiverFillLessPriorityFee` A fill pays the receiver the amount filled minus the current priority fee.	✅	🎯
ME-UT-05 `utMeFillSplitsFillerRewardAndServiceCut` A fill pays the filler the priority fee less the service cut, and the fee recipient exactly that cut.	✅	🎯
ME-UT-06 `utMeFillLeavesTheBookWithoutACorePosition` A fill never changes the exit book's own supply position in the lending market.	✅	🎯
ME-UT-07 `utMeOwnerRequestIdsReadsTheRequestedWindow` Reading an owner's request ids returns the window asked for, starting at the offset given.	✅	🎯
ME-UT-08 `utMeSupplyQuoteRoundsUpAtTheLivePrice` The backing a placement demands is the amount asked for, converted to shares at the live price and rounded up.	✅	🎯

Reachability

Properties	Status	Mutations
ME-RC-01 `rcMePlaceExitRequestIsReachable` A lender can open a request on an allow-listed market and own it.	✅	🎯
ME-RC-02 `rcMePlaceWithMinFillIsReachable` A lender can place a request that sets a floor under every partial fill.	✅	🎯
ME-RC-03 `rcMePlaceOneWeiAssetIsReachable` A lender can place a request for a single unit of assets.	✅	🎯
ME-RC-04 `rcMePlaceMaxUint128AssetsIsReachable` A lender can place a request as large as the request record can hold.	✅	🎯
ME-RC-05 `rcMePlaceRampedRatesIsReachable` A lender can place a request whose priority fee rate climbs strictly over time.	✅	🎯
ME-RC-06 `rcMePlaceZeroRampIsReachable` A lender can place a request that offers fillers no priority fee at all.	✅	🎯
ME-RC-07 `rcMePlaceDeadlineAtLifetimeCapIsReachable` A lender can set a deadline exactly at the maximum request lifetime.	✅	🎯
ME-RC-08 `rcMeFillPartialIsReachable` A filler can fill part of a request and leave it open for more.	✅	🎯
ME-RC-09 `rcMeFillFullCloseIsReachable` A filler can take the whole remaining amount and close the request as filled.	✅	🎯
ME-RC-10 `rcMeFillPaysReceiverIsReachable` A fill can credit the exit proceeds to the receiver the lender named.	✅	🎯
ME-RC-11 `rcMeFillPaysServiceFeeIsReachable` A fill under a positive service fee can credit the fee recipient its cut.	✅	🎯🎯
ME-RC-12 `rcMeCancelExitRequestIsReachable` An owner can cancel an open request, leaving nothing outstanding on it.	✅	🎯
ME-RC-13 `rcMeCancelLastWeiIsReachable` An owner can still cancel a partly filled request with one unit left.	✅	🎯

ME-RC-14 `rcMeSetServiceFeeIsReachable` The fee manager can set a positive service fee and its recipient.	✅	🎯
ME-RC-15 `rcMeSetServiceFeeAtCapIsReachable` The fee manager can set the service fee to its 25 percent cap.	✅	🎯
ME-RC-16 `rcMeSetServiceFeeDisableIsReachable` The fee manager can switch a positive service fee back off.	✅	🎯
ME-RC-17 `rcMeSetMarketAllowedIsReachable` The allow-list manager can open a market for exits and record its parameters.	✅	🎯
ME-RC-18 `rcMeSetMarketDelistIsReachable` The allow-list manager can remove a market from the allow list.	✅	🎯
ME-RC-19 `rcMeRecoverERC20IsReachable` The recoverer can send stray tokens held by the book to a recipient.	✅	🎯
ME-RC-20 `rcMeRecoverEtherIsReachable` The recoverer can empty the book's entire ether balance in one sweep.	✅	🎯
ME-RC-21 `rcMePlaceThenCancelIsReachable` A lender can cancel a request in the same block it was placed.	✅	🎯
ME-RC-22 `rcMeAllowThenPlaceIsReachable` A market just added to the allow list can immediately back a new request.	✅	🎯
ME-RC-23 `rcMeTwoOwnersCoexistIsReachable` Requests from two different owners can stand in the book at the same time.	✅	🎯
ME-RC-24 `rcMeLastRequestIdPositiveIsReachable` The book can report a positive last request id, taken from its own id counter.	✅	🎯
ME-RC-25 `rcMeOwnerRequestCountPositiveIsReachable` The book can report a positive request count for an owner, matching that owner's stored list.	✅	🎯
ME-RC-26 `rcMeMarketParamsOfResolvesIsReachable` A newly placed request can resolve to the parameters of the market it named.	✅	🎯
ME-RC-27 `rcMeCancelWhilePausedIsReachable` A lender can still cancel an open request while the book is paused.	✅	🎯
ME-RC-28 `rcMeFillBelowMinFillClosesRequestIsReachable` A fill below the request's minimum can still go through when it closes the whole remainder.	✅	🎯
ME-RC-29 `rcMeFillLeavesTheOtherMarketIsReachable` A fill can go through while a second funded market stands untouched.	✅	🎯
ME-RC-30 `rcMeSetServiceFeeClearsRecipientIsReachable` The fee manager can clear the fee recipient and leave the fee at zero.	✅	🎯
ME-RC-31 `rcMeFillAtExactShareBudgetIsReachable` A fill can go through when its share burn exactly meets the request's own limit.	✅	🎯
ME-RC-32 `rcMePauseResumeIsReachable` The book can be stopped and started again, and its reads answer while stopped.	✅	🎯

Reverts

Properties	Status	Mutations
ME-RV-01 `rvMeSetMarketAllowedRevertsIff` Changing a market's allow-list status fails exactly when the caller lacks the role or the market does not exist.	✅	🎯
ME-RV-02 `rvMeSetServiceFeeRevertsIff` Setting the service fee fails exactly when the caller lacks the role, the fee exceeds its cap, or no recipient is named.	✅	🎯
ME-RV-03 `rvMePlaceExitRequestRevertsWhenMarketNotAllowed` Placing an exit request against a market off the allow list fails.	✅	🎯
ME-RV-04 `rvMePlaceExitRequestRevertsWhenNotAuthorizedOnCore` Placing an exit request fails when the lender has not authorized the book over their position.	✅	🎯
ME-RV-05 `rvMePlaceExitRequestRevertsOnInsufficientBacking` Placing an exit request fails when the lender's supply position does not cover the amount.	⏱	🎯
ME-RV-06 `rvMePlaceExitRequestRevertsOnZeroAssets` Placing an exit request for a zero amount always fails.	✅	🎯
ME-RV-07 `rvMePlaceExitRequestRevertsOnHighPriorityFee` Placing a request whose end fee rate exceeds the 10 percent cap fails.	✅	🎯
ME-RV-08 `rvMePlaceExitRequestRevertsOnStartAboveEnd` Placing a request whose fee ramp starts above where it ends fails.	✅	🎯
ME-RV-09 `rvMePlaceExitRequestRevertsOnRampStartAfterEnd` Placing a request whose ramp start time is after its ramp end fails.	✅	🎯
ME-RV-10 `rvMePlaceExitRequestRevertsOnDeadlineInPast` Placing a request with a deadline not in the future fails.	✅	🎯
ME-RV-11 `rvMePlaceExitRequestRevertsOnDeadlineTooFar` Placing a request with a deadline past the maximum request lifetime fails.	✅	🎯
ME-RV-12 `rvMePlaceExitRequestRevertsOnRampPastDeadline` Placing a request whose fee ramp ends after its deadline fails.	✅	🎯
ME-RV-13 `rvMePlaceExitRequestRevertsOnAssetsCastOverflow` Placing a request bigger than the stored amount field can hold fails.	✅	🎯
ME-RV-14 `rvMePlaceExitRequestRevertsOnDeadlineCastOverflow` Placing a request with a deadline beyond the stored deadline field's range fails.	✅	🎯
ME-RV-15 `rvMePlaceExitRequestRevertsOnRampCastOverflow` Placing a request with ramp offsets beyond their stored fields' range fails.	✅	🎯
ME-RV-16 `rvMePlaceValidRequestSucceeds` A request that meets every placement condition is always accepted.	✅	🎯
ME-RV-17 `rvMeFillExitRequestRevertsWhenExpired` Filling an exit request after its deadline has passed always fails.	✅	🎯

ME-RV-18 `rvMeFillExitRequestRevertsWhenAuthorizationRevoked` Filling fails once the request's owner revokes the book's permission over their position.	✅	🎯
ME-RV-19 `rvMeFillExitRequestRevertsOnZeroBacking` Filling a request fails when its owner, unless it is the market's fee recipient, holds no supply position left.	✅	🎯
ME-RV-20 `rvMeFillValidRequestSucceeds` A backed, unexpired fill of an open request succeeds while the market carries no debt.	✅	🎯
ME-RV-21 `rvMeCancelExitRequestRevertsIff` Cancelling an exit request fails exactly when the caller is not its owner or it is not open.	✅	🎯
ME-RV-22 `rvMeRecoverEtherRevertsOnNotRecovererOrZeroRecipient` Sweeping the book's ether fails when the caller is not the recoverer or the recipient is zero.	✅	🎯
ME-RV-23 `rvMeRecoverTokenRevertsIff` Recovering a stray ERC-20 balance fails exactly when the caller is not the recoverer or the recipient is zero.	✅	🎯
ME-RV-24 `rvMeRecoverNftRevertsIff` Recovering a stray NFT fails exactly when the caller is not the recoverer or the recipient is zero.	✅	🎯
ME-RV-25 `rvMeRecoverMultiTokenRevertsIff` Recovering a stray ERC-1155 balance fails exactly when the caller is not the recoverer or the recipient is zero.	✅	🎯
ME-RV-26 `rvMeGetRequestsNeverReverts` Reading several exit requests in one call never fails.	✅	🎯🎯
ME-RV-27 `rvMeGetOwnerRequestIdsNeverReverts` Reading a page of an owner's request ids never fails, an out-of-range start included.	✅	🎯
ME-RV-28 `rvMePlaceExitRequestRevertsWhenPaused` A lender cannot place a new exit request while the book is stopped.	✅	🎯
ME-RV-29 `rvMeFillExitRequestRevertsWhenPaused` No one can fill an exit request while the book is stopped.	✅	🎯
ME-RV-30 `rvMeFillExitRequestRevertsOnPartialAtWholeTotalFloor` A request whose floor is its whole amount takes only a fill of the whole remainder.	✅	🎯

Access Control

Properties	Status	Mutations
ME-AC-01 `acMeOnlyRequestOwnerCancels` An exit request is cancelled only by the lender who owns it.	✅	🎯
ME-AC-02 `acMeOwnerIndexMovesOnlyForCaller` An owner's list of request ids changes only when that owner is the caller.	✅	🎯
ME-AC-03 `acMeOnlyFeeManagerSetsFeeConfig` Only the fee manager can change the service fee or its recipient.	✅	🎯
ME-AC-04 `acMeOnlyManagerMovesMarketAllowlist` Only the allow-list manager can add or remove a market from the allow-list.	✅	🎯

ME-AC-05 `acMeOnlyManagerWritesParamsRow` Only the allow-list manager can write a market's registered parameters.	✅	🎯
ME-AC-06 `acMeOnlyRoleAdminMovesRoles` Only a role's admin can grant or revoke it, apart from a holder giving up their own.	✅	🎯
ME-AC-07 `acMeOnlyRecovererSweepsEther` The book's ether balance falls only when the recoverer sweeps it.	✅	🎯

Oracle Outage

Properties	Status	Mutations
ME-OO-01 `ooMeFillNeverReadsTheOracle` Filling an exit request never reads the oracle of its own market.	✅	🎯

AdaptiveCurveIrm

- The rate model running its real bytecode; the router, the core behind it and every curve config linked as CVL models
- Curve configs answered by a model held inside the bands the config constructor enforces, so an applied config is assumed to be one its constructor validated
- `ExpLib.wExp` replaced by a bounded deterministic function, not assumed monotone
- WAD multiply and divide computed over unbounded integers, so their overflow reverts are uncovered
- The forwarded market pinned to borrowed at most supplied and a last update no later than now, except in the permissionless view rules

Valid State

Properties	Status	Mutations
ACI-VS-01 `vsAciRateWithinItsConfigBand` A priced market's rate always sits inside the bounds of the config it points at.	✅	🎯 🎯
ACI-VS-02 `vsAciPricedMarketHasConfig` A market that carries a rate always has a config attached.	✅	🎯
ACI-VS-03 `vsAciRolesNeverZero` The rate model always has a live owner and a live manager.	✅	🎯 🎯
ACI-VS-04 `vsAciZeroIdNeverConfigured` The zero market id never carries a rate or a config.	✅	🎯

Variable Transitions

Properties	Status	Mutations
ACI-VT-01 `vtAciConfiguredMarketNeverUnconfigures` A market that has a rate configuration never loses it, though the configuration can be switched.	✅	🎯
ACI-VT-02 `vtAciPricedMarketNeverUnprices` Once a market has a target rate, it never falls back to being unpriced.	✅	🎯

State Transitions

Properties	Status	Mutations
ACI-ST-01 `stAciFreshConfigNeverPricesMarket` Giving a market its first configuration leaves it unpriced until the first rate call.	✅	🎯
ACI-ST-02 `stAciConfigRenewalReclampsRateIntoNewBand` Applying a configuration to a priced market pulls its target rate into that configuration's band.	✅	🎯
ACI-ST-03 `stAciReclampBandComesFromTheNewConfig` The new target rate comes from the band of the configuration being applied, not the one replaced.	✅	🎯

High Level

Properties	Status	Mutations
ACI-HL-01 `hlAciBorrowRateWithinAmplifiedConfigBand` The quoted borrow rate stays inside the band the market's configuration allows, widened by its curve steepness.	⏱	🎯
ACI-HL-02 `hlAciBorrowRateCappedByMaxBorrowRate` When a configuration sets a borrow rate ceiling, the quoted rate never exceeds it.	✅	🎯
ACI-HL-03 `hlAciBorrowRateMonotoneInUtilization` The quoted borrow rate never falls when more of a market's supply is borrowed.	⏱	🎯
ACI-HL-04 `hlAciBorrowRateMatchesView` The previewed borrow rate equals the rate the writing call charges on the same state.	✅	🎯
ACI-HL-05 `hlAciBorrowRateWithinAmplifiedBandOnFirstInteraction` On a market's very first pricing, the quoted rate stays inside its configuration's steepness widened band.	✅	🎯
ACI-HL-06 `hlAciBorrowRateMonotoneInUtilizationWithoutAdaptation` With no time elapsed, the quoted rate still never falls as more is borrowed.	⏱	🎯

Unit Tests

Properties	Status	Mutations
ACI-UT-01 `utAciTransferOwnershipInstallsOwner` Handing over ownership installs the new owner and leaves the manager in place.	✅	🎯🎯
ACI-UT-02 `utAciSetManagerInstallsManager` Appointing a new manager installs that manager and leaves ownership unchanged.	✅	🎯🎯
ACI-UT-03 `utAciApplyIrmConfigBindsMarket` Applying a config leaves the market with a config attached and any existing rate at or above the new floor.	✅	🎯
ACI-UT-04 `utAciApplyIrmConfigTouchesOnlyTargetMarket` Applying a config to one market leaves every other market's rate and config untouched.	✅	🎯

ACI-UT-05 `utAciBorrowRateLeavesMarketPricedAndCapped` Pricing a market leaves it priced under the same config and never returns a rate above its ceiling.	✅	🎯🎯
ACI-UT-06 `utAciBorrowRateTouchesOnlyTargetMarket` Pricing one market leaves every other market's stored rate untouched.	✅	🎯
ACI-UT-07 `utAciBorrowRateViewIsCallerAgnostic` The quoted borrow rate is the same for every caller at the same moment.	✅	🎯🎯
ACI-UT-08 `utAciApplyIrmConfigStoresSuppliedConfigAddress` A market ends up pointing at exactly the config address that was applied.	✅	🎯🎯
ACI-UT-09 `utAciApplyIrmConfigKeepsAnInBandRateUnchanged` A rate already inside the new bounds survives a config swap unchanged.	✅	🎯

Config Trust Boundary

Properties	Status	Mutations
ACI-TB-01 `tbAciUnvalidatedConfigLiftsRateAboveTheSharedCeiling` A config the constructor would have rejected can push a live market's target rate above the shared ceiling.	✅	🎯

Reachability

Properties	Status	Mutations
ACI-RC-01 `rcAciBorrowRateFirstInteractionIsReachable` The router can price a newly configured market and give it a first target rate.	✅	🎯
ACI-RC-02 `rcAciBorrowRateAdaptsUpwardIsReachable` Pricing can carry a market's target rate upward when borrowing sits above target.	✅	🎯
ACI-RC-03 `rcAciBorrowRateAdaptsDownwardIsReachable` Pricing can carry a market's target rate downward when borrowing sits below target.	✅	🎯
ACI-RC-04 `rcAciBorrowRateReachesMaxClampIsReachable` A target rate below its ceiling can rise all the way onto that ceiling.	✅	🎯
ACI-RC-05 `rcAciBorrowRateReachesMinClampIsReachable` A target rate above its floor can fall all the way onto that floor.	✅	🎯
ACI-RC-06 `rcAciBorrowRateAtZeroUtilizationIsReachable` A market with nothing borrowed can still be quoted a positive interest rate.	✅	🎯
ACI-RC-07 `rcAciBorrowRateAtFullUtilizationIsReachable` A fully borrowed market can still be quoted a positive interest rate.	✅	🎯
ACI-RC-08 `rcAciBorrowRateViewIsReachable` Anyone can preview a positive borrow rate for a configured market.	✅	🎯
ACI-RC-09 `rcAciBorrowRateViewAboveWadUtilizationIsReachable` It is possible to preview a rate for a market that borrowed more than it holds.	✅	🎯
ACI-RC-10 `rcAciApplyIrmConfigOnFreshMarketIsReachable` The owner or the manager can give a market its first rate configuration.	✅	🎯

ACI-RC-11 `rcAciApplyIrmConfigReconfigureIsReachable` The owner or the manager can move a live market onto a different configuration, and it stays priced.	✅	🎯

Reverts

Properties	Status	Mutations
ACI-RV-01 `rvAciTransferOwnershipRevertsIff` Handing over ownership fails exactly when the caller is not the owner or the new owner is zero.	✅	🎯
ACI-RV-02 `rvAciSetManagerRevertsIff` Replacing the manager fails exactly when the caller is not the owner or the new manager is zero.	✅	🎯
ACI-RV-03 `rvAciApplyIrmConfigRevertsWhenNotAuthorized` Applying a rate configuration to a market fails for anyone but the owner or the manager.	✅	🎯🎯
ACI-RV-04 `rvAciBorrowRateRevertsWhenNotRouter` Pricing a market and storing its new rate fails for any caller but the router.	✅	🎯
ACI-RV-05 `rvAciBorrowRateRevertsWhenConfigNotSet` Pricing a market that has no rate configuration fails.	✅	🎯
ACI-RV-06 `rvAciBorrowRateViewRevertsWhenConfigNotSet` Previewing a rate for a market that has no configuration fails.	✅	🎯
ACI-RV-07 `rvAciBorrowRateViewRevertsOnFutureLastUpdate` Previewing a rate for a priced market dated in the future fails.	✅	🎯

Access Control

Properties	Status	Mutations
ACI-AC-01 `acAciOnlyOwnerMovesOwner` Only the current owner can hand over ownership.	✅	🎯
ACI-AC-02 `acAciOnlyOwnerMovesManager` Only the owner can replace the rate model's manager.	✅	🎯
ACI-AC-03 `acAciOnlyAuthorizedMovesMarketConfig` Only the owner or the manager can change a market's rate configuration.	✅	🎯
ACI-AC-04 `acAciOnlyRouterOrAuthorizedMovesRateAtTarget` Only the router, the owner or the manager can change a market's target rate.	✅	🎯
ACI-AC-05 `acAciRouterImmutable` The router address the rate model was deployed with never changes.	✅	🎯

LinearKinkIrm

- The rate model running its real bytecode; every kink config linked as a CVL model
- Kink configs answered by a model held inside the conditions the config constructor enforces, so an applied config is assumed to be one its constructor validated

- WAD multiply and divide computed over unbounded integers, so their overflow reverts are uncovered

Valid State

Properties	Status	Mutations
LKI-VS-01 `vsLkiKinkBelowFullUtilization` A market's kink always sits strictly below full utilization.	✅	🎯
LKI-VS-02 `vsLkiPeakRateWithinCap` A market's peak rate, charged at full utilization, never exceeds 800 percent per year.	✅	🎯
LKI-VS-03 `vsLkiUnsetRowCarriesNoParams` A market with no curve set carries no rate parameters at all.	✅	🎯
LKI-VS-04 `vsLkiRolesAreLive` The rate model always has a live owner and a live manager.	✅	🎯
LKI-VS-05 `vsLkiZeroIdRowStaysEmpty` The zero market id never carries curve parameters.	✅	🎯

Variable Transitions

Properties	Status	Mutations
LKI-VT-01 `vtLkiRolesNeverClear` An owner or manager address, once set, is never cleared, though it can be rotated.	✅	🎯
LKI-VT-02 `vtLkiConfiguredKinkNeverClears` A market keeps its curve as long as every config reports a positive kink.	✅	🎯

State Transitions

Properties	Status	Mutations
LKI-ST-01 `stLkiKinkRowWriteIsAConfiguredTuple` Any change to a market's curve leaves that market configured, never cleared in place.	✅	🎯

High Level

Properties	Status	Mutations
LKI-HL-01 `hlLkiRateMonotoneInUtilization` A market borrowed more heavily is never charged a lower interest rate.	✅	🎯
LKI-HL-02 `hlLkiRateWithinBaseAndPeak` The charged rate never falls below a market's base rate nor passes the 800 percent per year cap.	✅	🎯
LKI-HL-03 `hlLkiRateContinuousAtKink` The interest curve has no jump where its flat and steep legs meet.	✅	🎯🎯
LKI-HL-04 `hlLkiRateEqualsView` A market is charged exactly the rate the public view quotes for it.	✅	🎯
LKI-HL-05 `hlLkiRateDependsOnlyOnTheStoredRow`		

Two markets carrying the same stored curve are charged the same rate for the same state.	✅	🎯

Unit Tests

Properties	Status	Mutations
LKI-UT-01 `utLkiTransferOwnershipInstallsOwner` Handing over ownership installs the named successor and leaves the manager seat alone.	✅	🎯
LKI-UT-02 `utLkiSetManagerInstallsManager` Rotating the manager installs the named manager and leaves ownership alone.	✅	🎯
LKI-UT-03 `utLkiApplyIrmConfigLeavesRolesUntouched` Applying a curve to a market moves neither the owner seat nor the manager seat.	✅	🎯
LKI-UT-04 `utLkiApplyIrmConfigTouchesOnlyItsOwnMarket` Applying a curve to one market leaves every other market's curve untouched.	✅	🎯
LKI-UT-05 `utLkiBorrowRateViewIsCallerAgnostic` The quoted borrow rate is the same for every caller at the same moment.	✅	🎯
LKI-UT-06 `utLkiRateSaturatesAtBothEndsOfTheCurve` An idle market is charged its base rate and a fully borrowed one the peak rate.	✅	🎯
LKI-UT-07 `utLkiApplyIrmConfigCopiesAllFourFields` Configuring a fresh market copies all four rate parameters from the curve it was given.	✅	🎯🎯🎯
LKI-UT-08 `utLkiApplyIrmConfigOverwritesEveryField` Reconfiguring a live market replaces all four curve parameters, leaving nothing of the old curve.	✅	🎯🎯

Config Trust Boundary

Properties	Status	Mutations
LKI-TB-01 `rcLkiZeroKinkConfigDeconfiguresALiveMarket` Applying a curve whose kink is zero can strip a live market of its rate and stop its interest.	✅	🎯
LKI-TB-02 `rcLkiOverCapPeakRateIsStorable` Applying a curve can give a market a peak rate above the 800 percent per year cap.	✅	🎯

Reachability

Properties	Status	Mutations
LKI-RC-01 `rcLkiBelowKinkRateIsReachable` It is possible to price an idle configured market at exactly its base rate.	✅	🎯
LKI-RC-02 `rcLkiPeakRateReachesTheCap` It is possible for a fully borrowed market to be charged exactly the rate cap.	✅	🎯
LKI-RC-03 `rcLkiBorrowRateViewIsReachable` It is possible to quote a configured market through the public view and get a positive rate.	✅	🎯
LKI-RC-04 `rcLkiAboveWadUtilizationIsClamped` A market claiming more borrowed than supplied can still be priced, at the curve's peak.	✅	🎯

LKI-RC-05 `rcLkiApplyIrmConfigOnFreshMarketIsReachable` The owner or the manager can give a curve to a market that has none.	✅	🎯
LKI-RC-06 `rcLkiApplyIrmConfigReconfiguresIsReachable` The owner or the manager can move a configured market onto a different curve.	✅	🎯

Reverts

Properties	Status	Mutations
LKI-RV-01 `rvLkiTransferOwnershipRevertsIff` Handing over ownership fails exactly when the caller is not the owner or the new owner is zero.	✅	🎯
LKI-RV-02 `rvLkiSetManagerRevertsIff` Rotating the manager fails exactly when the caller is not the owner or the new manager is zero.	✅	🎯
LKI-RV-03 `rvLkiApplyIrmConfigRevertsWhenNotAuthorized` Applying a curve to a market fails for a caller who is neither the owner nor the manager.	✅	🎯
LKI-RV-04 `rvLkiBorrowRateRevertsIffConfigNotSet` Pricing a market for interest accrual fails exactly when that market has no curve set.	✅	🎯
LKI-RV-05 `rvLkiBorrowRateViewRevertsIffConfigNotSet` The public rate quote fails exactly when a market has no curve set.	✅	🎯

Access Control

Properties	Status	Mutations
LKI-AC-01 `acLkiOnlyOwnerModifiesOwner` Only the current owner can hand over ownership.	✅	🎯
LKI-AC-02 `acLkiOnlyOwnerModifiesManager` Only the owner can replace the rate model's manager.	✅	🎯
LKI-AC-03 `acLkiOnlyAuthorizedModifiesOptimalUtilization` Only the owner or the manager can move a market's kink, where its rate curve steepens.	✅	🎯
LKI-AC-04 `acLkiOnlyAuthorizedModifiesBaseRate` Only the owner or the manager can change a market's base rate.	✅	🎯
LKI-AC-05 `acLkiOnlyAuthorizedModifiesSlope1` Only the owner or the manager can change a market's slope below the kink.	✅	🎯
LKI-AC-06 `acLkiOnlyAuthorizedModifiesSlope2` Only the owner or the manager can change a market's steeper slope above the kink.	✅	🎯

IrmRouter

- The router running its real bytecode; the core and the routed rate model answered by free draws

Valid State

Properties	Status	Mutations
IRR-VS-01 `vsIrrZeroIdNeverRouted` The zero market id never carries a rate model.	✅	🎯

Variable Transitions

Properties	Status	Mutations
IRR-VT-01 `vtIrrRoutedMarketNeverUnroutes` A market that has a rate model never loses it, though the model can be switched.	✅	🎯
IRR-VT-02 `vtIrrRolesNeverClear` An owner or manager address, once set, is never cleared, though it can be rotated.	✅	🎯
IRR-VT-03 `vtIrrInitializationLatchIsOneWay` Once the router is initialized, nothing can turn that state back off.	✅	🎯🎯

Unit Tests

Properties	Status	Mutations
IRR-UT-01 `utIrrTransferOwnershipInstallsOwner` Handing over ownership installs the named successor and leaves the manager untouched.	✅	🎯
IRR-UT-02 `utIrrSetManagerInstallsManager` Replacing the manager installs the named address and leaves ownership untouched.	✅	🎯
IRR-UT-03 `utIrrSetMarketIrmRoutesTheMarket` Routing a market records exactly the rate model supplied and changes neither role.	✅	🎯🎯
IRR-UT-04 `utIrrSetMarketIrmTouchesOnlyItsOwnMarket` Routing one market leaves every other market's rate model untouched.	✅	🎯
IRR-UT-05 `utIrrBorrowRateRoutesAndWritesNothing` Pricing a market through the router changes nothing in the router.	✅	🎯🎯

Reachability

Properties	Status	Mutations
IRR-RC-01 `rcIrrBorrowRateIsReachable` Morpho can get a live borrow rate for a market that has a rate model.	✅	🎯
IRR-RC-02 `rcIrrBorrowRateViewIsReachable` An outsider holding no role can quote the borrow rate of a routed market.	✅	🎯
IRR-RC-03 `rcIrrSetMarketIrmFreshIsReachable` The owner or the manager can give a market its first rate model.	✅	🎯
IRR-RC-04 `rcIrrSetMarketIrmRoutingChainIsReachable` Two different markets can be routed one after the other, each to its own rate model.	✅	🎯
IRR-RC-05 `rcIrrInitializeIsReachable` It is possible to initialize a fresh router, installing the owner and the manager and locking initialization.	✅	🎯

Reverts

Properties	Status	Mutations
IRR-RV-01 `rvIrrInitializeAlwaysReverts` The router can be initialized only once.	✅	🎯
IRR-RV-02 `rvIrrSetMarketIrmRevertsOnGuardFailure` Routing a market fails for an unauthorized caller, a zero rate model, or a market that names another router.	✅	🎯🎯
IRR-RV-03 `rvIrrBorrowRateRevertsOnGuardFailure` Asking for a live rate fails for any caller other than Morpho and for a market with no rate model.	✅	🎯
IRR-RV-04 `rvIrrBorrowRateViewRevertsWhenIrmNotSet` Quoting a market that has no rate model fails.	✅	🎯🎯
IRR-RV-05 `rvIrrTransferOwnershipRevertsIff` Handing over ownership fails exactly when the caller is not the owner or the new owner is zero.	✅	🎯
IRR-RV-06 `rvIrrSetManagerRevertsIff` Replacing the manager fails exactly when the caller is not the owner or the new manager is zero.	✅	🎯

Access Control

Properties	Status	Mutations
IRR-AC-01 `acIrrOnlyOwnerModifiesOwner` Only the current owner can hand over ownership.	✅	🎯
IRR-AC-02 `acIrrOnlyOwnerModifiesManager` Only the owner can replace the router's manager.	✅	🎯
IRR-AC-03 `acIrrOnlyAuthorizedModifiesMarketIrm` Only the owner or the manager can point a market at a rate model.	✅	🎯
IRR-AC-04 `acIrrMorphoImmutable` The Morpho address the router trusts is fixed at deployment and never changes.	✅	🎯

IrmConfigFactory

- The factory running its real bytecode with both config templates compiled in beside it
- Config deployment unrolled once per run, twice where a rule compares a fresh deploy with a repeat
- The three trust boundary rules running over the two rate model scenes with the constructor bands dropped: witnesses of what an unvalidated config buys, not defects

Valid State

Properties	Status	Mutations
ICF-VS-01 `vsIcfZeroHashNeverResolvesToAnAdaptiveConfig` The empty parameter key never maps to an adaptive config.	✅	🎯🎯

ICF-VS-02 `vsIcfZeroHashNeverResolvesToALinearConfig` The empty parameter key never maps to a linear kink config.	✅	🎯

Variable Transitions

Properties	Status	Mutations
ICF-VT-01 `vtIcfAdaptiveRegistryIsWriteOnce` A registered adaptive config is never replaced and never cleared.	✅	🎯
ICF-VT-02 `vtIcfLinearRegistryIsWriteOnce` A registered linear kink config is never replaced and never cleared.	✅	🎯

High Level

Properties	Status	Mutations
ICF-HL-01 `hlIcfSameParamsResolveToOneAdaptiveConfig` Requesting the same adaptive parameters twice returns the same config and deploys nothing new.	✅	🎯 🎯
ICF-HL-02 `hlIcfSameParamsResolveToOneLinearConfig` Requesting the same linear kink parameters twice returns the same config and deploys nothing new.	✅	🎯
ICF-HL-03 `hlIcfDistinctParamsResolveToDistinctConfigs` Adaptive parameter sets that differ in any field always resolve to different configs.	✅	🎯

Unit Tests

Properties	Status	Mutations
ICF-UT-01 `utIcfGetOrDeployAdaptiveRegistersTheReturnedConfig` Requesting an adaptive config returns a real address that the registry then holds for those parameters.	✅	🎯
ICF-UT-02 `utIcfGetOrDeployLinearRegistersTheReturnedConfig` Requesting a linear kink config returns a real address that the registry then holds for those parameters.	✅	🎯
ICF-UT-03 `utIcfGetAdaptiveConfigMatchesTheRegistry` Looking up a registered adaptive config reports the same address a new request would return.	✅	🎯
ICF-UT-04 `utIcfGetLinearConfigMatchesTheRegistry` Looking up a registered linear kink config reports the same address a new request would return.	✅	🎯
ICF-UT-05 `utIcfAdaptiveDeployLeavesTheLinearRegistryUntouched` Deploying an adaptive config changes nothing in the linear kink registry.	✅	🎯
ICF-UT-06 `utIcfDeployedAdaptiveConfigClampsToTheRequestedBand` Applying a fresh adaptive config points the market at it and pulls its target rate into the requested band.	✅	🎯

ICF-UT-07 `utIcfDeployedLinearConfigCopiesTheRequestedParams` The linear kink curve a market prices with is exactly the one the factory was asked for.	✅	🎯
ICF-UT-08 `utIcfAdaptiveConfigGetterMirrorsItsImmutables` The adaptive config reports every curve parameter in the field it belongs to.	✅	🎯
ICF-UT-09 `utIcfLinearConfigGetterMirrorsItsImmutables` The linear kink config reports every parameter in the field it belongs to.	✅	🎯
ICF-UT-10 `utIcfLinearDeployLeavesTheAdaptiveRegistryUntouched` Deploying a linear kink config changes nothing in the adaptive registry.	✅	🎯

Config Trust Boundary

Properties	Status	Mutations
ICF-TB-01 `tbIcfUnvalidatedConfigDeconfiguresALiveKinkMarket` The owner can apply an unvalidated config that unsets a live market and stops its rate quotes.	✅	🎯
ICF-TB-02 `tbIcfUnvalidatedConfigStoresAPeakRateAboveTheRetiredCap` The owner can apply an unvalidated config whose peak borrow rate exceeds the cap the rate model once enforced.	✅	🎯
ICF-TB-03 `tbIcfUnvalidatedConfigClampsRateAtTargetOutsideTheProtocolBand` The owner can apply an unvalidated config that lifts a market's target rate above the protocol wide limit.	✅	🎯

Reachability

Properties	Status	Mutations
ICF-RC-01 `rcIcfFreshAdaptiveDeploymentIsReachable` A caller can deploy and register a config for an adaptive curve nobody has requested yet.	✅	🎯
ICF-RC-02 `rcIcfFreshLinearDeploymentIsReachable` A caller can deploy and register a config for a linear kink curve nobody has requested yet.	✅	🎯
ICF-RC-03 `rcIcfAdaptiveDedupHitIsReachable` A repeat adaptive request can be served from the registry without writing to it again.	✅	🎯
ICF-RC-04 `rcIcfLinearDedupHitIsReachable` A repeat linear kink request can be served from the registry without writing to it again.	✅	🎯

Reverts

Properties	Status	Mutations
ICF-RV-01 `rvIcfGetOrDeployAdaptiveRevertsIffParamsInvalid` Deploying an adaptive config fails exactly when the requested curve is outside the allowed bands.	✅	🎯🎯
ICF-RV-02 `rvIcfGetOrDeployLinearRevertsIffParamsInvalid` Deploying a linear kink config fails exactly when the kink or the peak rate is out of bounds.	✅	🎯🎯🎯🎯🎯🎯

ICF-RV-03 `rvIcfGetOrDeployAdaptiveNeverRevertsOnRegisteredParams` A request for adaptive parameters that already have a config never fails.	✅	🎯
ICF-RV-04 `rvIcfGetOrDeployLinearNeverRevertsOnRegisteredParams` A request for linear kink parameters that already have a config never fails.	✅	🎯

Mutation Testing

A mutant is a deliberate fault planted in the source under `mutations/`. A rule catches it when it fails on the mutated source, and one mutant may be caught by several rules. `./certora/mutations/run_mutation.sh <contract> <id>` plants it, runs the rule and restores the source. Each entry names its fault and shows the diff; the Mutations column of the property tables links to it.

Mutated source	Mutants	Rules that catch them	Entries
GuardianSignals	211	206	`mutations/GuardianSignals/README.md`
LidoLendMarketController	242	231	`mutations/LidoLendMarketController/README.md`
LidoLend	233	231	`mutations/LidoLend/README.md`
LidoLendERC4626	122	133	`mutations/LidoLendERC4626/README.md`
Erc4626ExitBook	109	117	`mutations/Erc4626ExitBook/README.md`
MorphoExitBook	129	145	`mutations/MorphoExitBook/README.md`
AdaptiveCurveIrm	28	30	`mutations/AdaptiveCurveIrm/README.md`
LinearKinkIrm	32	32	`mutations/LinearKinkIrm/README.md`
IrmRouter	19	16	`mutations/IrmRouter/README.md`
IrmConfigFactory	21	19	`mutations/IrmConfigFactory/README.md`
AdaptiveCurveConfig	4	3	`mutations/AdaptiveCurveConfig/README.md`
AdaptiveCurveMathLib	12	14	`mutations/AdaptiveCurveMathLib/README.md`
LinearKinkConfig	8	3	`mutations/LinearKinkConfig/README.md`
OwnerManagerBase	27	27	`mutations/OwnerManagerBase/README.md`

Reproducing the Results

The Certora Prover runs either remotely on Certora's cloud or locally from a build. Both modes share the setup below.

Prerequisites

These steps are for Ubuntu 24.04. A step-by-step walkthrough is in this setup tutorial.

1. Install Java (tested with JDK 21)

```
sudo apt update
sudo apt install default-jre
java -version
```

2. Install pipx, which keeps Python CLI tools in isolated environments

```
sudo apt install pipx
pipx ensurepath
```

3. Install the Certora CLI, pinned to the prover version of this report

```
pipx install certora-cli==8.16.2
```

4. Install solc-select and the compiler this project builds with

```
pipx install solc-select
solc-select install 0.8.36
solc-select use 0.8.36
```

5. Create the versioned symlink the confs name. They reference the compiler as `solc0.8.36` without the dash, while solc-select installs a generic `solc`:

```
mkdir -p ~/.local/bin
ln -sf ~/.solc-select/artifacts/solc-0.8.36/solc-0.8.36 ~/.local/bin/solc0.8.36
```

Check that `~/.local/bin` is on your `PATH`, and add it if not:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

6. From the repository root, fetch the library sources the contracts import. Some submodules are recorded with SSH URLs, so the first line rewrites them to HTTPS. The `lido-lend` and `lido-lend-irm` submodules are private lidofinance repositories, so fetching them needs read access:

```
git config url."https://github.com/".insteadOf "git@github.com:"
git submodule sync --recursive
git submodule update --init --recursive
```

Remote Execution

Set up a Certora key, available free through Certora's Discord or their website:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
```

> **Note:** a local prover, if installed, takes priority. To force the cloud, add `--server production`:
>
> ```
> certoraRun certora/confs/GuardianSignals/valid_state.conf --server production
> ```

Local Execution

Follow the build instructions in the CertoraProver repository (v8.16.2). Once built, the local prover takes priority over the cloud by default.

Running the Suites

Run from `lido-lend-markets/`, the directory that holds `src/` and `certora/`. The confs inherit their heap, concurrency and solver budget from `certora/confs/_base.conf`; a conf that needs a different heap or budget sets it in its own file.

The flags below appear in the commands that follow, and each answers one question:

Flag	What it does	When to reach for it
`--rule <name>`	runs one property instead of the conf	a property that did not decide, so it gets the budget rather than sharing it
`--exclude_rule <names>`	runs the conf without those properties	the rest of a conf whose one heavy property is being run separately
`--java_args '-Xmx<N>g'`	sets the JVM heap the scene is held in	a joint scene on a machine with less memory than the confs assume
`--max_concurrent_rules <n>`	caps properties in flight	a run that dies with no verdict: the solver processes live outside the heap and follow the core count

GuardianSignals

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/GuardianSignals/access_control.conf
certoraRun certora/confs/GuardianSignals/access_control_roles.conf
certoraRun certora/confs/GuardianSignals/frontier.conf
certoraRun certora/confs/GuardianSignals/high_level.conf
certoraRun certora/confs/GuardianSignals/reachability.conf
certoraRun certora/confs/GuardianSignals/reverts.conf
certoraRun certora/confs/GuardianSignals/reverts_heartbeat.conf
certoraRun certora/confs/GuardianSignals/transitions.conf
certoraRun certora/confs/GuardianSignals/unit_tests.conf
certoraRun certora/confs/GuardianSignals/valid_state.conf
```

LidoLendMarketController

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/LidoLendMarketController/access_control.conf
certoraRun certora/confs/LidoLendMarketController/access_control_roles.conf
certoraRun certora/confs/LidoLendMarketController/reachability.conf
certoraRun certora/confs/LidoLendMarketController/reentrancy.conf
certoraRun certora/confs/LidoLendMarketController/settlement_slice.conf
certoraRun certora/confs/LidoLendMarketController/transitions.conf
certoraRun certora/confs/LidoLendMarketController/unit_tests.conf
certoraRun certora/confs/LidoLendMarketController/valid_state.conf
```

`folded_high_level.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLendMarketController/folded_high_level.conf \
  --exclude_rule hlMcSettlementQuoteFitsBudget

certoraRun certora/confs/LidoLendMarketController/folded_high_level.conf --rule
hlMcSettlementQuoteFitsBudget
```

`folded_reverts.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendMarketController/folded_reverts.conf \
  --exclude_rule rvMcOnSettleRevertsAboveSettleableSupply
rvMcOnWithdrawRevertsWhenRemainingActiveInsufficient rvMcSettlementForNeverReverts

certoraRun certora/confs/LidoLendMarketController/folded_reverts.conf --rule
rvMcOnSettleRevertsAboveSettleableSupply
certoraRun certora/confs/LidoLendMarketController/folded_reverts.conf --rule
rvMcOnWithdrawRevertsWhenRemainingActiveInsufficient
certoraRun certora/confs/LidoLendMarketController/folded_reverts.conf --rule
rvMcSettlementForNeverReverts
```

`high_level.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLendMarketController/high_level.conf \
  --exclude_rule hlMcSettlementQuoteFitsBudget

certoraRun certora/confs/LidoLendMarketController/high_level.conf --rule hlMcSettlementQuoteFitsBudget
```

`reverts.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendMarketController/reverts.conf \
  --exclude_rule rvMcOnSettleRevertsAboveSettleableSupply
rvMcOnWithdrawRevertsWhenRemainingActiveInsufficient rvMcSettlementForNeverReverts

certoraRun certora/confs/LidoLendMarketController/reverts.conf --rule
rvMcOnSettleRevertsAboveSettleableSupply
certoraRun certora/confs/LidoLendMarketController/reverts.conf --rule
rvMcOnWithdrawRevertsWhenRemainingActiveInsufficient
certoraRun certora/confs/LidoLendMarketController/reverts.conf --rule rvMcSettlementForNeverReverts
```

LidoLend

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/LidoLend/access_control.conf
certoraRun certora/confs/LidoLend/accrue_interest.conf
certoraRun certora/confs/LidoLend/create_market.conf
certoraRun certora/confs/LidoLend/oracle_outage.conf
certoraRun certora/confs/LidoLend/reachability.conf
certoraRun certora/confs/LidoLend/reentrancy.conf
certoraRun certora/confs/LidoLend/reverts.conf
certoraRun certora/confs/LidoLend/unit_tests.conf
certoraRun certora/confs/LidoLend/variable_transitions.conf
```

`high_level.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLend/high_level.conf \
```

```
  --exclude_rule hlLlBorrowDebtValueBounded hlLlBorrowDebtValueBoundedAssetsMode
hlLlBorrowRepayAnyModeNoProfit hlLlBorrowSharePriceOnlyRisesOnAccrual
hlLlLastBorrowLiquidationLeavesOthersHealthy hlLlLiquidateLeavesOthersHealthy
hlLlLiquidateNeverRaisesDebtSharePrice hlLlLiquidateRevertsOnHealthyBorrower
hlLlLiquidateSeizureBoundedByIncentive hlLlLiquidateSocializesAtMostBorrowerDebt hlLlOthersStayHealthy
hlLlRepayCostBounded hlLlSettleBorrowAnchoredLeavesOthersHealthy hlLlSupplyBookValueBounded

certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlBorrowDebtValueBounded
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlBorrowDebtValueBoundedAssetsMode
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlBorrowRepayAnyModeNoProfit
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlBorrowSharePriceOnlyRisesOnAccrual
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlLastBorrowLiquidationLeavesOthersHealthy
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlLiquidateLeavesOthersHealthy
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlLiquidateNeverRaisesDebtSharePrice
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlLiquidateRevertsOnHealthyBorrower
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlLiquidateSeizureBoundedByIncentive
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlLiquidateSocializesAtMostBorrowerDebt
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlOthersStayHealthy
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlRepayCostBounded
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlSettleBorrowAnchoredLeavesOthersHealthy
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlSupplyBookValueBounded
```

`perf_s20_s25.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLend/perf_s20_s25.conf \
  --exclude_rule hlLlBorrowDebtValueBounded hlLlBorrowDebtValueBoundedAssetsMode
hlLlBorrowRepayAnyModeNoProfit hlLlBorrowSharePriceOnlyRisesOnAccrual
hlLlLastBorrowLiquidationLeavesOthersHealthy hlLlLiquidateLeavesOthersHealthy
hlLlLiquidateNeverRaisesDebtSharePrice hlLlLiquidateRevertsOnHealthyBorrower
hlLlLiquidateSeizureBoundedByIncentive hlLlLiquidateSocializesAtMostBorrowerDebt hlLlOthersStayHealthy
hlLlRepayCostBounded hlLlSettleBorrowAnchoredLeavesOthersHealthy hlLlSupplyBookValueBounded

certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlBorrowDebtValueBounded
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlBorrowDebtValueBoundedAssetsMode
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlBorrowRepayAnyModeNoProfit
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlBorrowSharePriceOnlyRisesOnAccrual
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule
hlLlLastBorrowLiquidationLeavesOthersHealthy
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlLiquidateLeavesOthersHealthy
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlLiquidateNeverRaisesDebtSharePrice
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlLiquidateRevertsOnHealthyBorrower
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlLiquidateSeizureBoundedByIncentive
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlLiquidateSocializesAtMostBorrowerDebt
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlOthersStayHealthy
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlRepayCostBounded
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlSettleBorrowAnchoredLeavesOthersHealthy
certoraRun certora/confs/LidoLend/perf_s20_s25.conf --rule hlLlSupplyBookValueBounded
```

`state_transitions.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLend/state_transitions.conf \
  --exclude_rule stLlSupplyShrinkIsAccounted

certoraRun certora/confs/LidoLend/state_transitions.conf --rule stLlSupplyShrinkIsAccounted
```

`valid_state.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLend/valid_state.conf \
  --exclude_rule tokenBackingSolvency

certoraRun certora/confs/LidoLend/valid_state.conf --rule tokenBackingSolvency
```

LidoLendERC4626

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/LidoLendERC4626/fast_settle.conf
certoraRun certora/confs/LidoLendERC4626/reachability.conf
certoraRun certora/confs/LidoLendERC4626/transitions.conf
certoraRun certora/confs/LidoLendERC4626/valid_state.conf
```

`erc4626.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/erc4626.conf \
  --exclude_rule depositAtAdvertisedMaxDoesNotRevert redeemAtAdvertisedMaxDoesNotRevert
withdrawAtAdvertisedMaxDoesNotRevert

certoraRun certora/confs/LidoLendERC4626/erc4626.conf --rule depositAtAdvertisedMaxDoesNotRevert
certoraRun certora/confs/LidoLendERC4626/erc4626.conf --rule redeemAtAdvertisedMaxDoesNotRevert
certoraRun certora/confs/LidoLendERC4626/erc4626.conf --rule withdrawAtAdvertisedMaxDoesNotRevert
```

`fast_convert.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLendERC4626/fast_convert.conf \
  --exclude_rule hlVaMintRedeemNoProfit

certoraRun certora/confs/LidoLendERC4626/fast_convert.conf --rule hlVaMintRedeemNoProfit
```

`fast_exit.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLendERC4626/fast_exit.conf \
  --exclude_rule hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial

certoraRun certora/confs/LidoLendERC4626/fast_exit.conf --rule
hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
```

`folded_erc4626.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/folded_erc4626.conf \
  --exclude_rule depositAtAdvertisedMaxDoesNotRevert redeemAtAdvertisedMaxDoesNotRevert
withdrawAtAdvertisedMaxDoesNotRevert

certoraRun certora/confs/LidoLendERC4626/folded_erc4626.conf --rule
depositAtAdvertisedMaxDoesNotRevert
certoraRun certora/confs/LidoLendERC4626/folded_erc4626.conf --rule redeemAtAdvertisedMaxDoesNotRevert
certoraRun certora/confs/LidoLendERC4626/folded_erc4626.conf --rule
withdrawAtAdvertisedMaxDoesNotRevert
```

`folded_heavy.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/folded_heavy.conf \
  --exclude_rule hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
hlVaFallbackExitBurnsProRataOfConsumedBudget hlVaMintRedeemNoProfit

certoraRun certora/confs/LidoLendERC4626/folded_heavy.conf --rule
hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
certoraRun certora/confs/LidoLendERC4626/folded_heavy.conf --rule
hlVaFallbackExitBurnsProRataOfConsumedBudget
certoraRun certora/confs/LidoLendERC4626/folded_heavy.conf --rule hlVaMintRedeemNoProfit
```

`folded_reverts.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLendERC4626/folded_reverts.conf \
  --exclude_rule rvVaFallbackExitDoesNotRevertWhenLiquidityCovers

certoraRun certora/confs/LidoLendERC4626/folded_reverts.conf --rule
rvVaFallbackExitDoesNotRevertWhenLiquidityCovers
```

`high_level.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/high_level.conf \
  --exclude_rule hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
hlVaFallbackExitBurnsProRataOfConsumedBudget hlVaMintRedeemNoProfit

certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule
hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule
hlVaFallbackExitBurnsProRataOfConsumedBudget
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule hlVaMintRedeemNoProfit
```

`perf_fold_budget_pricing_gates_sweep_payout.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/perf_fold_budget_pricing_gates_sweep_payout.conf \
  --exclude_rule hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
hlVaFallbackExitBurnsProRataOfConsumedBudget hlVaMintRedeemNoProfit

certoraRun certora/confs/LidoLendERC4626/perf_fold_budget_pricing_gates_sweep_payout.conf --rule
hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
certoraRun certora/confs/LidoLendERC4626/perf_fold_budget_pricing_gates_sweep_payout.conf --rule
hlVaFallbackExitBurnsProRataOfConsumedBudget
certoraRun certora/confs/LidoLendERC4626/perf_fold_budget_pricing_gates_sweep_payout.conf --rule
hlVaMintRedeemNoProfit
```

`perf_fold_pricing_gates_sweep_payout.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/perf_fold_pricing_gates_sweep_payout.conf \
  --exclude_rule hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
hlVaFallbackExitBurnsProRataOfConsumedBudget hlVaMintRedeemNoProfit

certoraRun certora/confs/LidoLendERC4626/perf_fold_pricing_gates_sweep_payout.conf --rule
hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
certoraRun certora/confs/LidoLendERC4626/perf_fold_pricing_gates_sweep_payout.conf --rule
hlVaFallbackExitBurnsProRataOfConsumedBudget
certoraRun certora/confs/LidoLendERC4626/perf_fold_pricing_gates_sweep_payout.conf --rule
hlVaMintRedeemNoProfit
```

`reverts.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLendERC4626/reverts.conf \
  --exclude_rule rvVaFallbackExitDoesNotRevertWhenLiquidityCovers

certoraRun certora/confs/LidoLendERC4626/reverts.conf --rule
rvVaFallbackExitDoesNotRevertWhenLiquidityCovers
```

LidoLend_LinkedController

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/LidoLend_LinkedController/oracle_outage.conf
certoraRun certora/confs/LidoLend_LinkedController/reentrancy.conf
```

`fast_hooks.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLend_LinkedController/fast_hooks.conf \
  --exclude_rule hlLwcSettleStrandsAtMostSubWeiDebt

certoraRun certora/confs/LidoLend_LinkedController/fast_hooks.conf --rule
hlLwcSettleStrandsAtMostSubWeiDebt
```

`folded_properties.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLend_LinkedController/folded_properties.conf \
  --exclude_rule hlLwcBorrowLeavesDebtCoveredByActiveCollateral
hlLwcMaxWithdrawableIsWithdrawableEndToEnd hlLwcSettleStrandsAtMostSubWeiDebt
hlLwcWithdrawCollateralLeavesDebtCoveredByActiveCollateral rcLwcAccrualLiftsFeeRecipientQuote
rcLwcSettleAtRootCutsDebt rcLwcSettleCutsDebtWithIdleLiquidityPresent

certoraRun certora/confs/LidoLend_LinkedController/folded_properties.conf --rule
hlLwcBorrowLeavesDebtCoveredByActiveCollateral
certoraRun certora/confs/LidoLend_LinkedController/folded_properties.conf --rule
hlLwcMaxWithdrawableIsWithdrawableEndToEnd
certoraRun certora/confs/LidoLend_LinkedController/folded_properties.conf --rule
hlLwcSettleStrandsAtMostSubWeiDebt
certoraRun certora/confs/LidoLend_LinkedController/folded_properties.conf --rule
hlLwcWithdrawCollateralLeavesDebtCoveredByActiveCollateral
certoraRun certora/confs/LidoLend_LinkedController/folded_properties.conf --rule
rcLwcAccrualLiftsFeeRecipientQuote
certoraRun certora/confs/LidoLend_LinkedController/folded_properties.conf --rule
rcLwcSettleAtRootCutsDebt
certoraRun certora/confs/LidoLend_LinkedController/folded_properties.conf --rule
rcLwcSettleCutsDebtWithIdleLiquidityPresent
```

`properties.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLend_LinkedController/properties.conf \
  --exclude_rule hlLwcBorrowLeavesDebtCoveredByActiveCollateral
hlLwcMaxWithdrawableIsWithdrawableEndToEnd hlLwcSettleStrandsAtMostSubWeiDebt
hlLwcWithdrawCollateralLeavesDebtCoveredByActiveCollateral rcLwcAccrualLiftsFeeRecipientQuote
rcLwcSettleAtRootCutsDebt rcLwcSettleCutsDebtWithIdleLiquidityPresent

certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule
hlLwcBorrowLeavesDebtCoveredByActiveCollateral
certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule
hlLwcMaxWithdrawableIsWithdrawableEndToEnd
certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule
hlLwcSettleStrandsAtMostSubWeiDebt
certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule
hlLwcWithdrawCollateralLeavesDebtCoveredByActiveCollateral
certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule
rcLwcAccrualLiftsFeeRecipientQuote
certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule rcLwcSettleAtRootCutsDebt
certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule
rcLwcSettleCutsDebtWithIdleLiquidityPresent
```

`valid_state.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLend_LinkedController/valid_state.conf \
  --exclude_rule activationBucketsBackedByCoreCollateral

certoraRun certora/confs/LidoLend_LinkedController/valid_state.conf --rule
activationBucketsBackedByCoreCollateral
```

LidoLendERC4626_LinkedCore

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/LidoLendERC4626_LinkedCore/folded_properties.conf
certoraRun certora/confs/LidoLendERC4626_LinkedCore/oracle_outage.conf
certoraRun certora/confs/LidoLendERC4626_LinkedCore/properties.conf
```

Erc4626ExitBook_LinkedVaultCore

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/Erc4626ExitBook_LinkedVaultCore/properties.conf
certoraRun certora/confs/Erc4626ExitBook_LinkedVaultCore/valid_state.conf
```

MorphoExitBook_LinkedCore

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/MorphoExitBook_LinkedCore/oracle_outage.conf
certoraRun certora/confs/MorphoExitBook_LinkedCore/valid_state.conf
```

`properties.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/MorphoExitBook_LinkedCore/properties.conf \
  --exclude_rule rvMePlaceExitRequestRevertsOnInsufficientBacking

certoraRun certora/confs/MorphoExitBook_LinkedCore/properties.conf --rule
rvMePlaceExitRequestRevertsOnInsufficientBacking
```

AdaptiveCurveIrm

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/AdaptiveCurveIrm/config_trust_boundary.conf
certoraRun certora/confs/AdaptiveCurveIrm/valid_state.conf
```

`properties.conf` holds rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/AdaptiveCurveIrm/properties.conf \
  --exclude_rule hlAciBorrowRateMonotoneInUtilization
hlAciBorrowRateMonotoneInUtilizationWithoutAdaptation hlAciBorrowRateWithinAmplifiedConfigBand

certoraRun certora/confs/AdaptiveCurveIrm/properties.conf --rule hlAciBorrowRateMonotoneInUtilization
certoraRun certora/confs/AdaptiveCurveIrm/properties.conf --rule
hlAciBorrowRateMonotoneInUtilizationWithoutAdaptation
certoraRun certora/confs/AdaptiveCurveIrm/properties.conf --rule
hlAciBorrowRateWithinAmplifiedConfigBand
```

LinearKinkIrm

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/LinearKinkIrm/config_trust_boundary.conf
certoraRun certora/confs/LinearKinkIrm/properties.conf
certoraRun certora/confs/LinearKinkIrm/valid_state.conf
```

IrmRouter

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/IrmRouter/properties.conf
certoraRun certora/confs/IrmRouter/valid_state.conf
```

IrmConfigFactory

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/IrmConfigFactory/config_trust_boundary.conf
certoraRun certora/confs/IrmConfigFactory/properties.conf
certoraRun certora/confs/IrmConfigFactory/round_trip.conf
certoraRun certora/confs/IrmConfigFactory/valid_state.conf
```

Running on a smaller machine

```
certoraRun certora/confs/LidoLend_LinkedController/properties.conf \
  --java_args '-Xmx24g' --max_concurrent_rules 2
```

Resources

- Certora Tutorials: official Certora documentation and guided tutorials
- AlexZoid personal site and FV resources: write-ups on verifying real protocols, alongside a curated collection of specs, examples and references
- Updraft Assembly & Formal Verification Course: comprehensive video course covering assembly and formal verification from the ground up
- Find Highs Using Certora Formal Verification: practical guide with a companion repo of simplified examples based on real code and bugs from private audits
- RareSkills Certora Book: structured tutorial covering CVL syntax, patterns, and common pitfalls